Finding Replication Origins in Bacterial Genomes

Algorithmic Warmup

Phillip Compeau

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Outline

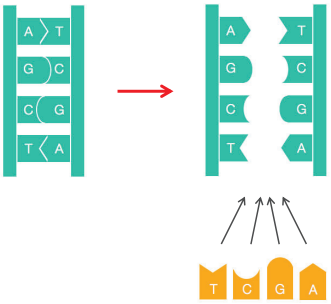
- **An Intro to DNA Replication**
- Hidden Messages in the Replication Origin
- Hunting for Frequent Words
- A Faster Frequent Words Approach
- Some Hidden Messages are More Surprising than Others
- An Explosion of Hidden Messages
- Replication Asymmetry Leads Us to the Replication Origin

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

The "Copying Mechanism"

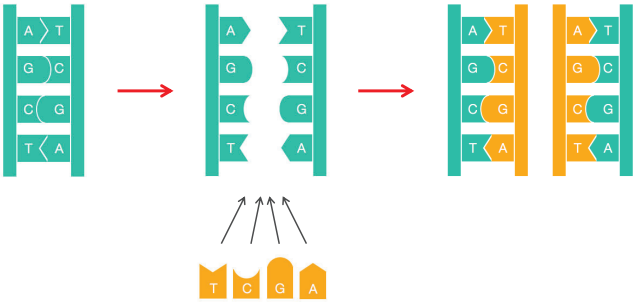
Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

The "Copying Mechanism"



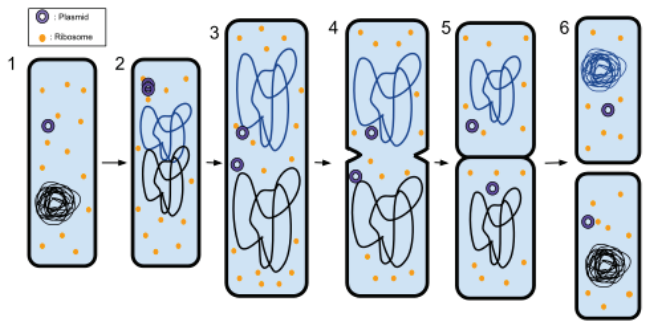
Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

The "Copying Mechanism"



Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

What a Biologist Sees...



Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

What a Computer Scientist Sees...

String: a contiguous collection of symbols.

What a Computer Scientist Sees...

String: a contiguous collection of symbols.

...ACTGATAACCCAGTATCAGACCAGTATCGAGGACGATACGTA...

DNA String

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

What a Computer Scientist Sees...

String: a contiguous collection of symbols.

...ACTGATAACCCAGTATCAGACCAGTATCGAGGACGATACGTA...

DNA String

↓

Complicated Biological Process

What a Computer Scientist Sees...

String: a contiguous collection of symbols.

...ACTGATAACCCAGTATCAGACCAGTATCGAGGACGATACGTA...

DNA String

↓

Complicated Biological Process

↓

Copy 1

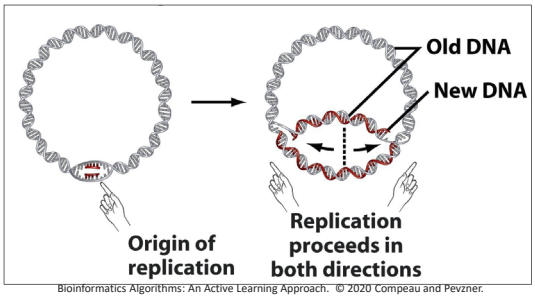
...ACTGATAACCCAGTATCAGACCAGTATCGAGGACGATACGTA...

...ACTGATAACCCAGTATCAGACCAGTATCGAGGACGATACGTA...

Copy 2

Origin of Replication

Replication begins in a region called the **replication origin** (denoted *ori*).



The Finding *ori* Problem

Origin of Replication Problem

- **Input:** A DNA string *genome*.
- **Output:** The location of *ori* in *genome*.

STOP: Is the Hidden Message Problem a computational problem?

Finding the Origin of Replication

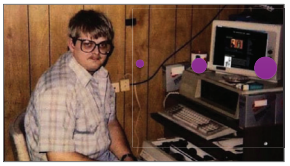
How can we find *ori* in a genome?

Let's hack out this DNA fragment. Can the genome replicate without it?

Finding the Origin of Replication

How can we find *ori* in a genome?

Let's hack out this DNA fragment. Can the genome replicate without it?



I need more information before I can hack this problem.

Looking for *ori*

Verified *ori* of *Vibrio cholerae*, the bacterium that causes cholera (~500 nucleotides):

```
atcaatgatcaacgtaagcttctaagcatgatcaaggtgctcacacagtttatccacaac
ctgagtggatgacatcaagataggtcgttgtatctccttcctctcgtactctcatgacca
cggaaagatgatcaagagaggatgatttcttggccatatcgcaatgaatacttgtgactt
gtgcttccaattgacatcttcagcgccatattgcgctggccaaggtgacggagcgggatt
acgaaagcatgatcatggctgttgttctgtttatcttgttttgactgagacttgttagga
tagacggtttttcatcactgactagccaaagccttactctgcctgacatcgaccgtaaat
tgataatgaatttacatgcttccgcgacgatttacctcttgatcatcgatccgattgaag
atcttcaattgttaattctcttgcctcgactcatagccatgatgagctcttgatcatgtt
tccttaaccctctatttttttacggaagaatgatcaagctgctgctcttgatcatcgtttc
```


Looking for *ori*

Verified *ori* of *Vibrio cholerae*, the bacterium that causes cholera (~500 nucleotides):

```
atcaatgatcaacgtaagcttctaagcatgatcaaggtgctcacacagtttatccacaac
ctgagtggatgacatcaagataggtcgttgtatctccttcctctcgtactctcatgacca
cggaaagatgatcaagagaggatgatttcttggccatatcgcaatgaatacttgtgactt
gtgcttccaattgacatcttcagcgccatattgcgctggccaaggtgacggagcgggatt
acgaaagcatgatcatggctgttgttctgtttatcttgttttgactgagacttgttagga
tagacggtttttcatcactgactagccaaagccttactctgcctgacatcgaccgtaaat
tgataatgaatttacatgcttccgcgacgatttacctcttgatcatcgatccgattgaag
atcttcaattgttaattctcttgcctcgactcatagccatgatgagctcttgatcatgtt
tccttaaccctctatttttttacggaagaatgatcaagctgctgctcttgatcatcgtttc
```

There must be a ***hidden message*** telling the cell to start replication here.

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

The Hidden Message Problem

Hidden Message Problem
- **Input:** A string *text* (representing *ori*).
- **Output:** A hidden message in *text*.

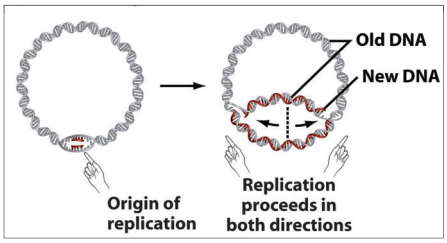
The Hidden Message Problem

Hidden Message Problem
- **Input:** A string *text* (representing *ori*).
- **Output:** A hidden message in *text*.

STOP: Is the Hidden Message Problem a computational problem?

We Have Two Scientific Problems

1. Given a bacterial genome (~3 Mbp), where is *ori*?



2. Given *ori* (~500 bp), what is the "hidden message" saying that replication should start here?

Outline

- An Intro to DNA Replication
- **Hidden Messages in the Replication Origin**
- Hunting for Frequent Words
- A Faster Frequent Words Approach
- Some Hidden Messages are More Surprising than Others
- An Explosion of Hidden Messages
- Replication Asymmetry Leads Us to the Replication Origin

Hidden Message Problem Revisited

Hidden Message Problem
- **Input:** A string *text* (representing *ori*).
- **Output:** A hidden message in *text*.

The notion of "**hidden message**" is not defined.

Hidden Message Problem Revisited

Hidden Message Problem
- **Input:** A string *text* (representing *ori*).
- **Output:** A hidden message in *text*.

Replication initiation is mediated by a protein called ***DnaA*.**

Hidden Message Problem Revisited

Hidden Message Problem
- **Input:** A string *text* (representing *ori*).
- **Output:** A hidden message in *text*.

Replication initiation is mediated by a protein called ***DnaA*.**

DnaA binds to a short segment in *ori* known as a ***DnaA* box**, a hidden message saying: "*bind here!*"

Hidden Message Problem Revisited

Hidden Message Problem
- **Input:** A string *text* (representing *ori*).
- **Output:** A hidden message in *text*.

Replication initiation is mediated by a protein called ***DnaA*.**

DnaA binds to a short segment in *ori* known as a ***DnaA* box**, a hidden message saying: "*bind here!*"

Hidden Message Problem Revisited

STOP: Would it make sense for an organism to have multiple *DnaA* boxes, or just one?

Replication initiation is mediated by a protein called ***DnaA***.

DnaA binds to a short segment in *ori* known as a ***DnaA box***, a hidden message saying: "*bind here!*"

Outline

- An Intro to DNA Replication
- Hidden Messages in the Replication Origin
- **Hunting for Frequent Words**
- A Faster Frequent Words Approach
- Some Hidden Messages are More Surprising than Others
- An Explosion of Hidden Messages
- Replication Asymmetry Leads Us to the Replication Origin

Counting Words

```
atcaatgatcaacgtaagcttctaagcatgatcaaggtgctcacacagtttatccacaac
ctgagtggatgacatcaagataggtcgttgtatctccttcctctcgtactctcatgacca
cggaaagatgatcaagagaggatgatttcttggccatatcgcaatgaatacttgtgactt
gtgcttccaattgacatcttcagcgccatattgcgctggccaaggtgacggagcgggatt
acgaaagcatgatcatggctgttgttctgtttatcttgttttgactgagacttgttagga
tagacggtttttcatcactgactagccaaagccttactctgcctgacatcgaccgtaaat
tgataatgaatttacatgcttccgcgacgatttacctcttgatcatcgatccgattgaag
atcttcaattgttaattctcttgcctcgactcatagccatgatgagctcttgatcatgtt
tccttaaccctctatttttacggaagaatgatcaagctgctgctcttgatcatcgtttc
```

We are looking for surprisingly frequent **substrings** (contiguous strings appearing within) this *ori*.

Counting Words


```
atcaatgatcaacgtaagcttctaagcatgatcaaggtgctcacacagtttatccacaac
ctgagtggatgacatcaagataggtcgttgtatctccttcctctcgtactctcatgacca
cggaaagatgatcaagagaggatgatttcttggccatatcgcaatgaatacttgtgactt
gtgcttccaattgacatcttcagcgccatattgcgctggccaaggtgacggagcgggatt
acgaaagcatgatcatggctgttgttctgtttatcttgttttgactgagacttgttagga
tagacggtttttcatcactgactagccaaagccttactctgcctgacatcgaccgtaaat
tgataatgaatttacatgcttccgcgacgatttacctcttgatcatcgatccgattgaag
atcttcaattgttaattctcttgcctcgactcatagccatgatgagctcttgatcatgtt
tccttaaccctctatttttacggaagaatgatcaagctgctgctcttgatcatcgtttc
```

We are looking for surprisingly frequent **substrings** (contiguous strings appearing within) this *ori*.

First: let's count how often a *given* substring occurs.

Counting Words Problem

Substring Counting Problem
- **Input:** A string *pattern* and a longer string *text*.
- **Output:** The number of times *pattern* occurs in *text*.

Counting Words Problem

Substring Counting Problem
- **Input:** A string *pattern* and a longer string *text*.
- **Output:** The number of times *pattern* occurs in *text*.

STOP: How many times does ATA occur in CGATATATCCATAG?

Counting Words Problem

Substring Counting Problem
- **Input:** A string *pattern* and a longer string *text*.
- **Output:** The number of times *pattern* occurs in *text*.

STOP: How many times does ATA occur in CGATATATCCATAG?

Answer: It can be 2 or 3. For this application, we will go with 3; that is, we count overlaps.

Substring Indexing

Key Point: We think of a string as just an array of symbols. (So it should be 0-indexed.)

Substring Indexing

Key Point: We think of a string as just an array of symbols. (So it should be 0-indexed.)

	0	1	2	3	4	5	6	7	8	9	10	11	12	13
text	C	G	A	T	A	T	A	T	C	C	A	T	A	G

Substring Indexing

Key Point: We think of a string as just an array of symbols. (So it should be 0-indexed.)

	0	1	2	3	4	5	6	7	8	9	10	11	12	13
text	C	G	A	T	A	T	A	T	C	C	A	T	A	G

The notation we use for this substring of *text* is:

text[7, 10]

Substring Indexing

Key Point: We think of a string as just an array of symbols. (So it should be 0-indexed.)

	0	1	2	3	4	5	6	7	8	9	10	11	12	13
text	C	G	A	T	A	T	A	T	C	C	A	T	A	G

STOP: How would we refer to this substring?

Substring Indexing

Key Point: We think of a string as just an array of symbols. (So it should be 0-indexed.)

	0	1	2	3	4	5	6	7	8	9	10	11	12	13
text	C	G	A	T	A	T	A	T	C	C	A	T	A	G

STOP: How would we refer to this substring?

Answer: *text*[0, 3]

Substring Indexing

Key Point: We think of a string as just an array of symbols. (So it should be 0-indexed.)

	0	1	2	3	4	5	6	7	8	9	10	11	12	13
text	C	G	A	T	A	T	A	T	C	C	A	T	A	G

STOP: What about this substring?

Answer: *text*[3, 6]

Substring Indexing

Key Point: We think of a string as just an array of symbols. (So it should be 0-indexed.)

	0	1	2	3	4	5	6	7	8	9	10	11	12	13
text	C	G	A	T	A	T	A	T	C	C	A	T	A	G

STOP: What do you notice?

Substring Indexing

Key Point: We think of a string as just an array of symbols. (So it should be 0-indexed.)

	0	1	2	3	4	5	6	7	8	9	10	11	12	13
text	C	G	A	T	A	T	A	T	C	C	A	T	A	G

STOP: What do you notice?

Answer: We can easily get the *length* of the substring by subtracting the lower index from the upper index. (Here, 6 - 3 = 3.)

Substring Indexing

Key Point: We think of a string as just an array of symbols. (So it should be 0-indexed.)

	0	1	2	3	4	5	6	7	8	9	10	11	12	13
text	C	G	A	T	A	T	A	T	C	C	A	T	A	G

STOP: How would we refer to the substring of *text* of length *k* starting at position *i*?

Substring Indexing

Key Point: We think of a string as just an array of symbols. (So it should be 0-indexed.)

	0	1	2	3	4	5	6	7	8	9	10	11	12	13
text	C	G	A	T	A	T	A	T	C	C	A	T	A	G

STOP: How would we refer to the substring of *text* of length *k* starting at position *i*?

Answer: *text*[*i*, *i*+*k*]. This will be very useful!

Substring Indexing

Key Point: We think of a string as just an array of symbols. (So it should be 0-indexed.)

	0	1	2	3	4	5	6	7	8	9	10	11	12	13
text	C	G	A	T	A	T	A	T	C	C	A	T	A	G

Note: We use the same notation for "subarrays" if we want to refer to a contiguous collection of values in an array.

Our Idea for Counting Patterns

	0	1	2	3	4	5	6	7	8	9	10	11	12	13
count = 0	C	G	A	T	A	T	A	T	C	C	A	T	A	G
count = 0	C	G	A	T	A	T	A	T	C	C	A	T	A	G
count = 1	C	G	A	T	A	T	A	T	C	C	A	T	A	G
count = 1	C	G	A	T	A	T	A	T	C	C	A	T	A	G
count = 2	C	G	A	T	A	T	A	T	C	C	A	T	A	G
count = 2	C	G	A	T	A	T	A	T	C	C	A	T	A	G
count = 2	C	G	A	T	A	T	A	T	C	C	A	T	A	G
count = 2	C	G	A	T	A	T	A	T	C	C	A	T	A	G
count = 2	C	G	A	T	A	T	A	T	C	C	A	T	A	G
count = 2	C	G	A	T	A	T	A	T	C	C	A	T	A	G
count = 3	C	G	A	T	A	T	A	T	C	C	A	T	A	G
count = 3	C	G	A	T	A	T	A	T	C	C	A	T	A	G

Our Idea for Counting Patterns

STOP: How many substrings of length *k* are there in a string of length *n*?

	0	1	2	3	4	5	6	7	8	9	10	11	12	13
count = 0	C	G	A	T	A	T	A	T	C	C	A	T	A	G
count = 0	C	G	A	T	A	T	A	T	C	C	A	T	A	G
count = 1	C	G	A	T	A	T	A	T	C	C	A	T	A	G
count = 1	C	G	A	T	A	T	A	T	C	C	A	T	A	G
count = 2	C	G	A	T	A	T	A	T	C	C	A	T	A	G
count = 2	C	G	A	T	A	T	A	T	C	C	A	T	A	G
count = 2	C	G	A	T	A	T	A	T	C	C	A	T	A	G
count = 2	C	G	A	T	A	T	A	T	C	C	A	T	A	G
count = 2	C	G	A	T	A	T	A	T	C	C	A	T	A	G
count = 2	C	G	A	T	A	T	A	T	C	C	A	T	A	G
count = 3	C	G	A	T	A	T	A	T	C	C	A	T	A	G
count = 3	C	G	A	T	A	T	A	T	C	C	A	T	A	G

Pattern Counting

```
PatternCount(pattern, text)
    count ← 0
    k ← len(pattern)
    n ← len(text)
    for every integer i between 0 and n – k
        if text[i, i+k] = pattern
            count ← count + 1
    return count
```

len(): A (typically built-in) function determining the length (number of symbols) in a string; also works for counting elements in an array.

The Frequent Words Problem

***k*-mer:** A string of length *k*.

The Frequent Words Problem

***k*-mer:** A string of length *k*.

A *k*-mer *pattern* is a **most frequent *k*-mer** in a string if no other *k*-mer is more frequent than *pattern.*

The Frequent Words Problem

***k*-mer:** A string of length *k*.

A *k*-mer *pattern* is a **most frequent *k*-mer** in a string if no other *k*-mer is more frequent than *pattern.*

Frequent Words Problem
- **Input:** A string *text* and an integer *k*.
- **Output:** All **most frequent *k*-mers** in *text*.

The Frequent Words Problem

***k*-mer:** A string of length *k*.

A *k*-mer *pattern* is a **most frequent *k*-mer** in a string if no other *k*-mer is more frequent than *pattern.*

Frequent Words Problem
- **Input:** A string *text* and an integer *k*.
- **Output:** All most frequent *k*-mers in *text*.

STOP: Now is this problem clearly stated?

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Solving the Frequent Words Problem

Frequent Words Problem
- **Input:** A string *text* and an integer *k*.
- **Output:** All most frequent *k*-mers in *text*.

```
atcaatgatcaacgtaagcttctaagcatgatcaaggtgctcacacagtttatccacaac
ctgagtggatgacatcaagataggtcgttgtatctccttcctctcgtactctcatgacca
cggaaagatgatcaagagaggatgatttcttggccatatcgcaatgaatacttgtgactt
gtgcttccaattgacatcttcagcgccatattgcgctggccaaggtgacggagcgggatt
acgaaagcatgatcatggctgttgttctgtttatcttgttttgactgagacttgttagga
tagacggtttttcatcactgactagccaaagccttactctgcctgacatcgaccgtaaat
tgataatgaatttacatgcttccgcgacgatttacctcttgatcatcgatccgattgaag
atcttcaattgttaattctcttgcctcgactcatagccatgatgagctcttgatcatgtt
tccttaaccctctatttttacggaagaatgatcaagctgctgctcttgatcatcgtttc
```

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Solving the Frequent Words Problem

Frequent Words Problem
- **Input:** A string *text* and an integer *k*.
- **Output:** All most frequent *k*-mers in *text*.

Example: If *text* = ACGTTTCACGTTTTACGG and *k* = 3, then the most frequent words are ACG and TTT (both occur three times).

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Solving the Frequent Words Problem

Frequent Words Problem
- **Input:** A string *text* and an integer *k*.
- **Output:** All most frequent *k*-mers in *text*.

Exercise: How might we solve this problem with an array? What subroutines would you find useful?

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

One Frequent Words Solution

1. Create an array *count* of length len(*text*) - *k* + 1.
2. For each *i*, set *count*[*i*] equal to the number of times *text*[*i*, *i*+*k*] appears in *text*.
3. Take *k*-mers having the maximum values of *count*[*i*].

Example: *text* = ACGTTTCACGTTTTACGG and *k* = 3.

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

One Frequent Words Solution

1. Create an array *count* of length len(*text*) - *k* + 1.
2. For each *i*, set *count*[*i*] equal to the number of times *text*[*i*, *i*+*k*] appears in *text*.
3. Take *k*-mers having the maximum values of *count*[*i*].

Example: *text* = ACGTTTCACGTTTTACGG and *k* = 3.

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
count																

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

One Frequent Words Solution

1. Create an array *count* of length len(*text*) - *k* + 1.
2. For each *i*, set *count*[*i*] equal to the number of times *text*[*i*, *i*+*k*] appears in *text*.
3. Take *k*-mers having the maximum values of *count*[*i*].

Example: *text* = ACGTTTCACGTTTTACGG and *k* = 3.

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
count	3															

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

One Frequent Words Solution

1. Create an array *count* of length len(*text*) - *k* + 1.
2. For each *i*, set *count*[*i*] equal to the number of times *text*[*i*, *i*+*k*] appears in *text*.
3. Take *k*-mers having the maximum values of *count*[*i*].

Example: *text* = ACGTTTCACGTTTTACGG and *k* = 3.

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
count	3	2														

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

One Frequent Words Solution

1. Create an array *count* of length len(*text*) - *k* + 1.
2. For each *i*, set *count*[*i*] equal to the number of times *text*[*i*, *i*+*k*] appears in *text*.
3. Take *k*-mers having the maximum values of *count*[*i*].

Example: *text* = ACGTTTCACGTTTTACGG and *k* = 3.

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
count	3	2	2													

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

One Frequent Words Solution

1. Create an array *count* of length len(*text*) - *k* + 1.
2. For each *i*, set *count*[*i*] equal to the number of times *text*[*i*, *i*+*k*] appears in *text*.
3. Take *k*-mers having the maximum values of *count*[*i*].

Example: *text* = ACGTTTCACGTTTTACGG and *k* = 3.

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
count	3	2	2	3												

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

One Frequent Words Solution

1. Create an array *count* of length len(*text*) - *k* + 1.
2. For each *i*, set *count*[*i*] equal to the number of times *text*[*i*, *i*+*k*] appears in *text*.
3. Take *k*-mers having the maximum values of *count*[*i*].

Example: *text* = ACGTTCACGTTTTACGG and *k* = 3.

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
count	3	2	2	3	1											

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

One Frequent Words Solution

1. Create an array *count* of length len(*text*) - *k* + 1.
2. For each *i*, set *count*[*i*] equal to the number of times *text*[*i*, *i*+*k*] appears in *text*.
3. Take *k*-mers having the maximum values of *count*[*i*].

Example: *text* = ACGTTTCACGTTTTACGG and *k* = 3.

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
count	3	2	2	3	1	1										

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

One Frequent Words Solution

1. Create an array *count* of length len(*text*) - *k* + 1.
2. For each *i*, set *count*[*i*] equal to the number of times *text*[*i*, *i*+*k*] appears in *text*.
3. Take *k*-mers having the maximum values of *count*[*i*].

Example: *text* = ACGTTTCACGTTTTACGG and *k* = 3.

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
count	3	2	2	3	1	1	1									

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

One Frequent Words Solution

1. Create an array *count* of length len(*text*) - *k* + 1.
2. For each *i*, set *count*[*i*] equal to the number of times *text*[*i*, *i*+*k*] appears in *text*.
3. Take *k*-mers having the maximum values of *count*[*i*].

Example: *text* = ACGTTTCACGTTTTACGG and *k* = 3.

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
count	3	2	2	3	1	1	1	3	2							

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

One Frequent Words Solution

1. Create an array *count* of length len(*text*) - *k* + 1.
2. For each *i*, set *count*[*i*] equal to the number of times *text*[*i*, *i*+*k*] appears in *text*.
3. Take *k*-mers having the maximum values of *count*[*i*].

Example: *text* = ACGTTTCACGTTTTACGG and *k* = 3.

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
count	3	2	2	3	1	1	1	3	2	2						

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

One Frequent Words Solution

1. Create an array *count* of length len(*text*) - *k* + 1.
2. For each *i*, set *count*[*i*] equal to the number of times *text*[*i*, *i*+*k*] appears in *text*.
3. Take *k*-mers having the maximum values of *count*[*i*].

Example: *text* = ACG**TTT**CACG**TTTT**ACGG and *k* = 3.

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
count	3	2	2	3	1	1	1	3	2	2	3					

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

One Frequent Words Solution

1. Create an array *count* of length len(*text*) - *k* + 1.
2. For each *i*, set *count*[*i*] equal to the number of times *text*[*i*, *i*+*k*] appears in *text*.
3. Take *k*-mers having the maximum values of *count*[*i*].

Example: *text* = ACG**TTT**CACG**TTTT**ACGG and *k* = 3.

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
count	3	2	2	3	1	1	1	3	2	2	3	3				

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

One Frequent Words Solution

1. Create an array *count* of length len(*text*) - *k* + 1.
2. For each *i*, set *count*[*i*] equal to the number of times *text*[*i*, *i*+*k*] appears in *text*.
3. Take *k*-mers having the maximum values of *count*[*i*].

Example: *text* = ACGTTTCACGTT**TTA**CGG and *k* = 3.

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
count	3	2	2	3	1	1	1	3	2	2	3	3	1			

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

One Frequent Words Solution

1. Create an array *count* of length len(*text*) - *k* + 1.
2. For each *i*, set *count*[*i*] equal to the number of times *text*[*i*, *i*+*k*] appears in *text*.
3. Take *k*-mers having the maximum values of *count*[*i*].

Example: *text* = ACGTTTCACGTTT**TAC**GG and *k* = 3.

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
count	3	2	2	3	1	1	1	3	2	2	3	3	1	1		

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

One Frequent Words Solution

1. Create an array *count* of length len(*text*) - *k* + 1.
2. For each *i*, set *count*[*i*] equal to the number of times *text*[*i*, *i*+*k*] appears in *text*.
3. Take *k*-mers having the maximum values of *count*[*i*].

Example: *text* = **ACG**TTTC**ACG**TTTT**ACG**G and *k* = 3.

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
count	3	2	2	3	1	1	1	3	2	2	3	3	1	1	3	1

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

One Frequent Words Solution

1. Create an array *count* of length len(*text*) - *k* + 1.
2. For each *i*, set *count*[*i*] equal to the number of times *text*[*i*, *i*+*k*] appears in *text*.
3. Take *k*-mers having the maximum values of *count*[*i*].

Example: *text* = ACGTTTCACGTTTT**ACGG** and *k* = 3.

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
count	3	2	2	3	1	1	1	3	2	2	3	3	1	1	3	1

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Solving the Frequent Words Problem

```
FrequentWords(text, k)
    freqPatterns ← an array of strings of length 0
    n ← Len(text)
    count ← array of integers of length n - k + 1
    for every integer i between 0 and n – k
        pattern ← text[i, i+k]
        count[i] ← PatternCount(pattern, text)
    max ← MaxArray(count)
    for every integer i between 0 and n - k
        if count[i] = max
            pattern ← text[i, i+k]
            freqPatterns ← Append(freqPatterns, pattern)
    freqPatterns ← RemoveDuplicates(freqPatterns)
    return freqPatterns
```

PatternCount: our pattern counting function from before
MaxArray: take maximum value in an array *a*
RemoveDuplicates: remove duplicates from list *patterns*

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Solving the Frequent Words Problem

```
FrequentWords(text, k)
    freqPatterns ← an array of strings of length 0
    n ← Len(text)
    count ← array of integers of length n - k + 1
    for every integer i between 0 and n – k
        pattern ← text[i, i+k]
        count[i] ← PatternCount(pattern, text)
    max ← MaxArray(count)
    for every integer i between 0 and n - k
        if count[i] = max
            pattern ← text[i, i+k]
            freqPatterns ← Append(freqPatterns, pattern)
    freqPatterns ← RemoveDuplicates(freqPatterns)
    return freqPatterns
```

STOP: This algorithm is *inefficient*; why? How could we make it better?

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Outline

- An Intro to DNA Replication
- Hidden Messages in the Replication Origin
- Hunting for Frequent Words
- **A Faster Frequent Words Approach**
- Some Hidden Messages are More Surprising than Others
- An Explosion of Hidden Messages
- Replication Asymmetry Leads Us to the Replication Origin

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Arrays/Slices Store Lists of Variables

H	i		T	h	e	r	e	!
0	1	2	3	4	5	6	7	8

1	1	2	3	5	8	13	21	34	55	89
0	1	2	3	4	5	6	7	8	9	10

"ACG"	"TTA"	"GAG"	"CCT"	"TAA"	"GGG"	"CAT"
0	1	2	3	4	5	6

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

What if the Indices Aren't Integers?

Would make things easier when finding frequent words …

Pattern	count
"AA"	17
"AC"	4
"CG"	15
"GA"	23
"GG"	3
"GT"	30
"TA"	18
"TG"	2
"TT"	24

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

What if the Indices Aren't Integers?

Map/Dictionary: An association of **keys** with **values**.

Pattern	count
"AA"	**17**
"AC"	**4**
"CG"	**15**
"GA"	**23**
"GG"	**3**
"GT"	**30**
"TA"	**18**
"TG"	**2**
"TT"	**24**

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

What if the Indices Aren't Integers?

Map/Dictionary: An association of **keys** with **values**.

We use a variable (say, *freq*) to refer to the map.

Pattern	count
"AA"	**17**
"AC"	**4**
"CG"	**15**
"GA"	**23**
"GG"	**3**
"GT"	**30**
"TA"	**18**
"TG"	**2**
"TT"	**24**

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

What if the Indices Aren't Integers?

Map/Dictionary: An association of **keys** with **values**.

We use a variable (say, *freq*) to refer to the map.

Value access is like arrays: *freq*["GT"]

Pattern	count
"AA"	**17**
"AC"	**4**
"CG"	**15**
"GA"	**23**
"GG"	**3**
"GT"	**30**
"TA"	**18**
"TG"	**2**
"TT"	**24**

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Note that not every 2-mer is a key...

Map/Dictionary: An association of **keys** with **values**.

We use a variable (say, *freq*) to refer to the map.

Value access is like arrays: *freq*["GT"]

Pattern	count
"AA"	**17**
"AC"	**4**
"CG"	**15**
"GA"	**23**
"GG"	**3**
"GT"	**30**
"TA"	**18**
"TG"	**2**
"TT"	**24**

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Rewriting Frequent Words Pseudocode

```
BetterFrequentWords(text, k)
    freqPatterns ← an empty array
    freqMap ← empty map
    n ← Len(text)
    for every integer i between 0 and n - k
        pattern ← text[i, i+k]
        if freqMap[pattern] doesn't exist
            freqMap[pattern] = 1
        else
            freqMap[pattern] ← freqMap[pattern] + 1
    maxCount ← MaxMap(freqMap)
    for all strings pattern in freqMap
        if freqMap[pattern] = maxCount
            freqPatterns ← Append(freqPatterns, pattern)
    return freqPatterns
```

Note: We don't need **RemoveDuplicates()** or **Count()**! And this is *much* faster!

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Shortening BetterFrequentWords()

```
BetterFrequentWords(text, k)
    freqPatterns ← an array of strings of length 0
    freqMap ← an empty map
    n ← Len(text)
    for every integer i between 0 and n - k
        pattern ← text[i, i+k]
        if freqMap[pattern] doesn't exist
            freqMap[pattern] = 1
        else
            freqMap[pattern] ← freqMap[pattern] + 1
    max ← MaxMap(freqMap)
    for all strings pattern in freqMap
        if freqMap[pattern] = max
            freqPatterns ← Append(freqPatterns, pattern)
    return freqPatterns
```

Subroutine time! We can shorten the code in **red**.

Shortening BetterFrequentWords()

```
BetterFrequentWords(text, k)
    freqPatterns ← an empty array of strings
    freqMap ← FrequencyMap(text, k)
    maxCount ← MaxValue(freqMap)
    for all strings pattern in freqMap
        if freqMap[pattern] = maxCount
            append pattern to freqPatterns
    return freqPatterns
```

```
FrequencyMap(text, k)
    freqMap ← an empty map
    n ← Len(text)
    for every integer i between 0 and n - k
        pattern ← text[i, i+k]
        if freqMap[pattern] doesn't exist
            freqMap[pattern] = 1
        else
            freqMap[pattern] ← freqMap[pattern] + 1
    return freqMap
```


Outline

- An Intro to DNA Replication
- Hidden Messages in the Replication Origin
- Hunting for Frequent Words
- A Faster Frequent Words Approach
- **Some Hidden Messages are More Surprising than Others**
- An Explosion of Hidden Messages
- Replication Asymmetry Leads Us to the Replication Origin

Returning to *ori* of *Vibrio cholerae*

```
atcaatgatcaacgtaagcttctaagcatgatcaaggtgctcacacagtttatccacaacctgagtgg
atgacatcaagataggtcgttgtatctccttcctctcgtactctcatgaccacggaaagatgatcaag
agaggatgatttcttggccatatcgcaatgaatacttgtgacttgtgcttccaattgacatcttcagc
gccatattgcgctggccaaggtgacggagcgggattacgaaagcatgatcatggctgttgttctgttt
atcttgttttgactgagacttgttaggatagacggtttttcatcactgactagccaaagccttactct
gcctgacatcgaccgtaaattgataatgaatttacatgcttccgcgacgatttacctcttgatcatcg
atccgattgaagatcttcaattgttaattctcttgcctcgactcatagccatgatgagctcttgatca
tgtttccttaaccctctatttttttacggaagaatgatcaagctgctgctcttgatcatcgtttc
```

k	3	4	5	6	7	8	9
count	25	12	8	8	5	4	3
k-mers	t ga	at ga	gat ca t gat c	t gat ca	at gat ca	at gat caa	at gat caag ct t gat cat t ct t gat ca ct ct t gat c

Returning to *ori* of *Vibrio cholerae*

```
atcaatgatcaacgtaagcttctaagcATGATCAAGgtgctcacacagtttatccacaacctgagtgg
atgacatcaagataggtcgttgtatctccttcctctcgtactctcatgaccacggaaagATGATCAAG
agaggatgatttcttggccatatcgcaatgaatacttgtgacttgtgcttccaattgacatcttcagc
gccatattgcgctggccaaggtgacggagcgggattacgaaagcatgatcatggctgttgttctgttt
atcttgttttgactgagacttgttaggatagacggtttttcatcactgactagccaaagccttactct
gcctgacatcgaccgtaaattgataatgaatttacatgcttccgcgacgatttacCTCTTGATCATcg
atccgattgaagatcttcaattgttaattctcttgcctcgactcatagccatgatgagCTCTTGATCA
TgtttccttaaccctctatttttttacggaagaATGATCAAGctgctgCTCTTGATCATcgtttc
```

Most frequent 9-mers in this *ori* (all appear 3 times):
ATGATCAAG, **CTTGATCAT**, **TCTTGGATCA**, **CTCTTGATC**

Returning to *ori* of *Vibrio cholerae*

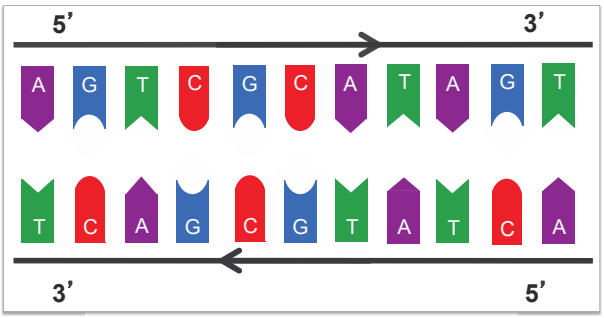
```
atcaatgatcaacgtaagcttctaagcATGATCAAGgtgctcacacagtttatccacaacctgagtgg
atgacatcaagataggtcgttgtatctccttcctctcgtactctcatgaccacggaaagATGATCAAG
agaggatgatttcttggccatatcgcaatgaatacttgtgacttgtgcttccaattgacatcttcagc
gccatattgcgctggccaaggtgacggagcgggattacgaaagcatgatcatggctgttgttctgttt
atcttgttttgactgagacttgttaggatagacggtttttcatcactgactagccaaagccttactct
gcctgacatcgaccgtaaattgataatgaatttacatgcttccgcgacgatttacCTCTTGATCATcg
atccgattgaagatcttcaattgttaattctcttgcctcgactcatagccatgatgagCTCTTGATCA
TgtttccttaaccctctatttttttacggaagaATGATCAAGctgctgCTCTTGATCATcgtttc
```

Most frequent 9-mers in this *ori* (all appear 3 times):
ATGATCAAG, **CTTGATCAT**, **TCTTGGATCA**, **CTCTTGATC**

STOP: Now what do you see?

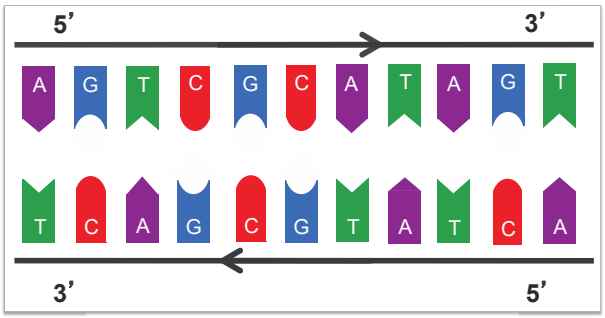
Complementarity of DNA

DNA is double-stranded, and the two strands are **reverse complements** of each other.



Complementarity of DNA

The reverse complement of AGTCGCATAGT is ACTATGCGACT.



Reverse Complement Problem

Reverse Complement Problem
- **Input:** A DNA string *text*.
- **Output:** The reverse complement of *text*.

Hidden Message Found!

```
atcaatgatcaacgtaagcttctaagcATGATCAAGgtgctcacacagtttatccacaacctgagtgg
atgacatcaagataggtcgttgtatctccttcctctcgtactctcatgaccacggaaagATGATCAAG
agaggatgatttcttggccatatcgcaatgaatacttgtgacttgtgcttccaattgacatcttcagc
gccatattgcgctggccaaggtgacggagcgggattacgaaagcatgatcatggctgttgttctgttt
atcttgttttgactgagacttgttaggatagacggtttttcatcactgactagccaaagccttactct
gcctgacatcgaccgtaaattgataatgaatttacatgcttccgcgacgatttacctCTTGATCATcg
atccgattgaagatcttcaattgttaattctcttgcctcgactcatagccatgatgagctCTTGATCA
TgtttccttaaccctctatttttttacggaagaATGATCAAGctgctgctCTTGATCATcgtttc
```

ATGATCAAG
TACTAGTTC

are *reverse complements* and likely *DnaA* boxes (*DnaA* does not know which strand it binds to).

Hidden Message Found!

```
atcaatgatcaacgtaagcttctaagcATGATCAAGgtgctcacacagtttatccacaacctgagtgg
atgacatcaagataggtcgttgtatctccttcctctcgtactctcatgaccacggaaagATGATCAAG
agaggatgatttcttggccatatcgcaatgaatacttgtgacttgtgcttccaattgacatcttcagc
gccatattgcgctggccaaggtgacggagcgggattacgaaagcatgatcatggctgttgttctgttt
atcttgttttgactgagacttgttaggatagacggtttttcatcactgactagccaaagccttactct
gcctgacatcgaccgtaaattgataatgaatttacatgcttccgcgacgatttacctCTTGATCATcg
atccgattgaagatcttcaattgttaattctcttgcctcgactcatagccatgatgagctCTTGATCA
TgtttccttaaccctctatttttttacggaagaATGATCAAGctgctgctCTTGATCATcgtttc
```

ATGATCAAG
TACTAGTTC

are *reverse complements* and likely *DnaA* boxes (*DnaA* does not know which strand it binds to).

It is **VERY SURPRISING** to find a 9-mer appearing **6 or more** times (with reverse complements) within ≈ 500 nucleotides.

Looking for other Hidden Messages?

STOP: Now that we know the "hidden message" in *Vibrio cholerae*, how would we look for a hidden message starting replication in *other* bacteria?

Hidden Messages in *T. petrophila*?

```
aactctatacctccttttttgtcgaatttgtgtgatttatagagaaaatcttattaactgaaactaa
aatggtaggtttggtggtaggttttgtgtacattttgtagtatctgatttttaattacatacccgta
tattgtattaaattgacgaacaattgcatggaattgaatatatgcaaaacaaacctaccaccaaac
tctgtattgaccatttttaggacaacttcagggtggtaggtttctgaagctctcatcaatagactat
tttagtctttacaaacaatattaccgttcagattcaagattctacaacgctgtttttaatgggcgtt
gcagaaaacttaccacctaaaatccagtatccaagccgatttcagagaaacctaccacttacctac
cacttacctaccacccgggtggtaagttgcagacattattaaaaacctcatcagaagcttgttcaa
aaatttcaatactcgaaacctaccacctgcgtcccctattatttactactactaataatagcagta
taattgatctgaaaagaggtggtaaaaaa
```

Not one occurrence of **ATGATCAAG** or **CTTGATCAT**!

Hidden Messages in *T. petrophila*?

```
aactctatacctccttttttgtcgaatttgtgtgatttatagagaaaatcttattaactgaaactaa
aatggtaggtttggtggtaggttttgtgtacattttgtagtatctgatttttaattacatacccgta
tattgtattaaattgacgaacaattgcatggaattgaatatatgcaaaacaaacctaccaccaaac
tctgtattgaccatttttaggacaacttcagggtggtaggtttctgaagctctcatcaatagactat
tttagtctttacaaacaatattaccgttcagattcaagattctacaacgctgtttttaatgggcgtt
gcagaaaacttaccacctaaaatccagtatccaagccgatttcagagaaacctaccacttacctac
cacttacctaccacccgggtggtaagttgcagacattattaaaaacctcatcagaagcttgttcaa
aaatttcaatactcgaaacctaccacctgcgtcccctattatttactactactaataatagcagta
taattgatctgaaaagaggtggtaaaaaa
```

Not one occurrence of **ATGATCAAG** or **CTTGATCAT**!

Applying Frequent Words Problem to this *ori*:
AACCTACCA, ACCTACCAC, GGTAGGTTT
TGGTAGGTT, AAACCTACC, CCTACCACC

Hidden Messages in *T. petrophila*?

```
aactctatacctccttttttgtcgaatttgtgtgatttatagagaaaatcttattaactgaaactaa
aatggtaggtttggtggtaggttttgtgtacattttgtagtatctgatttttaattacatacccgta
tattgtattaaattgacgaacaattgcatggaattgaatatatgcaaaacaaacctaccaccaaac
tctgtattgaccatttttaggacaacttcagggtggtaggtttctgaagctctcatcaatagactat
tttagtctttacaaacaatattaccgttcagattcaagattctacaacgctgtttttaatgggcgtt
gcagaaaacttaccacctaaaatccagtatccaagccgatttcagagaaacctaccacttacctac
cacttacctaccacccgggtggtaagttgcagacattattaaaaacctcatcagaagcttgttcaa
aaatttcaatactcgaaacctaccacctgcgtcccctattatttactactactaataatagcagta
taattgatctgaaaagaggtggtaaaaaa
```

Different genomes → different hidden messages

Applying Frequent Words Problem to this *ori*:
AACCTACCA, ACCTACCAC, GGTAGGTTT
TGGTAGGTT, AAACCTACC, CCTACCACC

Hidden Messages in *Thermotoga petrophila*

```
aactctatacctccttttttgtcgaatttgtgtgatttatagagaaaatcttattaactgaaactaa
aatggtaggtttGGTGGTAGGttttgtgtacattttgtagtatctgatttttaattacatacccgta
tattgtattaaattgacgaacaattgcatggaattgaatatatgcaaaacaaaCCTACCACCaaac
tctgtattgaccatttttaggacaacttcagGGTGGTAGGtttctgaagctctcatcaatagactat
tttagtctttacaaacaatattaccgttcagattcaagattctacaacgctgtttttaatgggcgtt
gcagaaaacttaccacctaaaatccagtatccaagccgatttcagagaaacctaccacttacctac
cactaCCTACCACCcgggtggtaagttgcagacattattaaaaacctcatcagaagcttgttcaa
aaatttcaatactcgaaaCCTACCACCtgcgtcccctattatttactactactaataatagcagta
taattgatctgaaaagaggtggtaaaaaa
```

CCTACCACC
GGATGGTGG

are candidate hidden messages.

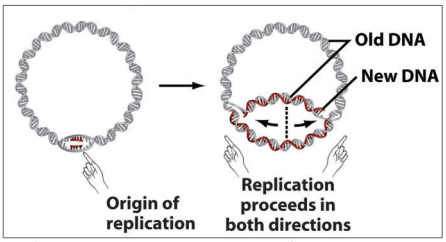
Outline

- An Intro to DNA Replication
- Hidden Messages in the Replication Origin
- Hunting for Frequent Words
- A Faster Frequent Words Approach
- Some Hidden Messages are More Surprising than Others
- **An Explosion of Hidden Messages**
- Replication Asymmetry Leads Us to the Replication Origin

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Returning to "Problem 1"

We have found hidden messages if *ori* is given. But we still don't know how to find *ori* in a (long) genome.



Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Bacteria with Unknown *ori*

STOP: Now that we know that "hidden messages" may differ, how could we look for *ori* in a newly sequenced bacterial genome?

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Finding *ori* Computationally

OLD strategy: given a previously **known** *ori* (500 nucleotide window), find **frequent words** (clumps) in *ori* as candidate *DnaA* boxes.

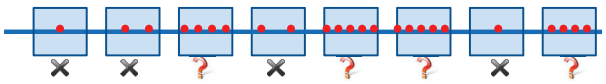
replication origin → frequent words

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Finding *ori* Computationally

OLD strategy: given a previously **known** *ori* (500 nucleotide window), find **frequent words** (clumps) in *ori* as candidate *DnaA* boxes.

replication origin → frequent words



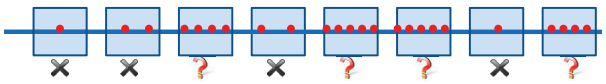
NEW strategy: find frequent words in **ALL** windows within a (3 million nucleotide) genome. Windows with **clumps** of frequent words are candidate replication origins.

frequent words → replication origin

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Finding *ori* Computationally

Exercise: Define a computational problem modeling our new strategy.



NEW strategy: find frequent words in **ALL** windows within a (3 million nucleotide) genome. Windows with **clumps** of frequent words are candidate replication origins.

frequent words → replication origin

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Defining and Hunting for "Clumps"

A *k*-mer forms an (***L***, ***t***)-**clump** inside *Genome* if there is a **short** (length ***L***) interval of *Genome* in which it appears **many** (at least ***t***) times.

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Defining and Hunting for "Clumps"

A *k*-mer forms an (***L***, ***t***)-**clump** inside *Genome* if there is a **short** (length ***L***) interval of *Genome* in which it appears **many** (at least ***t***) times.

Clump Finding Problem

- **Input:** A string *Genome* and integers *k* (length of a pattern), ***L*** (window length), and ***t*** (number of patterns in a clump).
- **Output:** All *k*-mers forming (***L***, ***t***)-**clumps** in *Genome*.

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Defining and Hunting for "Clumps"

```
FindClumps(text, k, L, t)
        patterns ← an array of strings of length 0
        n ← Len(text)
        for every integer i between 0 and n – L
                window ← text[i, i + L]
                freqMap ← FrequencyMap(window, k)
                for every string pattern in freqMap
                        if freqMap[pattern] >= t
                                patterns ← Append(patterns, pattern)
        patterns ← RemoveDuplicates(patterns)
        return patterns
```

Note: A complicated function can be made easier by using subroutines as building blocks.

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Defining and Hunting for "Clumps"

```
FindClumps(text, k, L, t)
        patterns ← an array of strings of length 0
        n ← Len(text)
        for every integer i between 0 and n – L
                window ← text[i, i + L]
                freqMap ← FrequencyMap(window, k)
                for every string pattern in freqMap
                        if freqMap[pattern] >= t
                                patterns ← Append(patterns, pattern)
        patterns ← RemoveDuplicates(patterns)
        return patterns
```

STOP (biologists only): Why is looking for clumps in bacterial genomes as a source of hidden messages destined to fail?

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

What's the Issue?

Genomes have *many* **repeats**, some more useful than others. Alu in humans is ~300 bp long and occurs (with some changes) 1 million times.

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

What's the Issue?

Genomes have *many* **repeats**, some more useful than others. Alu in humans is ~300 bp long and occurs (with some changes) 1 million times.

In *E. coli*, **1900+** *different* 9-mers form (500,3)-clumps. It is unclear which ones point to *ori*...

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Outline

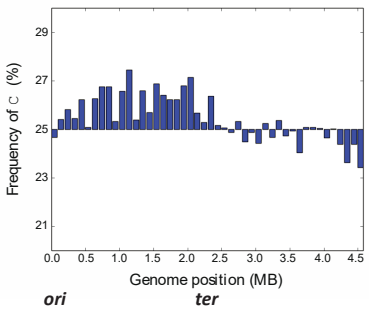
- An Intro to DNA Replication
- Hidden Messages in the Replication Origin
- Hunting for Frequent Words
- A Faster Frequent Words Approach
- Some Hidden Messages are More Surprising than Others
- An Explosion of Hidden Messages
- **Replication Asymmetry Leads Us to the Replication Origin**

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

A Surprising Pattern in Nucleotide Counts

Let's run a very simple computational analysis: take frequency of each nucleotide in 100,000 nucleotide windows of *E. coli* (verified *ori*).

Why would there be more C on half the genome?

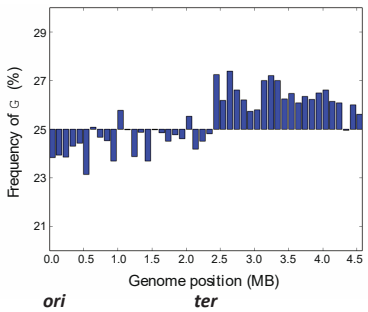


Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

A Surprising Pattern in Nucleotide Counts

Let's run a very simple computational analysis: take frequency of each nucleotide in 100,000 nucleotide windows of *E. coli* (verified *ori*).

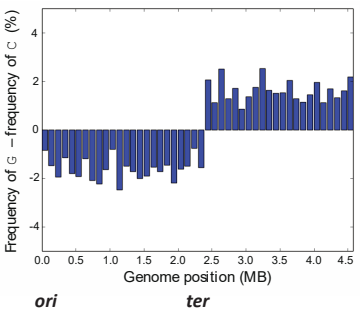
And why would the story be opposite when we count G's?



Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Taking Difference in G – C

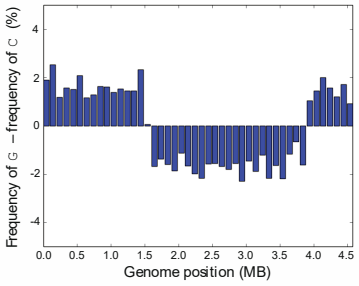
The pattern is even more stark if we take the *difference* between the frequency of G and the frequency of C ...



Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Taking Difference in G – C

And the pattern is still there even if we didn't know where *ori* was and start counting at some arbitrary spot.

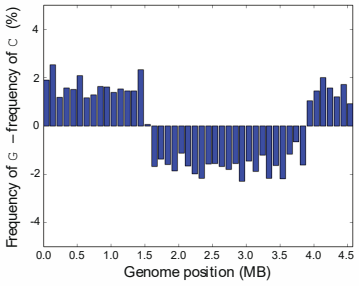


Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Taking Difference in G – C

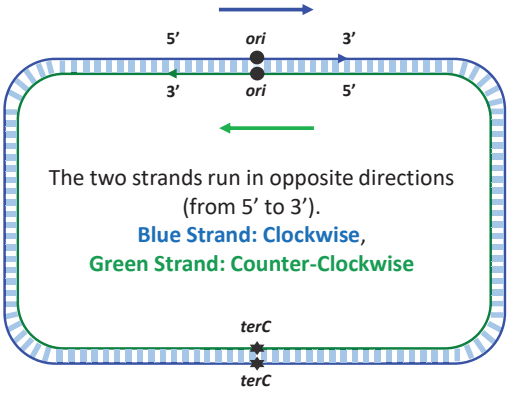
And the pattern is still there even if we didn't know where *ori* was and start counting at some arbitrary spot.

Let's learn more about replication in the hope of finding an answer...



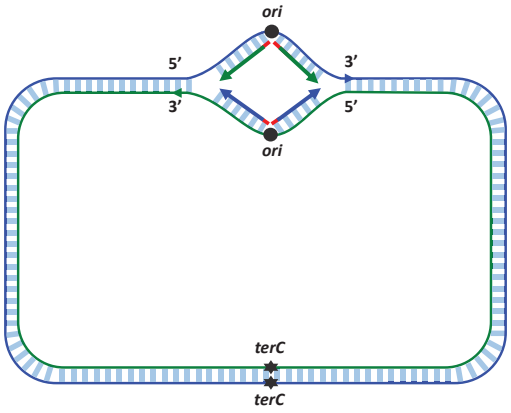
Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

DNA Strands Have Directions



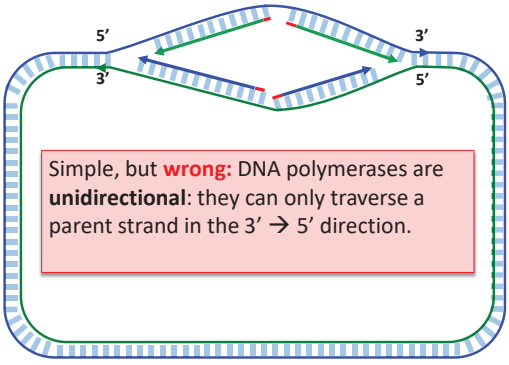
Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Four DNA Polymerases Can Do the Job



Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

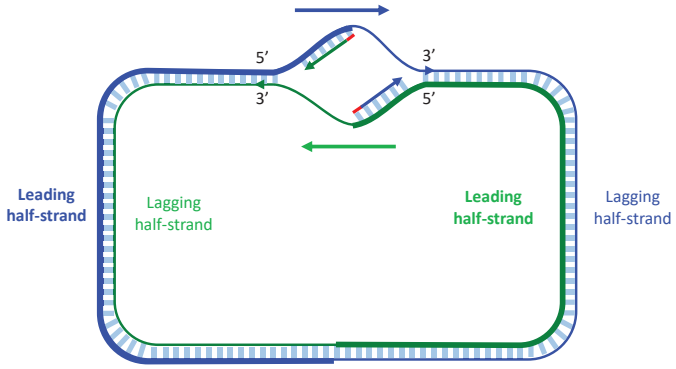
Continue as Replication Fork Enlarges



Simple, but **wrong:** DNA polymerases are **unidirectional**: they can only traverse a parent strand in the 3′ → 5′ direction.

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

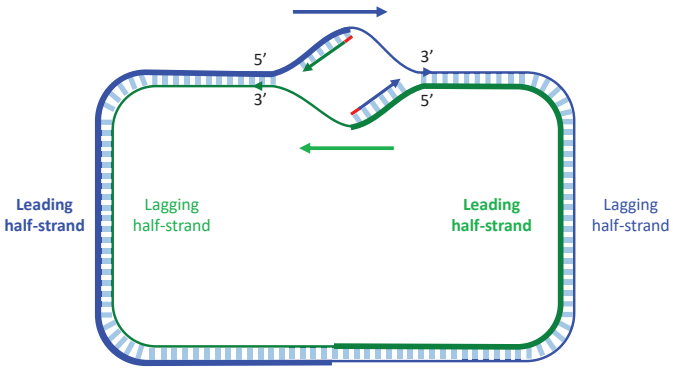
If you Were a **UNIDIRECTIONAL** DNA Polymerase, how Would you Replicate a Genome?



Big problem replicating lagging half-strands (thin lines).

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

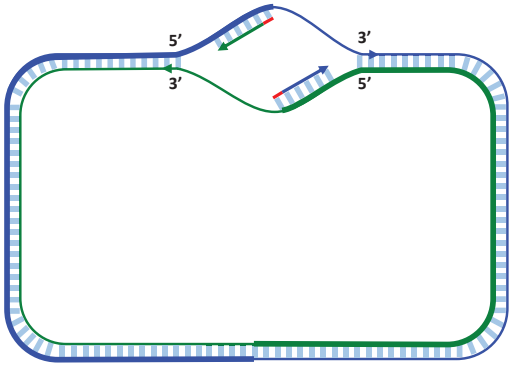
If you Were a **UNIDIRECTIONAL** DNA Polymerase, how Would you Replicate a Genome?



Note: Leading/lagging half-strands are *complementary*.

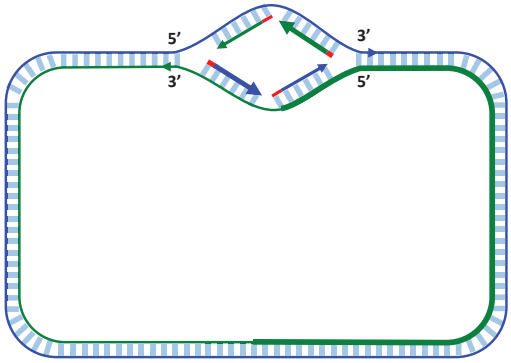
Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Wait until the Fork Opens and ...



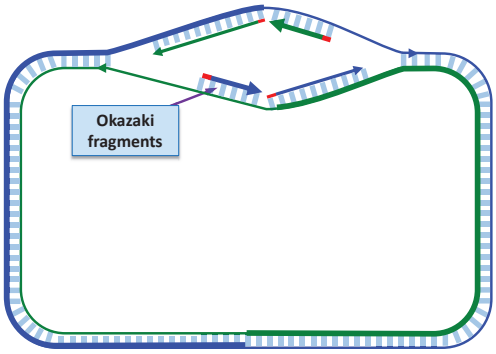
Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Wait until the Fork Opens and Replicate



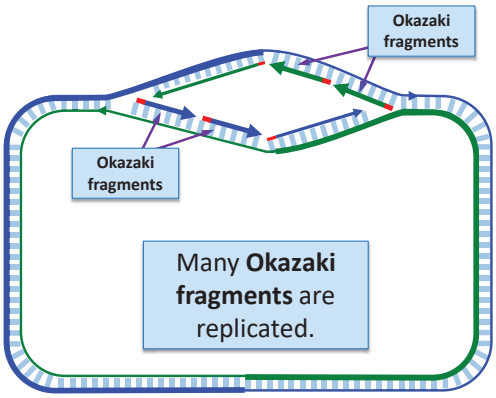
Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Iterate this Process



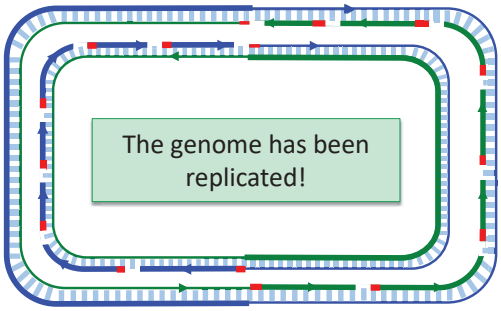
Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Iterate this Process



Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Okazaki Fragments Must Be Ligated

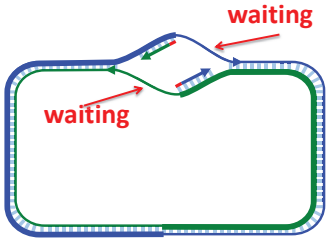


Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Different Lifestyles of Half-strands

The **leading half-strand** lives a **double-stranded** life most of the time.

The **lagging half-strand** spends a large portion of its life **single-stranded**, **waiting** to be replicated.



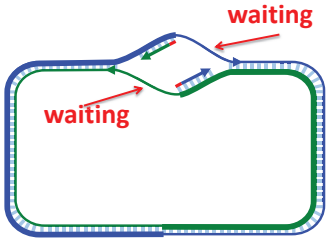
Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Different Lifestyles of Half-strands

The **leading half-strand** lives a **double-stranded** life most of the time.

The **lagging half-strand** spends a large portion of its life **single-stranded**, **waiting** to be replicated.

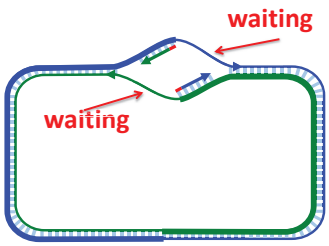
But why would a computer scientist care?



Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Asymmetry of Replication Affects Nucleotide Frequencies

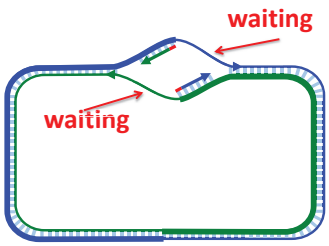
Single-stranded DNA has a much higher mutation rate than double-stranded DNA.



Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Asymmetry of Replication Affects Nucleotide Frequencies

Single-stranded DNA has a much higher mutation rate than double-stranded DNA.



Thus, if one nucleotide has a greater mutation rate, then we should observe its **shortage** on the lagging half-strand, since it is more often single-stranded!

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

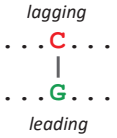
Deamination is the Answer

Cytosine (C) rapidly mutates into thymine (T) through **deamination**; deamination rates rise 100-fold when DNA is single-stranded!

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Deamination is the Answer

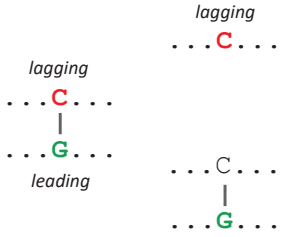
Cytosine (C) rapidly mutates into thymine (T) through **deamination**; deamination rates rise 100-fold when DNA is single-stranded!



Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Deamination is the Answer

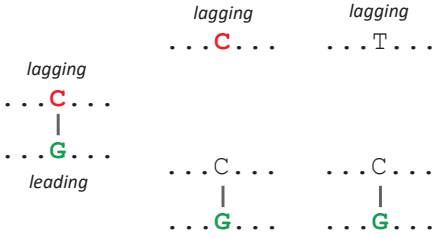
Cytosine (C) rapidly mutates into thymine (T) through **deamination**; deamination rates rise 100-fold when DNA is single-stranded!



Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Deamination is the Answer

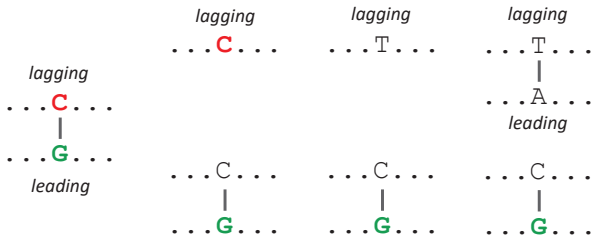
Cytosine (C) rapidly mutates into thymine (T) through **deamination**; deamination rates rise 100-fold when DNA is single-stranded!



Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

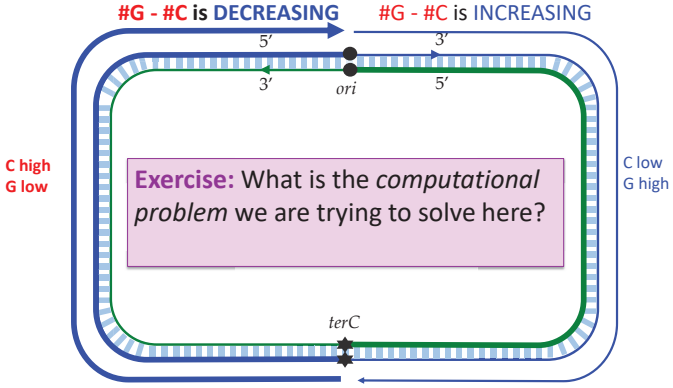
Deamination is the Answer

Cytosine (C) rapidly mutates into thymine (T) through **deamination**; deamination rates rise 100-fold when DNA is single-stranded!



Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Take a Walk Along the Genome

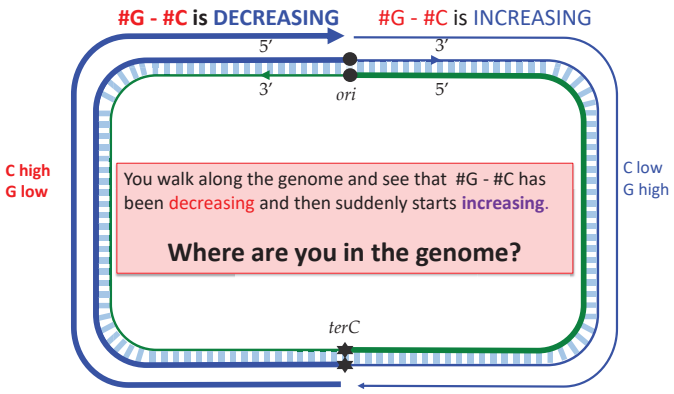


C high/G low → #G - #C is DECREASING as we walk along the LEADING half-strand

C low/G high → #G - #C is INCREASING as we walk along the LAGGING half-strand

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Take a Walk Along the Genome



C high/G low → #G - #C is DECREASING as we walk along the LEADING half-strand

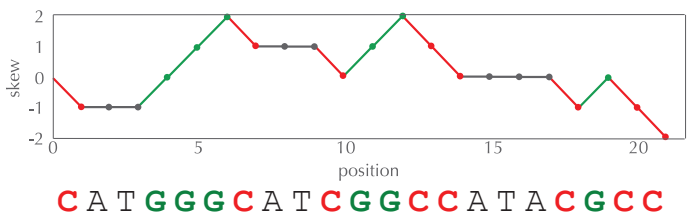
C low/G high → #G - #C is INCREASING as we walk along the LAGGING half-strand

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Skew Array/Diagram

Skew array: *Skew*[*k*] = #G - #C for the **first *k* nucleotides** of *Genome*.

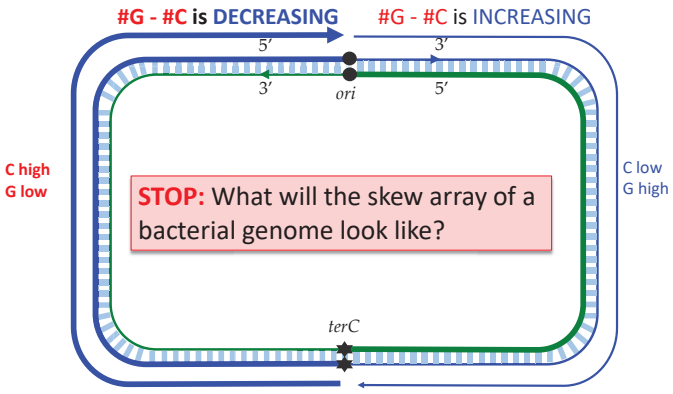
Skew diagram: Plot *Skew*[*k*] against *k*.



C A T G G G C A T C G G C C A T A C G C C

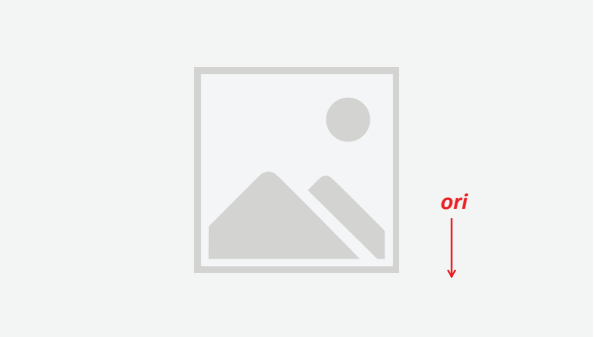
Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Skew Array/Diagram



Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Skew Diagram of *E. Coli*

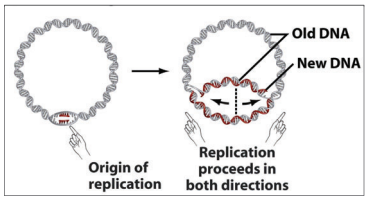


You walk along the genome and see that #G - #C have been decreasing and then suddenly starts increasing. Where are you in the genome?

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

We Have Now "Solved" Question 1!

Given a bacterial genome (~3 Mbp), where is *ori*?



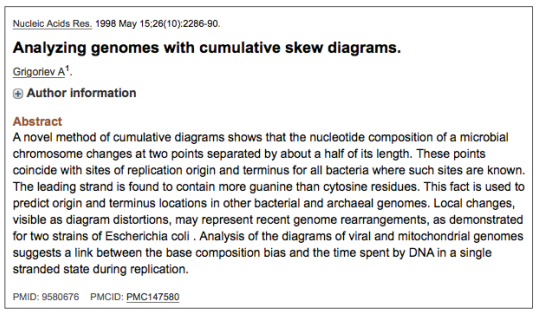
Minimum Skew Problem

- **Input:** A DNA string *genome*.
- **Output:** The min value of *Skew*[*k*] for *genome*.

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

We Have Now "Solved" Question 1!

Given a bacterial genome (~3 Mbp), where is *ori*?



Nucleic Acids Res. 1998 May 15;26(10):2286-90.

Analyzing genomes with cumulative skew diagrams.

Grigoriev A[1].

Author information

Abstract

A novel method of cumulative diagrams shows that the nucleotide composition of a microbial chromosome changes at two points separated by about a half of its length. These points coincide with sites of replication origin and terminus for all bacteria where such sites are known. The leading strand is found to contain more guanine than cytosine residues. This fact is used to predict origin and terminus locations in other bacterial and archaeal genomes. Local changes, visible as diagram distortions, may represent recent genome rearrangements, as demonstrated for two strains of Escherichia coli . Analysis of the diagrams of viral and mitochondrial genomes suggests a link between the base composition bias and the time spent by DNA in a single stranded state during replication.

PMID: 9580676 PMCID: PMC147580

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

We Found the Replication Origin in *E. Coli* BUT…

```
aatgatgatgacgtcaaaaggatccggataaaacatggtgattgcctcgcataacgcgg
tatgaaaatggattgaagcccgggccgtggattctactcaactttgtcggcttgagaaa
gacctgggatcctgggtattaaaaagaagatctatttatttagagatctgttctattgt
gatctcttattaggatcgcactgccctgtggataacaaggatccggcttttaagatcaa
caacctggaaaggatcattaactgtgaatgatcggtgatcctggaccgtataagctggg
atcagaatgaggggttatacacaactcaaaaactgaacaacagttgttctttggataac
taccggttgatccaagcttcctgacagagttatccacagtagatcgcacgatctgtata
cttatttgagtaaattaacccacgatcccagccattcttctgccggatcttccggaatg
tcgtgatcaagaatgttgatcttcagtg
```

But there are **no** frequent 9-mers (that appear three or more times) in this region of *E. coli*!

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

We Found the Replication Origin in *E. Coli* BUT…

```
aatgatgatgacgtcaaaaggatccggataaaacatggtgattgcctcgcataacgcgg
tatgaaaatggattgaagcccgggccgtggattctactcaactttgtcggcttgagaaa
gacctgggatcctgggtattaaaaagaagatctatttatttagagatctgttctattgt
gatctcttattaggatcgcactgccctgtggataacaaggatccggcttttaagatcaa
caacctggaaaggatcattaactgtgaatgatcggtgatcctggaccgtataagctggg
atcagaatgaggggttatacacaactcaaaaactgaacaacagttgttctttggataac
taccggttgatccaagcttcctgacagagttatccacagtagatcgcacgatctgtata
cttatttgagtaaattaacccacgatcccagccattcttctgccggatcttccggaatg
tcgtgatcaagaatgttgatcttcagtg
```

But there are **no** frequent 9-mers (that appear three or more times) in this region of *E. coli*!

STOP: Any ideas? Should we give up?

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Accounting for Point Mutations

```
aatgatgatgacgtcaaaaggatccggataaaacatggtgattgcctcgcataacgcgg
tatgaaaatggattgaagcccgggccgtggattctactcaactttgtcggcttgagaaa
gacctgggatcctgggtattaaaaagaagatctatttatttagagatctgttctattgt
gatctcttattaggatcgcactgcccTGTGGATAAcaaggatccggcttttaagatcaa
caacctggaaaggatcattaactgtgaatgatcggtgatcctggaccgtataagctggg
atcagaatgagggTTATACACAactcaaaaactgaacaacagttgttcTTTGGATAAC
taccggttgatccaagcttcctgacagagTTATCCACAgtagatcgcacgatctgtata
cttatttgagtaaattaacccacgatcccagccattcttctgccggatcttccggaatg
tcgtgatcaagaatgttgatcttcagtg
```

Frequent 9-mers (with 1 Mismatch and Reverse Complements) in putative *ori* of *E. coli*

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Complications

- Some bacteria have fewer *DnaA* boxes.
- Terminus of replication is often not located directly opposite to *ori*.
- The skew diagram is often more complex than in the case of *E. coli*.

Skew diagram of *T. petrophila*

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

How Do We Find Regulatory Motifs in DNA?

Greedy and Randomized Algorithms

Phillip Compeau and Pavel Pevzner.
Bioinformatics Algorithms: An Active Learning Approach

©2013 by Compeau and Pevzner. All rights reserved.

Outline

- **From Implanted Patterns to Regulatory Motifs**
  - **Implanting Patterns into Strings**
  - Do We Have a Clock Gene?
  - Implanted Motif Problem
  - Motif Finding Problem (Music Break)
  - Median String Problem
  - Greedy Motif Search
- How Rolling Dice Helps Us Find Regulatory Motifs
  - Randomized Motif Search
  - How do Bacteria Hibernate?
  - Gibbs Sampling
  - Pseudocounts

Generate Ten Random Sequences

Implant a Pattern **AAAAAAAGGGGGGG** at Randomly Chosen Positions

Where Are the Implanted Patterns Hiding?

The implanted patterns can be found by the Frequent Words algorithm.

Implant a Pattern **AAAAAAGGGGGGG** with 4 Random Mutations at Random Positions

A *(k,d)-motif*: a *k*-mer that appears in each sequence with at most *d* mismatches. **AAAAAAGGGGGGG is a (15,4)-motif**

Now Where are the Implanted Patterns?

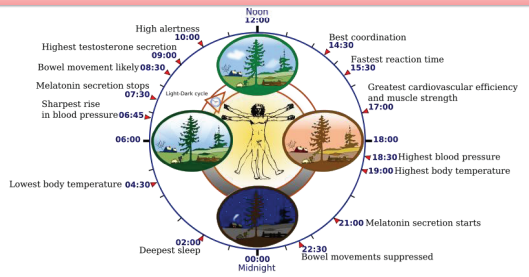
Would the Frequent Words algorithm find this hidden message?

Outline

- **From Implanted Patterns to Regulatory Motifs**
  - Implanting Patterns into Strings
  - **Do We Have a Clock Gene?**
  - Implanted Motif Problem
  - Motif Finding Problem (Music Break)
  - Median String Problem
  - Greedy Motif Search
- How Rolling Dice Helps Us Find Regulatory Motifs
  - Randomized Motif Search
  - How do Bacteria Hibernate?
  - Gibbs Sampling
  - Pseudocounts

Circadian Clock

Our schedules are controlled by a molecular timekeeper called the **circadian clock**.



How does each cell in our body know when it should decrease or increase **gene expression** of circadian genes?

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

How do Plants Know Whether it is Day or Night?

Plants need to **change gene expression** of 1000+ genes (photosynthesis, flowering, frost resistance, etc.) switching from day to night and vice-versa.

Who are the "molecular managers" telling genes to switch expression on and off in billions of cells?

Just 3 genes call the shots: CCA1, LCY, and TOC1.

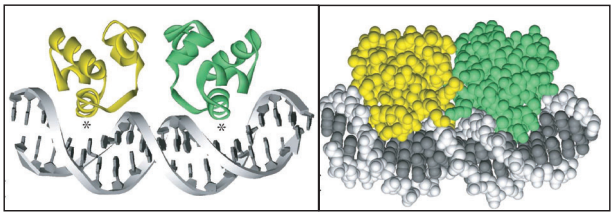
Night Blooming Cereus only blooms at night

Sunflower follows the sun

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

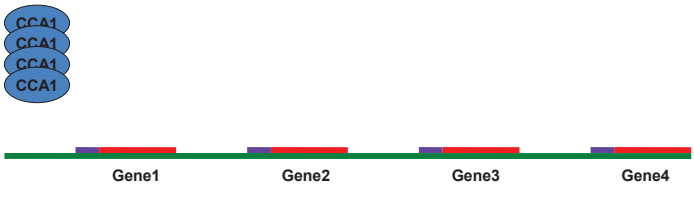
Transcription Factors and Their Binding Sites

CCA1, LCY, and TOC1 are **regulatory proteins** (a.k.a. **transcription factors**) control other genes by binding to short DNA fragments (**transcription factor binding sites**) in the upstream regions of these genes.



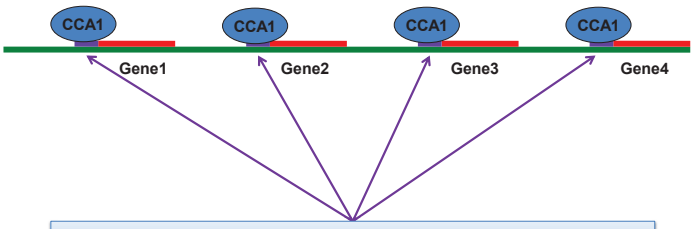
Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Transcription Factors Bind to Their Binding Sites



Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

How Does CCA1 Know Where to Bind?



There must be some hidden messages in these regions that tells CCA1 **WHERE** to bind.

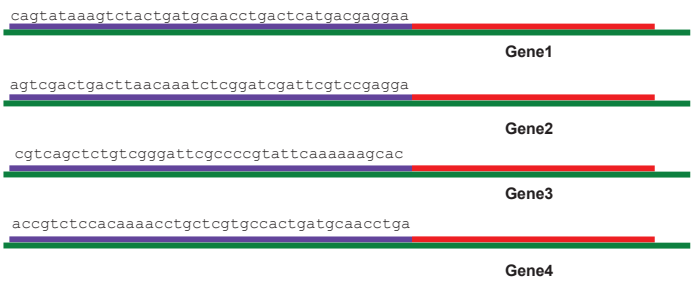
Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Transcription Factor Binding Sites

The hidden messages (motif **AAAAAATCT**): transcription factor binding sites of CCA1

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Where are the Hidden Messages Hiding?



Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Outline

- **From Implanted Patterns to Regulatory Motifs**
  - Implanting Patterns into Strings
  - Do We Have a Clock Gene?
  - **Implanted Motif Problem**
  - Motif Finding Problem (Music Break)
  - Median String Problem
  - Greedy Motif Search
- How Rolling Dice Helps Us Find Regulatory Motifs
  - Randomized Motif Search
  - How do Bacteria Hibernate?
  - Gibbs Sampling
  - Pseudocounts

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Randomly Implant **AAAAAAGGGGGGG** with 4 Random Mutations at 4 Random Positions

Would the Frequent Words algorithm find this (15,4) motif?

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Implanted Motifs Problem

Implanted Motif Problem. Finding *(k,d)*-motifs in a set of strings.

- **Input:** A set of strings *Dna*, and integers *k* (motif length) and *d* (maximal number of mismatches in a motif).
- **Output:** All *(k,d)*-motifs in *Dna*.

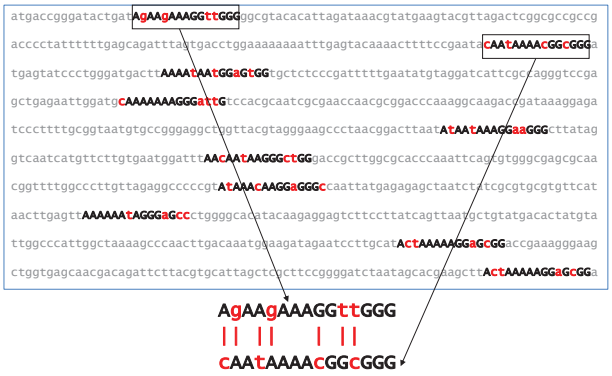
Should we explore ALL 4^k *k*-mers to solve the Implanted Motif Problem?

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Finding Implanted Motifs by Pairwise Comparison

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Finding Implanted Motifs by Pairwise Comparison



Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Why Pairwise Comparison Won't Work

```
AgAAgAAAGGttGGG  Implanted instance 1
 |  |     ||    4 mismatches
AAAAAAAAGGGGGGG + Pattern
|  |  |     |  4 mismatches
cAAtAAAAcGGGGGc  Implanted instance 2
```

The implanted instances have up to **4 + 4 = 8 mutations** – **they don't even look like similar strings!**

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Resorting to Motif Enumeration Instead

```
MotifEnumeration(Dna, k, d)
for each k-mer a in Dna
  for each k-mer a' differing from a by at most d mutations
    if a' is a (k,d)-mer
      output a'
```

Would this simple (albeit slow) algorithm work for finding **real** biological motifs?

MotifEnumeration assumes that **EACH** sequence has a *k*-mer similar to an (unknown) implanted pattern. This condition does not hold for noisy biological datasets.

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Outline

- **From Implanted Patterns to Regulatory Motifs**
  - Implanting Patterns into Strings
  - Do We Have a Clock Gene?
  - Implanted Motif Problem
  - **Motif Finding Problem (Music Break)**
  - Median String Problem
  - Greedy Motif Search
- How Rolling Dice Helps Us Find Regulatory Motifs
  - Randomized Motif Search
  - How do Bacteria Hibernate?
  - Gibbs Sampling
  - Pseudocounts

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

From Motifs to a Consensus String

Motifs

```
T C G G G G g T T T t t
c C G G t G A c T T a C
a C G G G G A T T T t C
T t G G G G A c T T t t
a a G G G G A c T T C C
T t G G G G A c T T C C
T C G G G G A T T c a t
T C G G G G A T T c C t
T a G G G G A a c T a C
T C G G G t A T a a C C
```


Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

From Motifs to a Consensus String

Motifs

T C G G G G g T T T t t
c C G G t G A c T T a C
a C G G G G A T T T t C
T t G G G G A c T T t t
a a G G G G A c T T C C
T t G G G G A c T T C C
T C G G G G A T T c a t
T C G G G G A T T c C t
T a G G G G A a c T a C
T C G G G t A T a a C C

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

From Motifs to a Consensus String

Motifs

T C G G G G g T T T t t
c C G G t G A c T T a C
a C G G G G A T T T t C
T t G G G G A c T T t t
a a G G G G A c T T C C
T t G G G G A c T T C C
T C G G G G A T T c a t
T C G G G G A T T c C t
T a G G G G A a c T a C
T C G G G t A T a a C C

Motif Logo

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

From Motifs to a Consensus String

Motifs

T C G G G G g T T T t t
c C G G t G A c T T a C
a C G G G G A T T T t C
T t G G G G A c T T t t
a a G G G G A c T T C C
T t G G G G A c T T C C
T C G G G G A T T c a t
T C G G G G A T T c C t
T a G G G G A a c T a C
T C G G G t A T a a C C

Consensus(*Motifs*) T C G G G G A T T T C C

Motif Logo

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

From Motifs to a Consensus String

Motifs

T C G G G G g T T T t t
c C G G t G A c T T a C
a C G G G G A T T T t C
T t G G G G A c T T t t
a a G G G G A c T T C C
T t G G G G A c T T C C
T C G G G G A T T c a t
T C G G G G A T T c C t
T a G G G G A a c T a C
T C G G G t A T a a C C

Consensus(*Motifs*) T C G G G G A T T T C C

Motif Logo

Score(*Motifs*) 3 + 4 + 0 + 0 + 1 + 1 + 5 + 2 + 3 + 6 + 4 = 30

Score(*Motifs*): # of lowercase letters in motif matrix

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

The Motif Finding Problem

Motif Finding Problem: *Given a collection of strings, find a set of k-mers, one from each string, that minimizes the score of the resulting motif.*

- **Input:** A collection of strings *Dna* and an integer *k*.
- **Output:** A collection *Motifs* of *k*-mers, one from each string in *Dna*, minimizing *Score*(*Motifs*) among all possible choices of *k*-mers.

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Brute Force Motif Search

Brute Force Motif Search: Compute score of every possible choice of *k*-mers in *Dna*.

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Brute Force Motif Search

Brute Force Motif Search: Compute score of every possible choice of *k*-mers in *Dna*.

STOP and Think: What is this approach's runtime?

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Brute Force Motif Search

Brute Force Motif Search: Compute score of every possible choice of *k*-mers in *Dna*.

STOP and Think: What is this approach's runtime?

Say that *Dna* consists of *t* strings of length *n*:

- There are $(n - k + 1)^t$ different choices of *Motifs*
- Scoring the matrix requires $k*t$ steps.

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Brute Force Motif Search

Brute Force Motif Search: Compute score of every possible choice of *k*-mers in *Dna*.

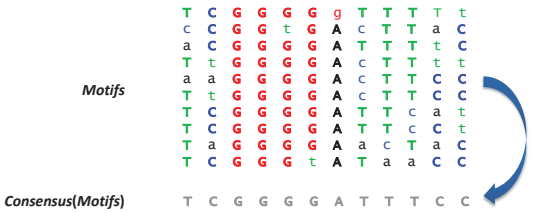
STOP and Think: What is this approach's runtime?

Say that *Dna* consists of *t* strings of length *n*:
- There are $(n - k + 1)^t$ different choices of *Motifs*
- Scoring the matrix requires $k*t$ steps.

Overall runtime ~ $O(n^t * k * t)$ – **too slow!**

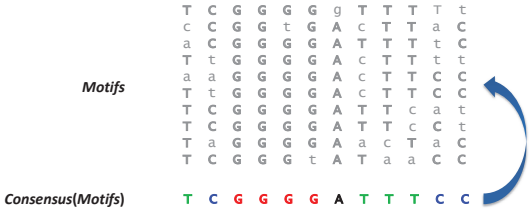
Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Changing Our Perspective



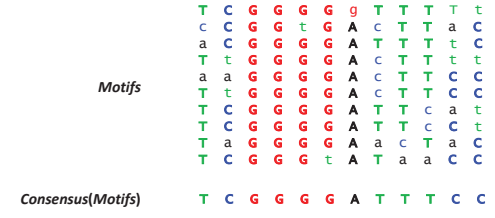
Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Changing Our Perspective



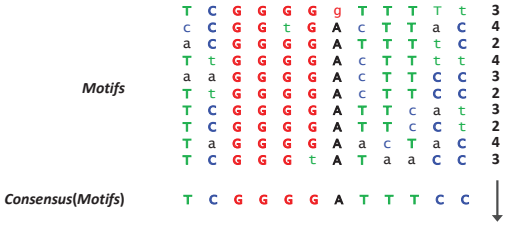
Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Changing Our Perspective



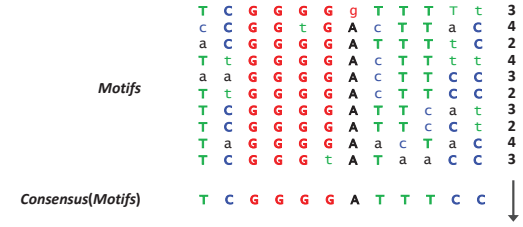
Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Changing Our Perspective



Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

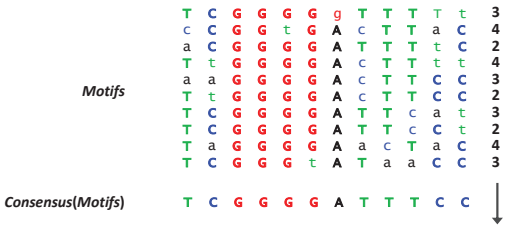
Changing Our Perspective



Number of lowercase symbols in a row = Hamming distance between that row and *Consensus*(*Motifs*).

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Changing Our Perspective



Number of lowercase symbols in a row = Hamming distance between that row and *Consensus*(*Motifs*).

Score(*Motifs*) = *d*(*Consensus*(*Motifs*), *Motifs*)

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

The Equivalent Motif Finding Problem

Equivalent Motif Finding Problem: *Given a collection of strings, find a pattern and a collection of k-mers (one from each string) that minimizes the distance between all possible patterns and all possible collections of k-mers.*
- **Input:** A collection of strings *Dna* and an integer *k*.
- **Output:** A *k*-mer *Pattern* and a collection of *k*-mers *Motifs*, one from each string in *Dna*, minimizing *d*(*Pattern*, *Motifs*) among all possible choices of *Pattern* and *Motifs*.

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

The Equivalent Motif Finding Problem

Equivalent Motif Finding Problem: *Given a collection of strings, find a pattern and a collection of k-mers (one from each string) that minimizes the distance between all possible patterns and all possible collections of k-mers.*

- **Input:** A collection of strings *Dna* and an integer *k*.
- **Output:** A *k*-mer *Pattern* and a collection of *k*-mers *Motifs*, one from each string in *Dna*, minimizing *d*(*Pattern*, *Motifs*) among all possible choices of *Pattern* and *Motifs*.

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Have We Just Made Our Task More Difficult?

Motif Finding Problem. Find a set of *k*-mers *Motifs*, one from each sequence in *Dna*, minimizing *Score(Motifs)*.

Equivalent Motif Finding Problem. Find *k*-mer *Pattern* **AND** a set of *k*-mers *Motifs*, one from each sequence in *Dna*, minimizing *d(Pattern, Motifs)*.

The key insight for solving the Equivalent Motif Finding Problem: given *Pattern*, we can quickly find *Motifs* minimizing *d(Pattern,Motifs)*.

↓

No need to explore all *Motifs* like in **Motif Finding Problem**! Instead, the **Median String Problem** explores all *k*-mers *Pattern*

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Outline

- **From Implanted Patterns to Regulatory Motifs**
  - Implanting Patterns into Strings
  - Do We Have a Clock Gene?
  - Implanted Motif Problem
  - Motif Finding Problem (Music Break)
  - **Median String Problem**
  - Greedy Motif Search
- How Rolling Dice Helps Us Find Regulatory Motifs
  - Randomized Motif Search
  - How do Bacteria Hibernate?
  - Gibbs Sampling
  - Pseudocounts

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Distance between a *k*-mer and a (longer) String

Distance *d*(GATTCTCA, GCAAAGACGCTGACCAA) = ?

```
G A T T C T C A
| | | | | | | |
G C A A A G A C G C T G A C C A A
```

Distance: 7

d(*Pattern*, *String*):
minimum distance between *Pattern* and all *k*-mers in *String*

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Distance between a *k*-mer and a (longer) String

d(GATTCTCA, GCAAAGACGCTGACCAA) = ?

```
G A T T C T C A
|   | | | |   |
G C A A A G A C G C T G A C C A A
```

Distance: 7 6

d(*Pattern*, *String*):
minimum distance between *Pattern* and all *k*-mers in *String*

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Distance between a *k*-mer and a (longer) String

d(GATTCTCA, GCAAAGACGCTGACCAA) = ?

```
G A T T C T C A
|   | | | | | |
G C A A A G A C G C T G A C C A A
```

Distance: 7 6 7

d(*Pattern*, *String*):
minimum distance between *Pattern* and all *k*-mers in *String*

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Distance between a *k*-mer and a (longer) String

d(GATTCTCA, GCAAAGACGCTGACCAA) = ?

```
G A T T C T C A
|   |   |   |
G C A A A G A C G C T G A C C A A
```

Distance: 7 6 7 5

d(*Pattern*, *String*):
minimum distance between *Pattern* and all *k*-mers in *String*

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Distance between a *k*-mer and a (longer) String

d(GATTCTCA, GCAAAGACGCTGACCAA) = ?

```
G A T T C T C A
| | | | | | | |
G C A A A G A C G C T G A C C A A
```

Distance: 7 6 7 5 8

d(*Pattern*, *String*):
minimum distance between *Pattern* and all *k*-mers in *String*

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Distance between a *k*-mer and a (longer) String

d(GATTCTCA, GCAAAGACGCTGACCAA) = ?

d(*Pattern*, *String*):
minimum distance between *Pattern* and all *k*-mers in *String*

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Distance between a *k*-mer and a (longer) String

d(GATTCTCA, GCAAAGACGCTGACCAA) = ?

d(*Pattern*, *String*):
minimum distance between *Pattern* and all *k*-mers in *String*

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Distance between a *k*-mer and a (longer) String

d(GATTCTCA, GCAAAGACGCTGACCAA) = ?

d(*Pattern*, *String*):
minimum distance between *Pattern* and all *k*-mers in *String*

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Distance between a *k*-mer and a (longer) String

d(GATTCTCA, GCAAAGACGCTGACCAA) = ?

d(*Pattern*, *String*):
minimum distance between *Pattern* and all *k*-mers in *String*

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Distance between a *k*-mer and a (longer) String

d(GATTCTCA, GCAAAGACGCTGACCAA) = ?

d(*Pattern*, *String*):
minimum distance between *Pattern* and all *k*-mers in *String*

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Distance between a *k*-mer and a (longer) String

d(GATTCTCA, GCAAAGACGCTGACCAA) = 3

d(*Pattern*, *String*):
minimum distance between *Pattern* and all *k*-mers in *String*

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Distance between a *k*-mer and a **Set** of Strings

Distance between a *k-mer* and a *set of strings Dna* = {Dna_1, … , DNA_t}:

$d(k\text{-mer}, Dna) = \sum_{\text{all strings in } Dna} d(k\text{-mer}, Dna_i)$

Pattern = AAA

```
ttaccttAAc 1
gATATctgtc 1
Acggcgttcg 2
ccctAAAgag 0
cgtcAgAggt 1

d(AAA, Dna)= 5
```

A **median string** for the set of strings *Dna*:
a *k-mer* minimizing distance *d*(*k-mer*, *Dna*) over all possible *k*-mers.

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Median String Problem

Median String Problem. Finding a median string.

- **Input:** A set of sequences *Dna* and an integer *k*.
- **Output:** A *k-mer* minimizing distance *d*(*k-mer*, *Dna*) among all *k*-mers.

```
MedianString(Dna, k)
  best-k-mer ← AAA · · · AA
  for each k-mer from AAA · · · AA to TTT · · · TT
    if d(k-mer, Dna) < distance(best-k-mer, Dna)
      best-k-mer ← k-mer
  return(best-k-mer)
```

Runtime: $4^k \cdot n \cdot t \cdot k$ (for *Dna* with *t* sequences of length *n*).

Motif Finding Problem	**Runtime:** $n^t \cdot k \cdot t$
versus	
Median String Problem	**Runtime:** $4^k \cdot n \cdot t \cdot k$

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Motif Finding Problem = Median String Problem

Motif Finding Problem versus Median String Problem	Runtime: $n^t \cdot k \cdot t$ Runtime: $4^k \cdot n \cdot t \cdot k$

Although **MedianString** is much faster than **BruteForceMotifSearch**, it is still slow for large *k*.

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Outline

- **From Implanted Patterns to Regulatory Motifs**
  - Implanting Patterns into Strings
  - Do We Have a Clock Gene?
  - Implanted Motif Problem
  - Motif Finding Problem (Music Break)
  - Median String Problem
  - **Greedy Motif Search**
- How Rolling Dice Helps Us Find Regulatory Motifs
  - Randomized Motif Search
  - How do Bacteria Hibernate?
  - Gibbs Sampling
  - Pseudocounts

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

From Motifs to Profile

Motifs

```
T C G G G G g T T T t t
c C G G t G A c T T a C
a C G G G G A T T T t C
T t G G G G A c T T t t
a a G G G G A c T T C C
T t G G G G A c T T C C
T C G G G G A T T c a t
T C G G G G A T T c C t
T a G G G G A a c T a C
T C G G G T A T a a C C
```

Profile(Motifs)

```
A: .2 .2  0  0  0  0 .9 .1 .1 .1 .3  0
C: .1 .6  0  0  0  0  0 .4 .1 .2 .4 .6
G:  0  0  1  1 .9 .9 .1  0  0  0  0  0
T: .7 .2  0  0 .1 .1  0 .5 .8 .7 .3 .4
```

frequency of nucleotide *i* in column *j*

```
T C G G G G A C T T C C
```

Each column of a profile represents a biased 4-sided **die** with A, C, G, and T on its sides. Thus, a profile corresponds to *k* dice.

What is the probability that *k* rolls of such dice generate a given string ?

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Scoring *k*-mers with a *Profile*

Given the following *Profile*:

A	1/2	7/8	3/8	0	1/8	0
C	1/8	0	1/2	5/8	3/8	0
T	1/8	1/8	0	0	1/4	7/8
G	1/4	0	1/8	3/8	1/4	1/8

The probability of the consensus string:

Pr(AAACCT|*Profile*) = ???

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Scoring *k*-mers with a *Profile*

Given the following *Profile*:

A	1/2	7/8	3/8	0	1/8	0
C	1/8	0	1/2	5/8	3/8	0
T	1/8	1/8	0	0	1/4	7/8
G	1/4	0	1/8	3/8	1/4	1/8

The probability of the consensus string:

Pr(AAACCT|*Profile*) =

1/2 x 7/8 x 3/8 x 5/8 x 3/8 x 7/8 = 0.033646

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Scoring *k*-mers with a *Profile*

Given the following *Profile*:

A	1/2	7/8	3/8	0	1/8	0
C	1/8	0	1/2	5/8	3/8	0
T	1/8	1/8	0	0	1/4	7/8
G	1/4	0	1/8	3/8	1/4	1/8

The probability of another string:

Pr(AAACCT|*Profile*) =

1/2 x 7/8 x 3/8 x 5/8 x 3/8 x 7/8 = 0.033646

Pr(ATACAG|*Profile*) =

1/2 x 1/8 x 3/8 x 5/8 x 1/8 x 1/8 = 0.001602

The closer *k*-mer is to the consensus string, the larger Pr(*k*-mer| *Profile*) is.

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

What is the *Profile*-most probable 6-mer in CTATAAACCTTACAT?

A	1/2	7/8	3/8	0	1/8	0
C	1/8	0	1/2	5/8	3/8	0
T	1/8	1/8	0	0	1/4	7/8
G	1/4	0	1/8	3/8	1/4	1/8

***Profile*-most probable *k*-mer in a sequence**: the *k*-mer with the highest Pr(*k-mer* | *Profile*) among all *k*-mers in this sequence.

6-mer	Prob(6-mer|*Profile*)	
CTATAAACCTTACAT	1/8 x 1/8 x 3/8 x 0 x 1/8 x 0	0
CTATAAACCTTACAT	1/2 x 7/8 x 0 x 0 x 1/8 x 0	0
CTATAAACCTTACAT	1/2 x 1/8 x 3/8 x 0 x 1/8 x 0	0
CTATAAACCTTACAT	1/8 x 7/8 x 3/8 x 0 x 3/8 x 0	0
CTATAAACCTTACAT	1/2 x 7/8 x 3/8 x 5/8 x 3/8 x 7/8	.0336
CTATAAACCTTACAT	1/2 x 7/8 x 1/2 x 5/8 x 1/4 x 7/8	.0299
CTATAAACCTTACAT	1/2 x 0 x 1/2 x 0 1/4 x 0	0
CTATAAACCTTACAT	1/8 x 0 x 0 x 0 x 0 x 1/8 x 0	0
CTATAAACCTTACAT	1/8 x 1/8 x 0 x 0 x 3/8 x 0	0
CTATAAACCTTACAT	1/8 x 1/8 x 3/8 x 5/8 x 1/8 x 7/8	.0004

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

GreedyMotifSearch

Given a set of *k*-mers $Motif_1, \ldots, Motif_{i-1}$ in the first *i-1* sequences in *Dna*:

- form *Profile* from them
- form $Motif_1, \ldots Motif_{i-1}, Motif_i$ by adding the *Profile*-most probable *k*-mer $Motif_i$ in the *i*-th sequence to the growing set of *k*-mers

ITERATE

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Outline

- From Implanted Patterns to Regulatory Motifs
  - Implanting Patterns into Strings
  - Do We Have a Clock Gene?
  - Implanted Motif Problem
  - Motif Finding Problem (Music Break)
  - Median String Problem
  - Greedy Motif Search
- **How Rolling Dice Helps Us Find Regulatory Motifs**
  - **Randomized Motif Search**
  - How do Bacteria Hibernate?
  - Gibbs Sampling
  - Pseudocounts

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

From Motifs to Profile to Motifs to Profile to Motifs…

Given *Motifs*, we can construct

Profile(*Motifs*)

Given an arbitrary *Profile* and a set of sequences *Dna*, we can construct

Motifs(*Profile*, *Dna*)

as a set of *Profile*-most probable *k*-mers in each sequence from *Dna*.

Iterate!

Motifs(*Profile*(*Motifs*(*Profile*(*Motifs*),*Dna*)),*Dna*)),*Dna*)…

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

RandomizedMotifSearch

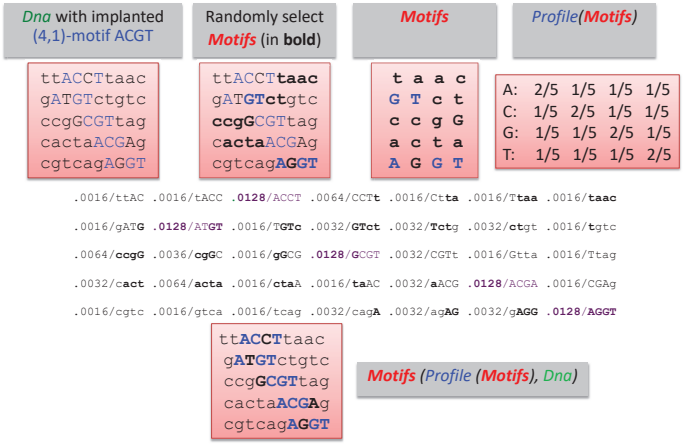
RandomizedMotifSearch(*Dna*, *k*, *t*)
  randomly select *k*-mers *Motifs* = (*Motif*$_1$, … ,*Motif*$_t$) in each string from *Dna*
  bestMotifs ← *Motifs*
  while forever
    Profile ← *Profile*(*Motifs*)
    Motifs ← *Motifs*(*Profile*, *Dna*)
    if *Score*(*Motifs*) < *Score*(*bestMotifs*)
      bestMotifs ← *Motifs*
    else
      return(*bestMotifs*)

How in the World Can This Algorithm Find Anything?

When we form *k*-mers *randomly,* it results in a **uniform** expected *Profile* with every entry 1/4. Such a uniform profile is useless because it does not provide any clues about the implanted motif.

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

RandomizedMotifSearch in Action



Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

How in the World Can This Algorithm Find Anything?

When we select *k*-mers *randomly,* it results in a **uniform** expected *Profile* where every entry is 1/4. Such a uniform profile is useless because it does not provide any clues about the implanted motif.

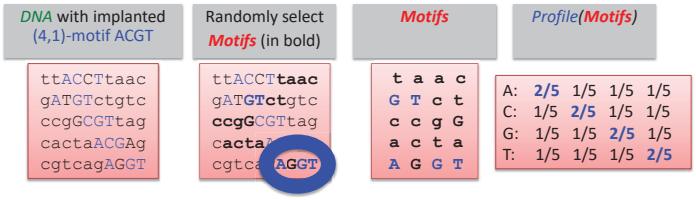
But strings in *Dna* are not truly random since they include implanted motifs!

These implanted motifs result in a **biased** expected *Profile*.

Where does this statistical bias point to?

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

RandomizedMotifSearch in Action



The statistical bias points in the direction of the implanted (4,1)-motif ACGT because one of the randomly selected 4-mers happened to be the implanted pattern AGGT simply by chance.

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

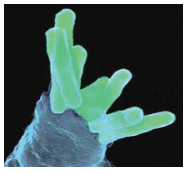
Outline

- **From Implanted Patterns to Regulatory Motifs**
  - Implanting Patterns into Strings
  - Do We Have a Clock Gene?
  - Implanted Motif Problem
  - Motif Finding Problem (Music Break)
  - Median String Problem
  - Greedy Motif Search
- How Rolling Dice Helps Us Find Regulatory Motifs
  - Randomized Motif Search
  - **How do Bacteria Hibernate?**
  - Gibbs Sampling
  - Pseudocounts

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

How Do Tuberculosis Bacteria Hibernate to Hide from Antibiotics?

Tuberculosis (TB) caused 25% of all deaths in 19th century Europe.



TB is caused by *Mycobacterium tuberculosis (MTB)*. MTB strains that resist all antibiotics are now emerging.

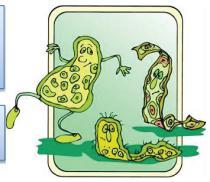
MTB can persist for decades without causing disease: one-third of the world's population carry latent MTB.

How does MTB switch from a latent to active state?

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

TB and Dormancy Survival Regulator (DosR)

"Hibernating" MTB forms metabolically dormant **spores** that can survive in tough conditions (e.g., oxygen shortage).



Oxygen shortage (**hypoxia**) is associated with latent forms of TB.

Using DNA arrays, biologists found 25 genes whose expression significantly changes in hypoxia. They conjectured that expression of these genes is controlled by a transcription factor that was named **Dormancy Survival Regulator** (**DosR**).

Can **MedianString** and **RandomizedMotifSearch** discover a regulatory motif that controls the expression of these genes?

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Analyzing Dormancy Survival Regulator

We have no idea what the length of a motif (k) is.

	MedianString	score	RandomizedMotifSearch	score
k=8:	CATCGGCC	11	CCGACGGG	13
k=9:	GGCGGGGAC	16	CCATCGGCC	16
k=10:	GGTGGCCACC	19	CCATCGGCCC	21
k=11:	GGACTTCCGGC	20	ACCTTCGGCCC	25
k=12:	GGACTTCCGGCC	23	GGACCAACGGCC	28

Length k = 20 motif returned by **RandomizedMotifSearch**:

CGGGGACCTACGGCCCTAGC

How well our predictions match the real binding sites?

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Entropy of a Motif

Profile	A:	.2	.2	0	0	0	0	.9	.1	.1	.1	.3	0
	C:	.1	.6	0	0	0	0	0	.4	.1	.2	.4	.6
	G:	0	0	1	1	.9	.9	.1	0	0	0	0	0
	T:	.7	.2	0	0	.1	.1	0	.5	.8	.7	.3	.4

Every column of the profile matrix is a **probability distribution.**

Entropy of a probability distribution $(p_1, \ldots, p_n)$ is

$$-\sum_j p_j \cdot \log_2(p_j)$$

through all probabilities

Entropy of **uniform** distribution **(¼, ¼, ¼, ¼)** is -4· ¼ $\log_2$(¼)=**2 (maximal entropy)**.
Entropy of a **completely conserved** column like **(1, 0, 0, 0)** is **0 (minimal entropy).**

The entropy varies from **0** to **2**, entropy of the 1st column (0.2, 0.1, 0.0, 0.7):

$$-(0.2 \cdot \log_2 0.2 + 0.1 \cdot \log_2 0.1 + 0.0 \cdot \log_2 0.0 + 0.7 \cdot \log_2 0.7) \approx 1.147$$

Entropy of a motif – sum of entropies of columns of its profile matrix.

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Motif Logo

Profile(Motifs)	A:	.2	.2	0	0	0	0	.9	.1	.1	.1	.3	0
	C:	.1	.6	0	0	0	0	0	.4	.1	.2	.4	.6
	G:	0	0	1	1	.9	.9	.1	0	0	0	0	0
	T:	.7	.2	0	0	.1	.1	0	.5	.8	.7	.3	.4

Consensus(Motifs)	T	C	G	G	G	G	A	T	T	T	C	C

motif logo

The **total height** of 4 nucleotides in each position: **2 - entropy.**

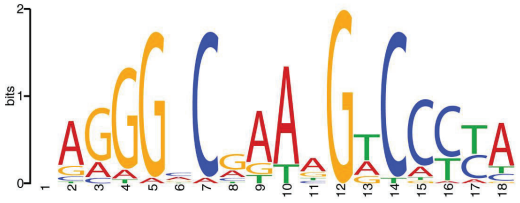
The **relative sizes** of 4 nucleotides in each position: frequencies in the column.

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Predicted vs. Real Dormancy Survival Regulator Binding Site

Length 20 motif returned by **RandomizedMotifSearch**:

CGGGGACCTACGGCCCTAGC



Motif logo of real Dormancy Survival Regulator binding sites

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Outline

- **From Implanted Patterns to Regulatory Motifs**
  - Implanting Patterns into Strings
  - Do We Have a Clock Gene?
  - Implanted Motif Problem
  - Motif Finding Problem (Music Break)
  - Median String Problem
  - Greedy Motif Search
- How Rolling Dice Helps Us Find Regulatory Motifs
  - Randomized Motif Search
  - How do Bacteria Hibernate?
  - **Gibbs Sampling**
  - Pseudocounts

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Gibbs Sampling

RandomizedMotifSearch may replace all k-mers in a single iteration and thus may potentially discard a nearly correct motif.

Gibbs sampling replaces a single k-mer at each iteration and thus moves with more caution in the space of all motifs.

```
ttaccttaac      ttaccttaac
gatatctgtc      gatatctgtc
acggcgttcg  →   acggcgttcg
ccctaaagag      ccctaaagag
cgtcagaggt      cgtcagaggt
```

RandomizedMotifSearch
(may change **all** k-mers in 1 iteration)

```
ttaccttaac      ttaccttaac
gatatctgtc      gatatctgtc
acggcgttcg  →   acggcgttcg
ccctaaagag      ccctaaagag
cgtcagaggt      cgtcagaggt
```

GibbsSampler
(changes a **single** k-mer in 1 iteration)

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

GibbsSampler in Action

Randomly select ***Motifs*** (in **bold**)

```
ttACCTtaac
gATGTctgtc
ccgGCGTtag
cactaACGAg
cgtcagAGGT
```

Randomly remove a k-mer in ***Motifs***

```
ttACCTtaac
gATGTctgtc
----------
cactaACGAg
cgtcagAGGT
```

Motifs matrix

t	a	a	c
G	T	c	t
a	c	t	a
A	G	G	T

Choose a new st… in the deleted se… on rolling…

ITERATE

Calculate the probabilities of all k-mers in the deleted string **ccgG**CGTtag

Profile matrix

A:	2/4	1/4	1/4	1/4
C:	0	1/4	1/4	1/4
G:	1/4	1/4	1/4	0
T:	1/4	1/4	1/4	2/4

0 (**ccgG**) **0** (**cgGC**) **0** (**gGCG**) **1/128** (**GCGT**) **0** (CGTt) **1/256** (GTta) **0** (Ttag)

Roll a 7-sided die with probabilities of sides proportional to probabilities of k-mers

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Gibbs Sampler Outline

1. Form *Motifs* by randomly selecting a *k*-mer in each sequence
2. Randomly choose one of selected *k*-mers (from *RemovedSequence)* and remove it from *Motifs*.
3. Create *Profile* from the remaining *k*-mers in *Motifs*.
4. For each *k*-mer in *RemovedSequence*, calculate *Pr(k-mer | Profile)* resulting in *n-k+1* probabilities:
$$p_1, p_2, \ldots, p_{n-k+1}.$$
5. Roll a die (with *n-k+1* sides) where probability of ending up at side *i* is proportional to p_i.
6. Choose a new starting position based on rolling the die. Add the *k*-mer starting at this position in *RemovedSequence* to *Motifs*.
7. **Repeat steps 2-6.**

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Gibbs Sampler in Action

Randomly select ***Motifs*** (in **bold**)

```
ttACCTtaac
gATGTctgtc
ccgGCGTtag
cactaACGAg
cgtcagAGGT
```

Randomly remove a *k*-mer in ***Motifs***

```
ttACCTtaac
gATGTctgtc
----------
cactaACGAg
cgtcagAGGT
```

Motifs matrix

t	a	a	c
G	T	c	t
---	---	---	---
a	c	t	a
A	G	G	T

Count matrix

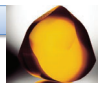
A:	2	1	1	1
C:	0	1	1	1
G:	1	1	1	0
T:	1	1	1	2

Calculate the probabilities of all *k*-mers in the deleted string **ccgGCGTtag**

Profile matrix

A:	2/4	1/4	1/4	1/4
C:	0	1/4	1/4	1/4
G:	1/4	1/4	1/4	0
T:	1/4	1/4	1/4	2/4

0 (ccgG) 0 (cgGC) 0 (gGCG) 1/128 (GCGT) 0 (CGTt) 1/256 (GTta) 0 (Ttag)



Should we roll a two-sided die???

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Cromwell's Rule

Oliver Cromwell made an appeal to the Church of Scotland, urging them to see the error of their alliance with the English royalists:

I beseech you, in the bowels of Christ, think it possible that you may be mistaken.

Cromwell's rule: the use of probabilities of 0 or 1 should be avoided.

We should leave a little probability that the sun will not rise tomorrow.

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Outline

- **From Implanted Patterns to Regulatory Motifs**
  - Implanting Patterns into Strings
  - Do We Have a Clock Gene?
  - Implanted Motif Problem
  - Motif Finding Problem (Music Break)
  - Median String Problem
  - Greedy Motif Search
- How Rolling Dice Helps Us Find Regulatory Motifs
  - Randomized Motif Search
  - How do Bacteria Hibernate?
  - Gibbs Sampling
  - **Pseudocounts**

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Laplace's Rule of Succession

In small datasets, there is always a chance that a possible event does not occur (e.g., zeroes in *Count* matrix).

Randomized algorithms do not like zeroes and introduce **pseudocounts** that inflate the probabilities of rare events and eliminate empirical zero-frequencies.

Laplace calculated the probability that the sun will not rise tomorrow, given that it has risen every day for the past 5000 years (1 in 1826251)

His estimate was ridiculed because his opponents failed to understand its importance.

If we repeat an experiment that can result in a success or failure ***n*** times and get ***s*** successes, what is the probability that the next repetition will be a success?

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Pseudocounts

If $X_1, \ldots, X_{n+1}$ are *conditionally* independent random boolean variables (failure 0 or success 1) then:

$$\Pr(X_{n+1}=1 | X_1+\ldots+X_n=s) = (s+1)/(n+2)$$

Since we have the prior knowledge that both success and failure are possible, we essentially made ***n+2*** rather than ***n*** observations since we (implicitly) observed one success and one failure before we even started the experiments.

Thus, we have made ***n+2*** observations (known as **pseudocounts**) with ***s+1*** successes.

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

What would Laplace Do to Get Rid of Zeroes?

```
ttACCTtaac
gATGTctgtc
ccgGCGTtag
cactaACGAg
cgtcagAGGT
```

```
ttACCTtaac
gATGTctgtc
----------
cactaACGAg
cgtcagAGGT
```

t	a	a	c
G	T	c	t
---	---	---	---
a	c	t	a
A	G	G	T

Choose a new starting position in the deleted sequence based on rolling the die

Update *Count* Matrix with pseudocounts

A:	2+1	1+1	1+1	1+1
C:	0+1	1+1	1+1	1+1
G:	1+1	1+1	1+1	0+1
T:	1+1	1+1	1+1	2+1

Recalculate the probabilities of all 4-mers in the deleted string **ccgGCGTtag**

Update *Profile* Matrix

A:	3/8	2/8	2/8	2/8
C:	1/8	2/8	2/8	2/8
G:	2/8	2/8	2/8	1/8
T:	2/8	2/8	2/8	3/8

1/1024 (ccgG) 2/1024 (cgGC) 2/1024 (gGCG) **6/1024 (GCGT)** 3/1024 (CGTt) 4/1024 (GTta) 2/1024 (Ttag)

Roll a 7-sided rather than 2-sided die!

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Gibbs Sampler Continues

Randomly select ***Motifs*** (in bold)

```
ttACCTtaac
gATGTctgtc
ccgGCGTtag
cactaACGAg
cgtcagAGGT
```

Randomly remove a *k*-mer in ***Motifs***

```
----------
gATGTctgtc
ccgGCGTtag
cactaACGAg
cgtcagAGGT
```

Motifs matrix

---	---	---	---
G	T	c	t
G	C	G	T
a	c	t	a
A	G	G	T

Choose a new starting position in the deleted sequence based on rolling the die

Pseudocount Matrix

A:	2+1	0+1	0+1	1+1
C:	0+1	2+1	1+1	0+1
G:	2+1	1+1	2+1	0+1
T:	0+1	1+1	1+1	3+1

Calculate the probabilities of all *k*-mers in the deleted string **ttACCTtaac**

Construct *Profile* using Laplace's Rule

A:	3/8	1/8	1/8	2/8
C:	1/8	3/8	2/8	1/8
G:	3/8	2/8	3/8	1/8
T:	1/8	2/8	2/8	4/8

2/4096 (ttAC) 2/4096 (tACC) 72/4096 (ACCT) 24/4096 (CCTt) 8/4096 (CTta) 4/4096 (Ttaa) 1/4096 (taac)

Roll a 7-sided die

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

Gibbs Sampler Continues and Continues

Randomly select *Motifs* (in bold)

```
ttACCTtaac
gATGTctgtc
ccgGCGTtag
cactaACGAg
```

Randomly remove a *k*-mer in ***Motifs***

```
ttACCTtaac
gATGTctgtc
ccgGCGTtag
----------
```

Motifs matrix

```
A  C  C  T
G  T  c  t
G  C  G  T
-----------
```

We have already captured four out of five correct 4-mers and will almost surely find all implanted motifs at one of the next iterations.

Calculate the probabilities of all *k*-mers in the deleted string `ttACCTtaac`

Create *Profile* using Laplace's Rule

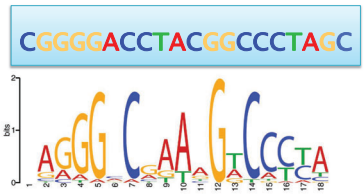
```
A: 3/8 1/8 1/8 1/8
C: 1/8 3/8 3/8 1/8
G: 3/8 2/8 3/8 1/8
T: 1/8 2/8 1/8 5/8
```

2/4096 (ttAC) 2/4096 (tACC) 72/4096 (ACCT) 24/4096 (CCTt) 8/4096 (CTta) 4/4096 (Ttaa) 1/4096 (taac)

Roll a 7-sided die

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

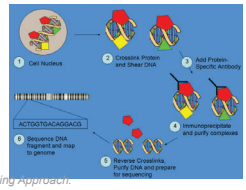
The Challenge of Predicting Degenerative Motifs



Even the best motif finding algorithms fail to find some highly degenerative motifs, e.g., (15,5)-motifs are nearly impossible to find if they are randomly inserted in **1000**-nucleotide long random strings.

We can still solve the Implanted Motif Problem but it will find irrelevant 15-mers

ChIP Sequencing provides an experimental solution to this problem by reducing the length of sequences to search for motifs. For example, if (15,5)-motif is inserted in **100**-nucleotide (rather than **1000**) long random strings, it will be easier to find!



Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2013 Compeau and Pevzner.

CSE4065 – Fall 2022
Computational Genomics
Genome Assemble

Slides are from:
Bioinformatics Algorithms: An Active Learning Approach

String Composition Problem

- Assume we are given a string and we want to obtain its k-mer composition
  - The collection of all k-mers forming the string (including repeated *k*-mers)
- Representation: $Composition_k(Text)$
- What is $Composition_3$(TATGGGGTGC)?
- $Composition_3$(TATGGGGTGC) = {ATG, GGG, GGG, GGT, GTG, TAT, TGC, TGG}.
- *k*-mers are listed in **lexicographic order** rather than in the order of their appearance in TATGGGGTGC.
- The correct ordering of the reads is unknown when they are generated.

The String Reconstruction Problem – Example I

- Consider the following example of a 3-mer composition:
  AAT ATG GTT TAA TGT
- Which 3-mer can be used to start the string?
- The string should start with TAA because there is no 3-mer ending in TA.
- Which 3-mer should be selected next?
- The next 3-mer in the string should start with AA. There is only one 3-mer satisfying this condition, AAT

The String Reconstruction Problem – Example I cnt.

- Consider the following example of a 3-mer composition:
  AAT ATG GTT TAA TGT
- The first and second 3-mers are:
- TAA AAT
- How to generate the string?

TAA
AA**T**
AT**G**
TG**T**
GT**T**
TAATGTT

The String Reconstruction Problem – Example II

- Consider the following example of a 3-mer composition:
  AAT ATG ATG ATG CAT CCA GAT GCC GGA GGG GTT TAA TGC TGG TGT
- Construct the string: **TAATGCCATGGGATGTT**

TAA
AAT
ATG
TGC
GCC
CCA
CAT
ATG
TGG
GGG
GGA
GAT
ATG
TGT
GTT

The String Reconstruction Problem with Repeating k-mers

- How can we solve the problem if we have k-mers that repeat multiple times?
- Repeats in a genome necessitate some way of looking ahead to see the correct assembly in advance.
- We can use graphs to solve this problem.
  - Find a Hamiltonian path in the **overlap graph** - find a path visiting every *node* exactly once
  - Find an Eulerian path in the **de Bruijn graph** - find a path visiting every *edge* exactly once

Eulerian graphs

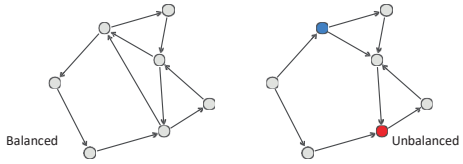
- Euler's Theorem states that the following two conditions are sufficient to guarantee that an arbitrary graph is Eulerian.
- Any Eulerian graph must be balanced:
  - The number of incoming edges at any node must be equal to the number of outgoing edges at that node.
  - We define the **indegree** and **outdegree** of a node v (denoted $IN(v)$ and $OUT(v)$) as the number of edges leading into and out of v.
  - A node v is **balanced** if $IN(v) = OUT(v)$
  - A graph is **balanced** if all its nodes are balanced.
- Any Eulerian graph must be strongly connected:
  - A directed graph is **strongly connected** if it is possible to reach any node from every other node.

Euler's Theorem

Euler's Theorem: Every balanced, strongly connected directed graph is Eulerian.

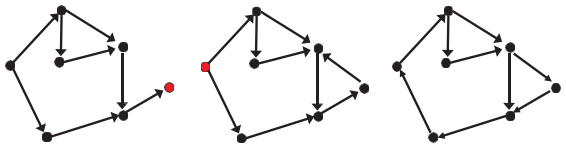
Euler's Theorem for Directed Graphs

Indegree: Number of edges leading into a node.
Outdegree: Number of edges leading out of a node.
Balanced graph: Every node has indegree equal to outdegree.



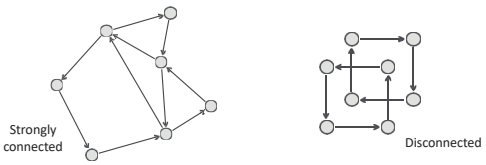
A Graph is **Eulerian** if It Contains an Eulerian Cycle.

Is this graph Eulerian?



Euler's Theorem for Directed Graphs

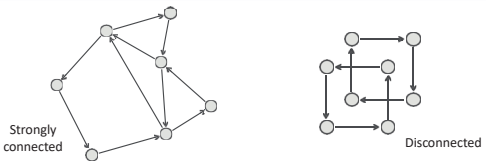
Strongly connected graph: A graph where it is possible to reach every node from any other node.



Euler's Theorem for Directed Graphs

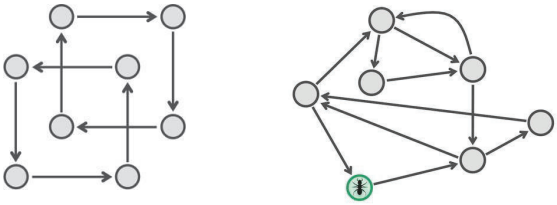
Strongly connected graph: A graph where it is possible to reach every node from any other node.

Euler's Theorem: Every balanced, strongly connected graph has an Eulerian cycle.

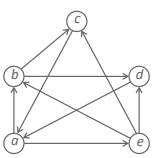


Strongly connected graphs

- A directed graph is **strongly connected** if it is possible to reach any node from every other node.
- Are the following graphs strongly connected and balanced?



Two Ways to Represent Graphs Computationally

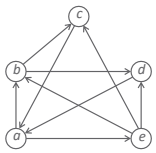


Adjacency Matrix

	a	b	c	d	e
a	0	1	0	0	1
b	0	0	1	1	0
c	1	0	0	0	0
d	1	0	0	0	0
e	0	1	1	1	0

Adjacency matrix: $A_{i,j} = 1$ if there is an edge connecting node i to node j; $A_{i,j} = 0$ otherwise.

Two Ways to Represent Graphs Computationally



Adjacency Matrix

	a	b	c	d	e
a	0	1	0	0	1
b	0	0	1	1	0
c	1	0	0	0	0
d	1	0	0	0	0
e	0	1	1	1	0

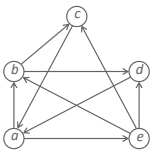
Adjacency List

a	b, e
b	c, d
c	a
d	a
e	b, c, d

Adjacency matrix: $A_{i,j} = 1$ if there is an edge connecting node i to node j; $A_{i,j} = 0$ otherwise.

Adjacency list: Dictionary; "key" node i;"value" is list of nodes that i is connected to.

How Changing a Graph Changes Adjacency Matrix/List

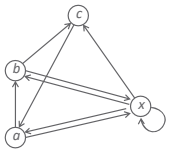


Adjacency Matrix

	a	b	c	d	e
a	0	1	0	0	1
b	0	0	1	1	0
c	1	0	0	0	0
d	1	0	0	0	0
e	0	1	1	1	0

Adjacency List

a	b, e
b	c, d
c	a
d	a
e	b, c, d



Adjacency Matrix

	a	b	c	x
a	0	1	0	1
b	0	0	1	1
c	1	0	0	0
x	1	1	1	1

Adjacency List

a	b, x
b	c, x
c	a
x	a, b, c, x

Similar Problems with Different Fates

Hamiltonian Cycle Problem — ***NP-Complete***

Input: a network with n nodes.
Output: "Yes" if there is a cycle visiting every ***node*** in the network; "No" otherwise.

Eulerian Cycle Problem — ***Polynomial***

Input: a network with n nodes.
Output: "Yes" if there is a cycle visiting every ***edge*** in the network; "No" otherwise.

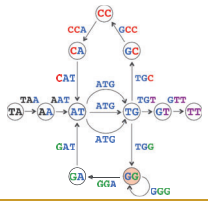
Euler's Theorem is "Constructive"

Key Point: This is a "constructive proof", meaning it implies an algorithm for finding an Eulerian cycle.

```
EulerianCycle(Graph)
    v ← arbitrary node in Graph
    Cycle ← randomly walk starting at v (don't revisit edges) until cycle
    while there are unexplored edges in Graph
        newStart ← node in Cycle with unexplored edges
        Cycle' ← cycle formed by traversing Cycle (starting at newStart)
                 and then randomly walking
        Cycle ← Cycle'
    return Cycle
```

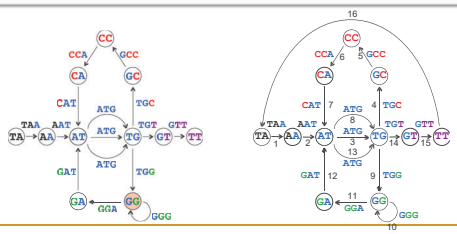

From Eulerian Cycles to Paths

STOP: How do we find an Eulerian *path* in this graph?



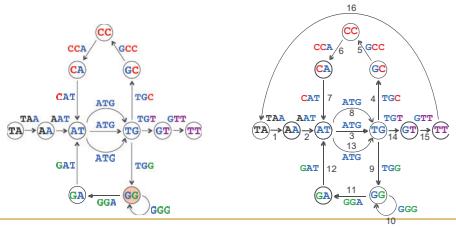
From Eulerian Cycles to Paths

Answer: Simply draw an edge connecting the two unbalanced nodes to form a balanced graph. Eulerian cycle on right = Eulerian path on left.



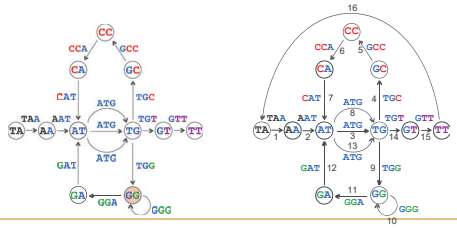
From Eulerian Cycles to Paths

STOP: Why will the augmented de Bruijn graph on the right be balanced for any collection of strings *Patterns*?



From Eulerian Cycles to Paths

Answer: For every node v in de Bruijn graph, Indegree(v) and Outdegree(v) are both equal to # of patterns containing v as prefix/suffix, respectively.



We Can Assemble a Genome!

String Reconstruction Problem: *Reconstruct a string from its k-mer composition.*
Input: An integer *k* and a collection *Patterns* of *k*-mers.
Output: A string *Text* with *k*-mer composition equal to *Patterns* (if such a string exists).

1. Form de Bruijn graph *G* from *Patterns*.

We Can Assemble a Genome!

String Reconstruction Problem: *Reconstruct a string from its k-mer composition.*
Input: An integer *k* and a collection *Patterns* of *k*-mers.
Output: A string *Text* with *k*-mer composition equal to *Patterns* (if such a string exists).

1. Form de Bruijn graph *G* from *Patterns*.
2. Add edge to make modified graph *G'* balanced.

We Can Assemble a Genome!

String Reconstruction Problem: *Reconstruct a string from its k-mer composition.*
Input: An integer *k* and a collection *Patterns* of *k*-mers.
Output: A string *Text* with *k*-mer composition equal to *Patterns* (if such a string exists).

1. Form de Bruijn graph *G* from *Patterns*.
2. Add edge to make modified graph *G'* balanced.
3. Find Eulerian cycle in *G'*.

We Can Assemble a Genome!

String Reconstruction Problem: *Reconstruct a string from its k-mer composition.*
Input: An integer *k* and a collection *Patterns* of *k*-mers.
Output: A string *Text* with *k*-mer composition equal to *Patterns* (if such a string exists).

1. Form de Bruijn graph *G* from *Patterns*.
2. Add edge to make modified graph *G'* balanced.
3. Find Eulerian cycle in *G'*.
4. Infer Eulerian path in *G* from this cycle.

We Can Assemble a Genome!

String Reconstruction Problem: *Reconstruct a string from its k-mer composition.*
Input: An integer *k* and a collection *Patterns* of *k*-mers.
Output: A string *Text* with *k*-mer composition equal to *Patterns* (if such a string exists).

1. Form de Bruijn graph *G* from *Patterns*.
2. Add edge to make modified graph *G'* balanced.
3. Find Eulerian cycle in *G'*.
4. Infer Eulerian path in *G* from this cycle.
5. Convert "genome path" into string *Text*.

de Bruijn Assembly in Research

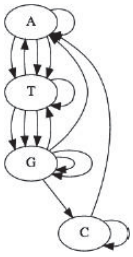
An Eulerian path approach to DNA fragment assembly | PNAS
https://www.pnas.org/content/98/17/9748
by PA Pevzner - 2001 - Cited by 1230 - Related articles
Aug 14, 2001 - Pavel A. **Pevzner**, Haixu Tang, and Michael S. Waterman ... Our new **euler** algorithm is very different from this natural ..... **Google Scholar**.

Velvet: Algorithms for de novo short read assembly using de Bruijn ...
https://www.ncbi.nlm.nih.gov/pmc/articles/PMC2336801/
by DR Zerbino - 2008 - Cited by 7502 - Related articles
Mar 17, 2008 - **Velvet** represents a new approach to assembly that can leverage very short reads in ... The **EULER** assembler (Pevzner et al. ... s 2001 **paper**.
Abstract · Results · Discussion · Methods

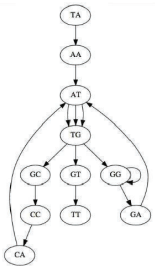
SPAdes: A New Genome Assembly Algorithm and Its ... - NCBI - NIH
https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3342519/
by A Bankevich - 2012 - Cited by 4624 - Related articles
We present the **SPAdes assembler**, introducing a number of new algorithmic ... De Bruijn graphs, PDBGs, and several other graphs in this **paper** are special ...
Assembly in SPAdes: An ... · Stage 2: Utilization of ... · Additional Details on ...

Longer Reads Means More Accuracy

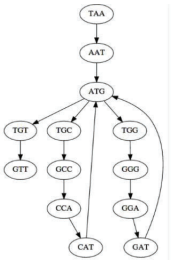
DeBruijn2(TAATGCCATGGGATGTT)



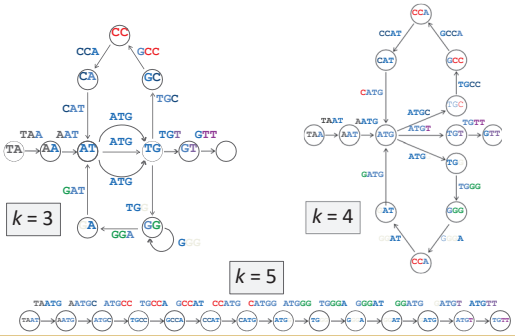
DeBruijn3(TAATGCCATGGGATGTT)



DeBruijn4(TAATGCCATGGGATGTT)



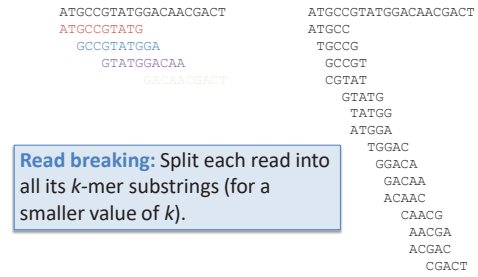
Longer Reads Means More Accuracy



Complications of Genome Assembly

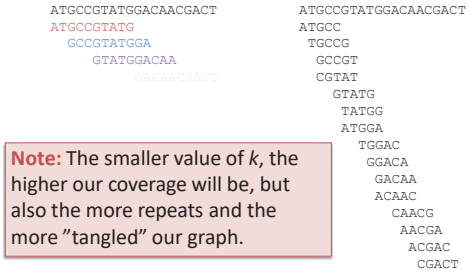
1. Reads have **imperfect "coverage"** of the underlying genome – there may be some regions that are not covered by any reads.
2. Sequencing machines are **error-prone**.
3. Exact multiplicities of *k*-mers are **unknown**.
4. DNA is **double-stranded** (and may consist of **multiple chromosomes**).

Boosting Coverage through Read Breaking



Read breaking: Split each read into all its *k*-mer substrings (for a smaller value of *k*).

Boosting Coverage through Read Breaking



Note: The smaller value of *k*, the higher our coverage will be, but also the more repeats and the more "tangled" our graph.

How Do We Compare Biological Sequences?

Dynamic Programming and Divide-and Conquer Algorithms

Phillip Compeau and Pavel Pevzner.
Bioinformatics Algorithms: An Active Learning Approach

©2018 by Compeau and Pevzner. All rights reserved.

How Do We Compare Biological Sequences

- **The Alignment Game and the Longest Common Subsequence**
- The Manhattan Tourist Problem
- Dynamic Programming and Backtracking Pointers
- From Manhattan to the Alignment Graph
- From Global to Local Alignment
- Penalizing Insertions and Deletions in Sequence Alignment
- Space-Efficient Sequence Alignment
- Multiple Sequence Alignment

Sequence Alignment

- The Hamming distance, which counts mismatches in two strings, rigidly assumes that we align the *i*-th symbol of one sequence against the *i*-th symbol of the other.
- However, since biological sequences are subject to insertions and deletions, it is often the case that the *i*-th symbol of one sequence corresponds to a symbol at a completely different position in the other sequence.
- The goal is to find the most appropriate correspondence of symbols.

Example

- ATGCATGC and TGCATGCA have no matching positions, and so their Hamming distance is equal to 8:
- **ATGCATGC**
  TGCATGCA
- These strings have seven matching positions if we align them differently:
- **ATGCATGC-**
  - TGCATGCA
- Strings ATGCTTA and TGCATTAA have more subtle similarities:
- **ATGC -TTA-**
  - TGCATTAA

Good alignment: Matches as many symbols as possible.

How to find matched symbols?

- Maximizing the number of matched symbols in two strings: You have two choices.
  - You can remove the first symbol from each sequence,
    - you earn a point if the symbols match
  - You can remove the first symbol from either of the two sequences
    - you earn no points but may set yourself up to earn more points in later moves.
- Your goal is to maximize the number of points.

The Alignment Game

```
A T G T T A T A
A T C G T C C
```

Alignment Game (maximizing the number of points):

- Remove the 1st symbol from each sequence
  - 1 point if the symbols match, 0 points if they don't match
- Remove the 1st symbol from one of the sequences
  - 0 points

The Alignment Game

```
A T G T T A T A
A T C G T C C
+1
```


The Alignment Game

```
A T G T T A T A
A T C G T C C
+1+1
```


The Alignment Game

```
A T - G T T A T A
A T C G T C C
+1+1
```


The Alignment Game

```
A T - G T T A T A
A T C G T C C
+1+1  +1
```


The Alignment Game

```
A T - G T T A T A
A T C G T C C
+1+1  +1+1
```


The Alignment Game

```
A T - G T T A T A
A T C G T - C C
+1+1  +1+1
```


The Alignment Game

```
A T - G T T A T A
A T C G T - C C
+1+1  +1+1
```

The Alignment Game

```
A T - G T T A T A
A T C G T - C - C
+1+1  +1+1
```


The Alignment Game

```
A T - G T T A T A
A T C G T - C - C
+1+1  +1+1      =4
```


Sequence Alignment: Definition

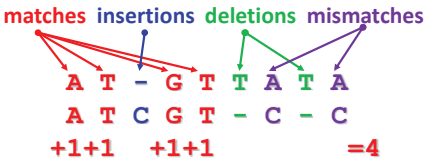
- Define an alignment of sequences *v* and *w* as a two-row matrix such that
  - the first row contains the symbols of *v* (in order)
  - the second row contains the symbols of *w* (in order)
  - space symbols (called gap symbols and shown as dashes) may be interspersed throughout both strings
    - two space symbols are not aligned against each other.
- The alignment of ATGTTATA and ATCGTCC:

  A T - G T T A T A
  A T C G T - C - C
- An alignment presents one possible scenario by which *v* could have evolved into *w*.
- The alignment above has four matches, two mismatches, one insertion, and two deletions.

Sequence Alignment: Definition – cnt.

- Matches: Columns containing the same letter in both rows
- Mismatches: Columns containing different letters which represent single-nucleotide substitutions.
- Indels: Columns containing a space symbol:
  - Insertion: Column containing a space symbol in the top row of the alignment
    - implies the insertion of a symbol when transforming *v* into *w*;
  - Deletion: Column containing a space symbol in the bottom row of the alignment
    - indicates the deletion of a symbol when transforming *v* into *w*.
- The matches in an alignment of two strings define a **common subsequence** of the two strings, or a sequence of symbols appearing in the same order (although not necessarily consecutively) in both strings.

What Is the Sequence Alignment?



Alignment of two sequences is a two-row matrix:

1st row: symbols of the 1st sequence (in order) interspersed by "-"
2nd row: symbols of the 2nd sequence (in order) interspersed by "-"

Longest Common Subsequence

```
A T - G T T A T A
A T C G T - C - C
```

Matches in alignment of two sequences (**ATGT**) form their **Common Subsequence**

Longest Common Subsequence Problem: Find a longest common subsequence of two strings.
- **Input:** Two strings.
- **Output:** A longest common subsequence of these strings.

Longest Common Subsequence

- **ATGT** is a common subsequence of **ATGT**TATA and **AT**C**GT**CC.
- An alignment of two strings maximizing the number of matches therefore corresponds to a **longest common subsequence (LCS)** of these strings.
- Note that two strings may have more than one longest common subsequence.
- **Longest Common Subsequence Problem:** *Find a longest common subsequence of two strings.*
  Input: Two strings.
  Output: A longest common subsequence of these strings.

Question

- Find all longest common subsequences of the strings ACTGCA and CATCGC
- CTGC
- ATGC
- ACGC

Solution of the Longest Common Subsequence Problem

- There are many different ways to solve this problem
- We will start with dynamic programming
- To apply dynamic programming to the Longest Common Subsequence Problem, we will start with Manhattan Tourist Problem

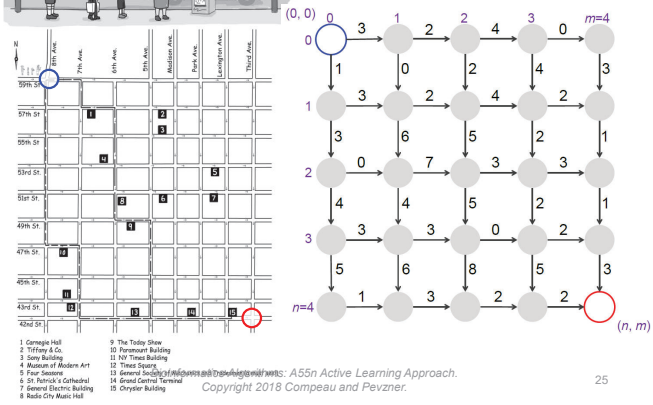
How Do We Compare Biological Sequences

- The Alignment Game and the Longest Common Subsequence
- **The Manhattan Tourist Problem**
- Dynamic Programming and Backtracking Pointers
- From Manhattan to the Alignment Graph
- From Global to Local Alignment
- Penalizing Insertions and Deletions in Sequence Alignment
- Space-Efficient Sequence Alignment
- Multiple Sequence Alignment

Manhattan Tourist Problem

- You are a tourist in Midtown Manhattan
- You want to see as many sights as possible on your way from the corner of 59th Street and 8th Avenue to the corner of 42nd Street and 3rd Avenue
- You are short on time, and at each intersection, you can only move south (↓) or east (→).
- You can choose from many different paths through the map, but no path will visit all the sights.
- The challenge of finding a legal path through the city that visits the most sights is called the **Manhattan Tourist Problem**.

From Manhattan to a Grid Graph



Problem Representation

- Represent the map of Manhattan as a directed graph *ManhattanGraph*
- model each intersection as a node
- represent each city block between two intersections as a directed edge indicating the legal direction of travel (↓ or →)
- Assign each directed edge a **weight** equal to the number of attractions along the corresponding block.
- The starting node is called the **source node** and the ending node is called the **sink node**.
- Adding the weights along a path from the source to the sink yields the number of attractions along that path
- To solve the Manhattan Tourist Problem, find a **maximum-weight path** connecting the source to the sink (also called a **longest path**) in *ManhattanGraph*.

Manhattan Tourist Problem

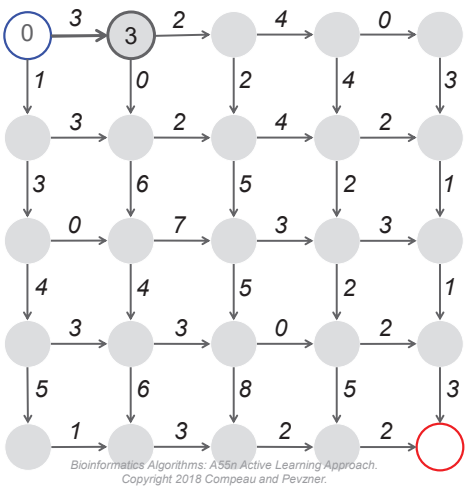
Manhattan Tourist Problem: Find a longest path in a rectangular city grid.

- **Input:** A weighted rectangular grid.
- **Output:** A longest path from the source to the sink in the grid.

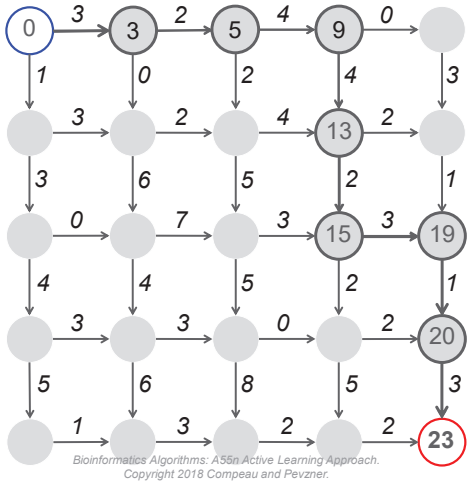
Solution?

- Applying a brute force approach to the Manhattan Tourist Problem is impractical because the number of paths is huge.
- A sensible greedy approach would choose between the two possible directions at each node (→ or ↓) according to how many attractions you would see if you moved only one block south versus moving one block east.
- Unfortunately, this greedy strategy may miss the longest path in the long run.

Greedy algorithm?



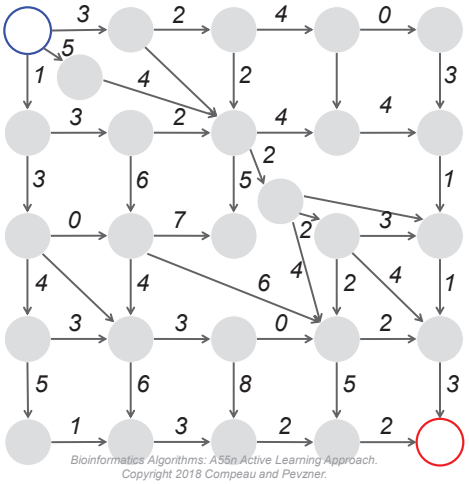
Greedy algorithm?



Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

Search for Longest Paths in a Directed Graph

From a regular to an irregular grid



Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

Do You See a Connection between the Manhattan Tourist and the Alignment Game?

Longest Path in a Directed Graph Problem: Find a longest path between two nodes in an edge-weighted directed graph.

- **Input:** An edge-weighted directed graph with source and sink nodes.
- **Output:** A longest path from source to sink in the directed graph.

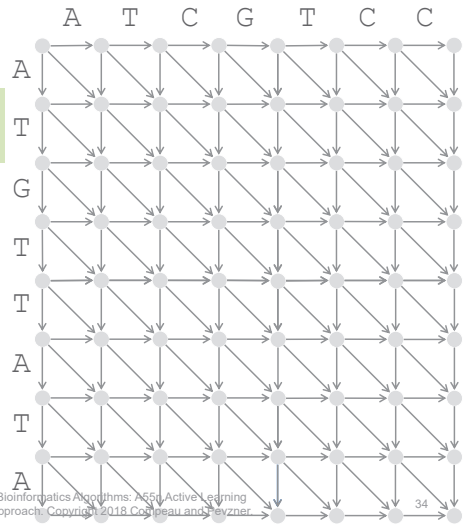
```
A T - G T T A T A
A T C G T - C - C
↘ ↘ → ↘ ↘ ↓ ↘ ↓ ↘
```

Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

?
alignment → path

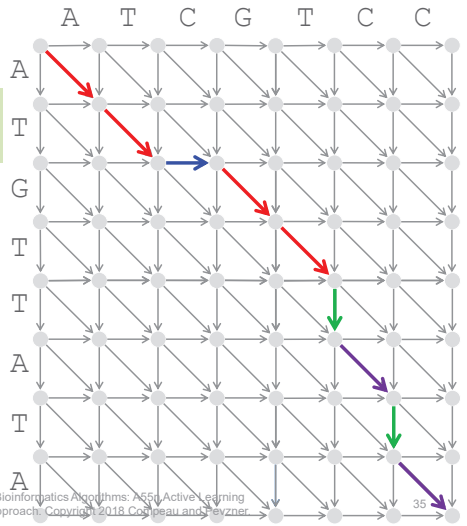
```
A T - G T T A T A
A T C G T - C - C
↘ ↘ → ↘ ↘ ↓ ↘ ↓ ↘
```



Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

?
alignment → path

```
A T - G T T A T A
A T C G T - C - C
↘ ↘ → ↘ ↘ ↓ ↘ ↓ ↘
```

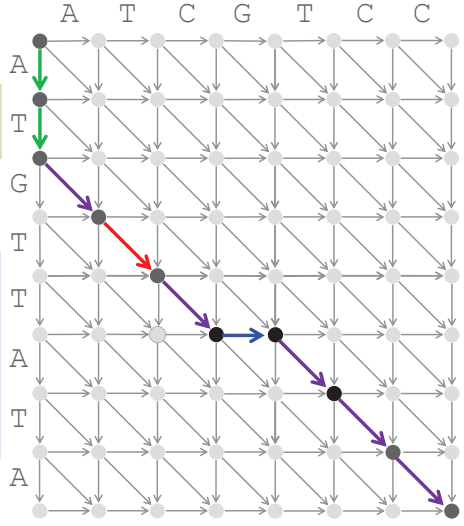


Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

?
path → alignment

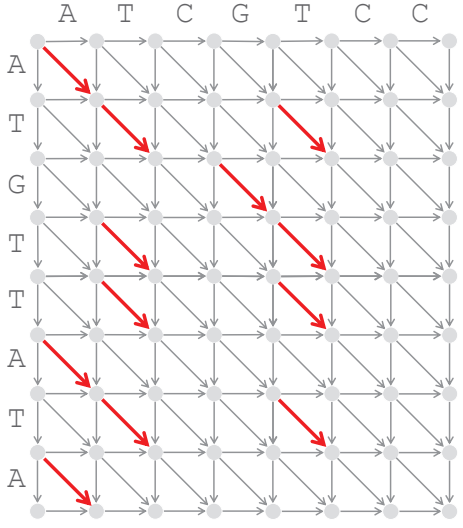
```
A T G T T - A T A
- - A T C G T C C
↓ ↓ ↘ ↘ ↘ → ↘ ↘ ↘
```

highest-scoring alignment
=
longest path in a properly built Manhattan



How to built a Manhattan for the Alignment Game and the Longest Common Subsequence Problem?

Diagonal red edges correspond to matching symbols and have scores 1

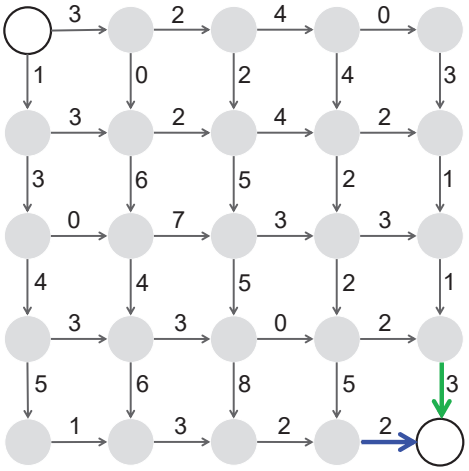


How Do We Compare Biological Sequences

- The Alignment Game and the Longest Common Subsequence
- The Manhattan Tourist Problem
- **Dynamic Programming and Backtracking Pointers**
- From Manhattan to the Alignment Graph
- From Global to Local Alignment
- Penalizing Insertions and Deletions in Sequence Alignment
- Space-Efficient Sequence Alignment
- Multiple Sequence Alignment

Bioinformatics Algorithms: A55n Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

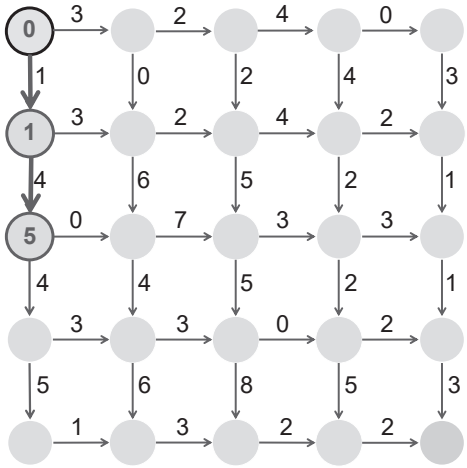
There are only 2 ways to arrive to the sink: by moving **South ↓** or by moving **East →**

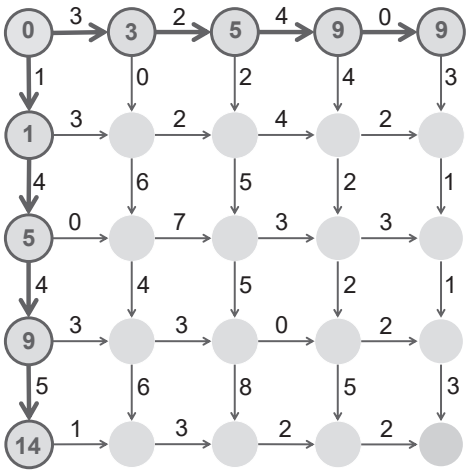


South or East?

SouthOrEast(*n*,*m*)
  if *n*=0 and *m*=0
    return 0
  if *n*>0 and *m*>0
    x ← **SouthOrEast**(*n*-1,*m*)+weight of edge "↓"into (*n*,*m*)
    y ← **SouthOrEast**(*n*,*m*-1)+ weight of edge "→"into (*n*,*m*)
    return max{*x*,*y*}
  return -infinity

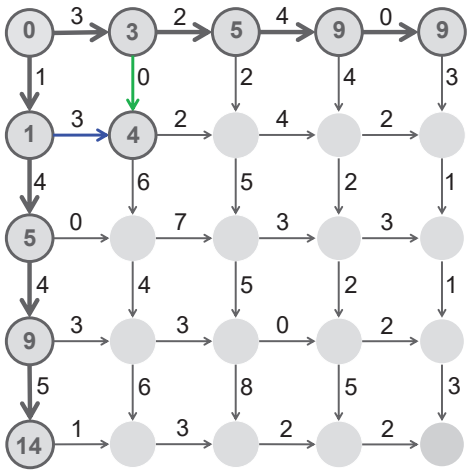
Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.



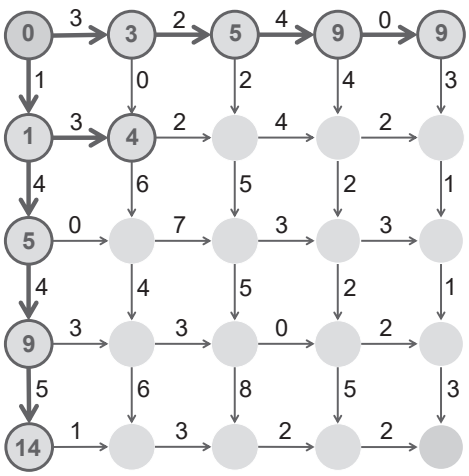


South or East?

1+3 > 3+0

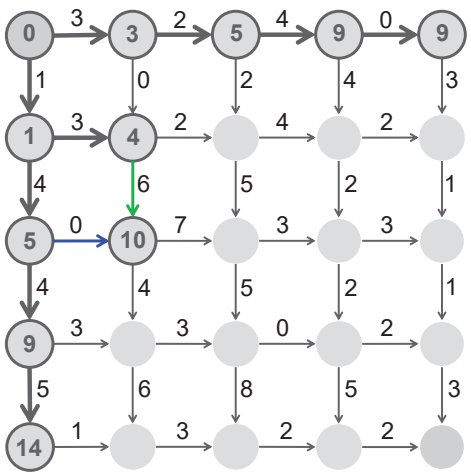


We arrived to (1,1) by the **bold** edge:

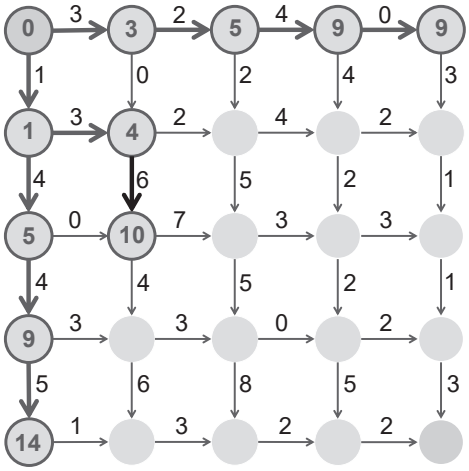


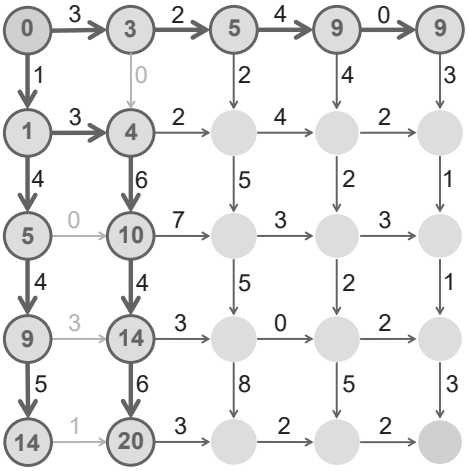
South or East?

5+0 < 4+6



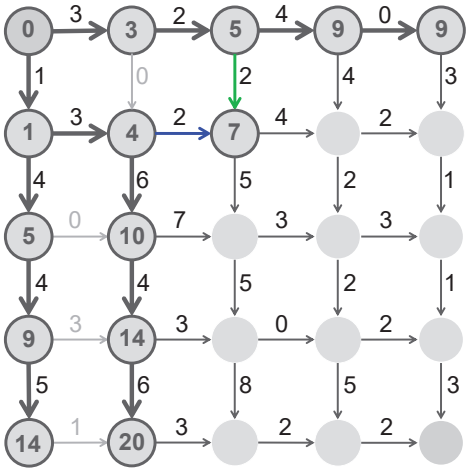
We arrived to (2,1) by the **bold** edge:





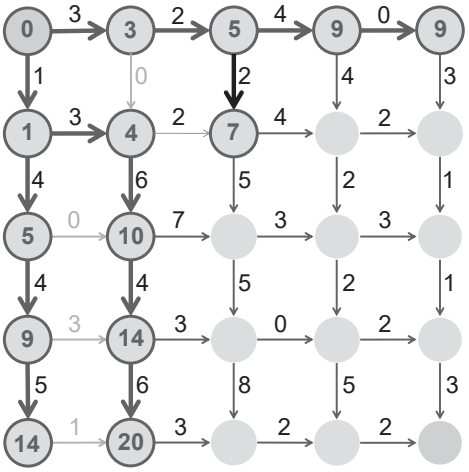
South or East?

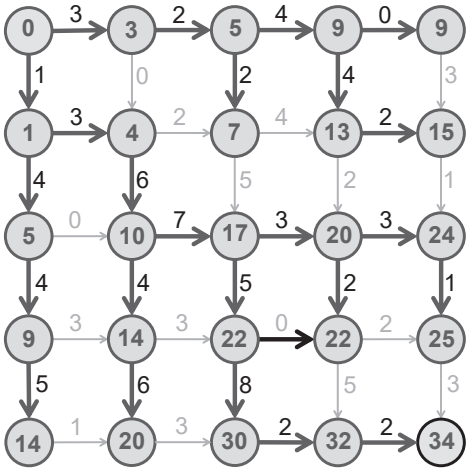
5+2 > 4+2



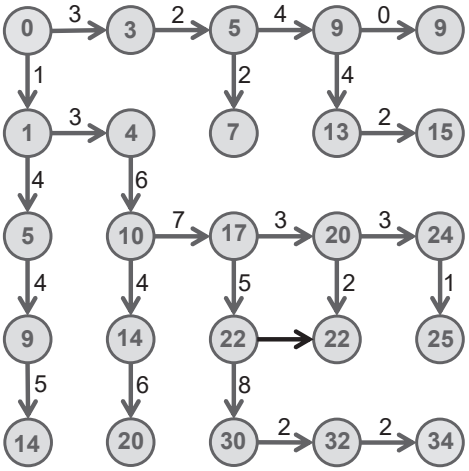
South or East?

5+2 > 4+2





Backtracking pointers: the best way to get to each node



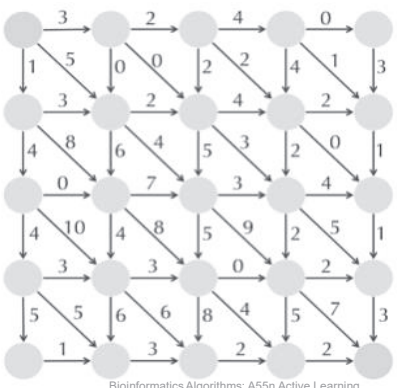
Dynamic Programming Recurrence

$s_{i,j}$: the length of a longest path from (0,0) to (i,j)

$$s_{i,j} = \max \begin{cases} s_{i-1,j} + \text{weight of edge "}\downarrow\text{" into } (i,j) \\ s_{i,j-1} + \text{weight of edge "}\rightarrow\text{" into } (i,j) \end{cases}$$

Example

- Find the longest path

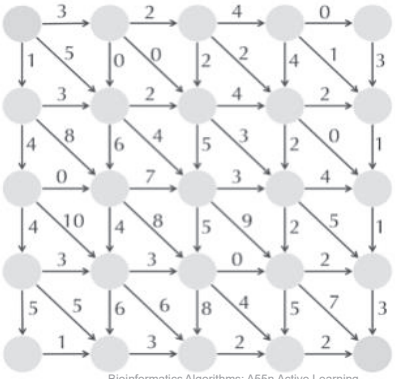


Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

Example

- Find the longest path: 35



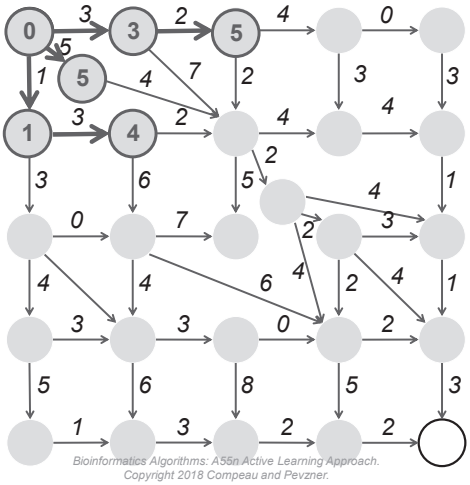
Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

How Do We Compare Biological Sequences

- The Alignment Game and the Longest Common Subsequence
- The Manhattan Tourist Problem
- Dynamic Programming and Backtracking Pointers
- **From Manhattan to the Alignment Graph**
- From Global to Local Alignment
- Penalizing Insertions and Deletions in Sequence Alignment
- Space-Efficient Sequence Alignment
- Multiple Sequence Alignment

Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

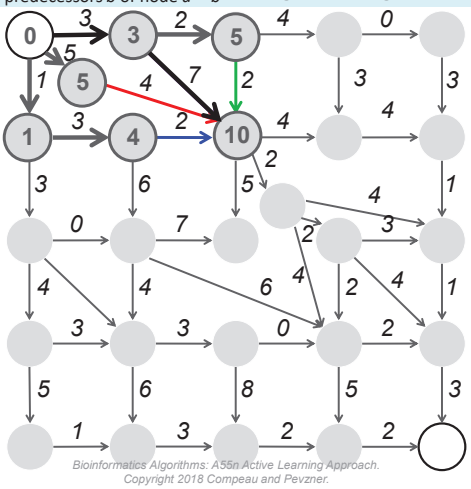
How does the recurrence change for this graph?



Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

$s_a = \max_{\text{all predecessors } b \text{ of node } a}\{s_b + \text{weight of edge from } b \text{ to } a\}$

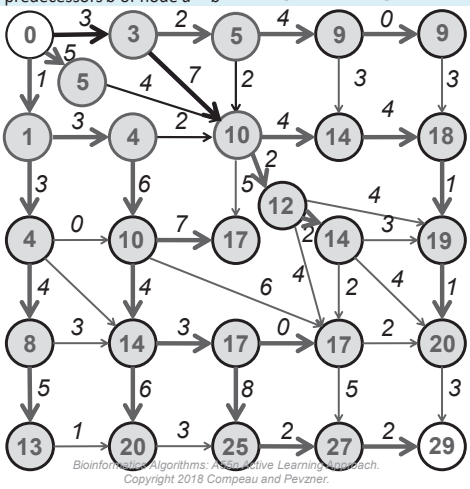
4 choices:
- 5 + 2
- 3 + 7
- 5 + 4
- 4 + 2



Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

$s_a = \max_{\text{all predecessors } b \text{ of node } a}\{s_b + \text{weight of edge from } b \text{ to } a\}$

4 choices:
- 5 + 2
- 3 + 7
- 5 + 4
- 4 + 2

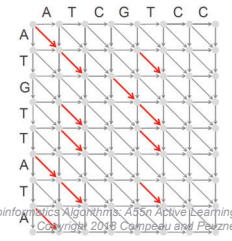


Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

Dynamic Programming Recurrence for the Alignment Graph

$s_{i,j}$: the length of a longest path from (0,0) to (i,j)

$$s_{i,j} = \max \begin{cases} s_{i-1,j} + \text{weight of edge "↓" into } (i,j) \\ s_{i,j-1} + \text{weight of edge "→" into } (i,j) \\ s_{i-1,j-1} + \text{weight of edge "↘" into } (i,j) \end{cases}$$



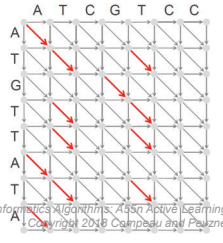
red edges ↘ – weight 1
other edges – weight 0

Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

Dynamic Programming Recurrence for the Longest Common Subsequence Problem

$s_{i,j}$: the length of a longest path from (0,0) to (i,j)

$$s_{i,j} = \max \begin{cases} s_{i-1,j} + 0 \\ s_{i,j-1} + 0 \\ s_{i-1,j-1} + 1, \text{ if } v_i = w_j \end{cases}$$

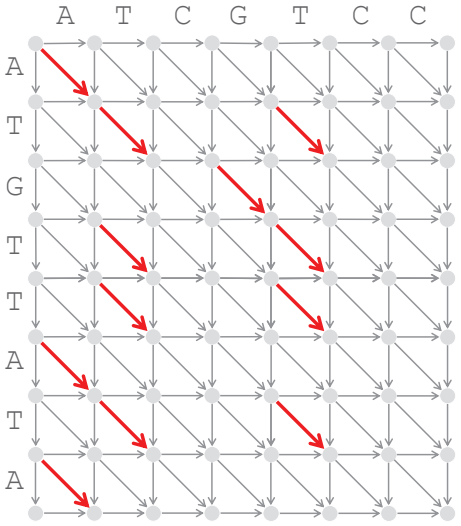


red edges ↘ – weight 1
other edges – weight 0

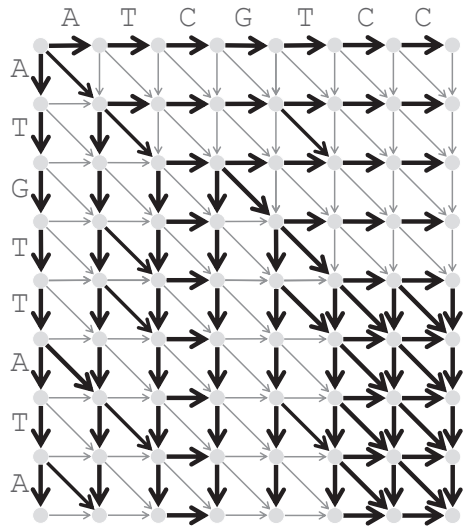
Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

backtracking pointers for the Longest Common Subsequence

red edges ↘ – weight 1
other edges – weight 0



backtracking pointers for the Longest Common Subsequence

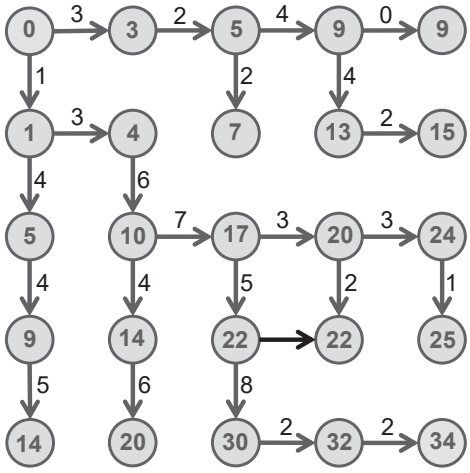


Computing Backtracking Pointers

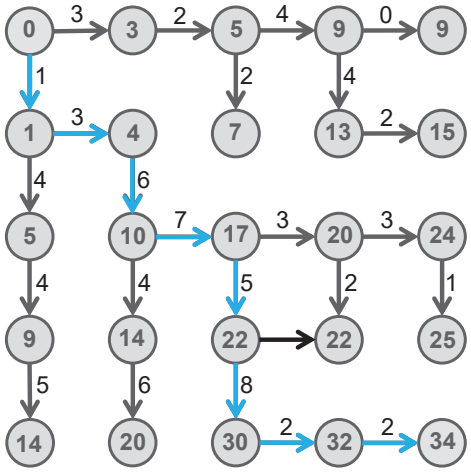
$$s_{i,j} \leftarrow \max\{ s_{i,j-1}+0,\ s_{i-1,j}+0,\ s_{i-1,j-1}+1 \text{ if } v_i=w_j \}$$

$$backtrack_{i,j} \leftarrow \{ \text{“}\rightarrow\text{”}, \text{ if } s_{i,j}=s_{i,j-1};\ \text{“}\downarrow\text{”}, \text{ if } s_{i,j}=s_{i-1,j};\ \text{“}\searrow\text{”}, \text{ if } s_{i,j}=s_{i-1,j-1}+1$$

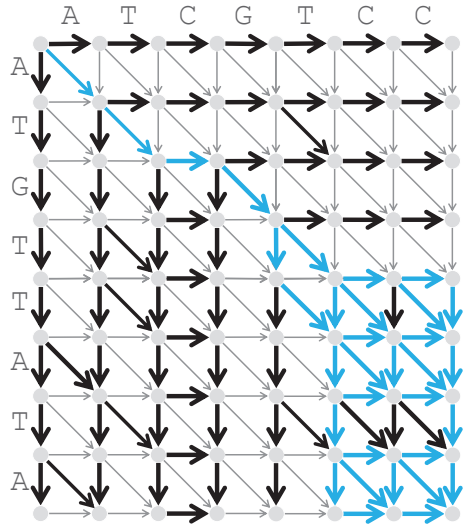
Why did we store the **backtracking pointers?**



What is the optimal alignment path?



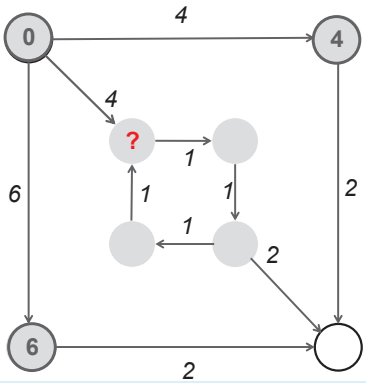
backtracking pointers for the Longest Common Subsequence



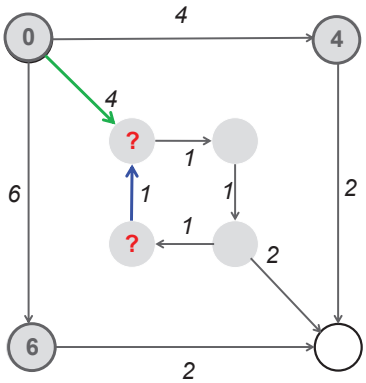
Using Backtracking Pointers to Compute LCS

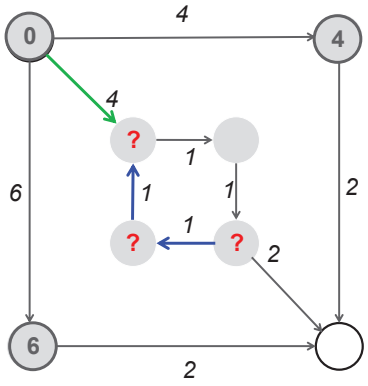
```
OutputLCS (backtrack, v, i, j)
  if i = 0 or j = 0
    return
  if backtrack_{i,j} = "→"
      OutputLCS (backtrack, v, i, j-1)
    else if backtrack_{i,j} = "↓"
      OutputLCS (backtrack, v, i-1, j)
    else
      OutputLCS (backtrack, v, i-1, j-1)
      output v_i
```


Computing Scores of ALL Predecessors

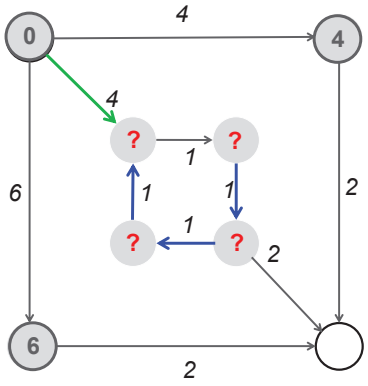


$s_a = \max_{\text{ALL predecessors } b \text{ of node } a}\{s_b + \text{weight of edge from } b \text{ to } a\}$



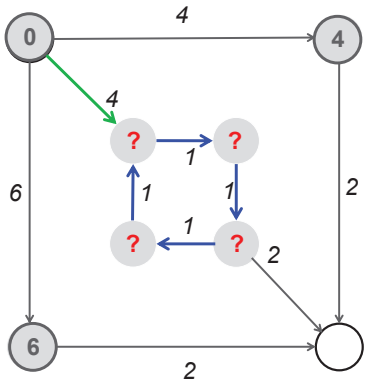


Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.



Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

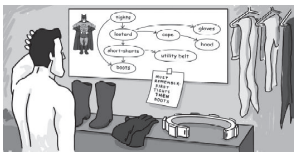
A Vicious Cycle



Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

In What Order Should We Explore Nodes of the Graph?

$s_a = \max_{\text{ALL predecessors } b \text{ of node } a}\{s_b + \text{weight of edge from } b \text{ to } a\}$

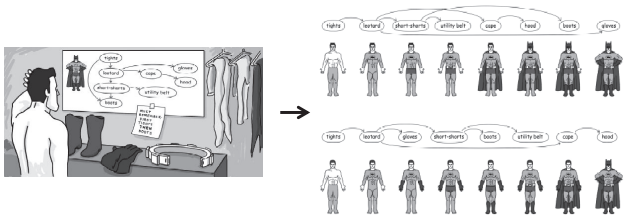


- By the time a node is analyzed, the scores of all its predecessors should already be computed.
- If the graph has a **directed cycle**, this condition is impossible to satisfy.
- **Directed Acyclic Graph** (**DAG**): a graph without directed cycles.

Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

Topological Ordering

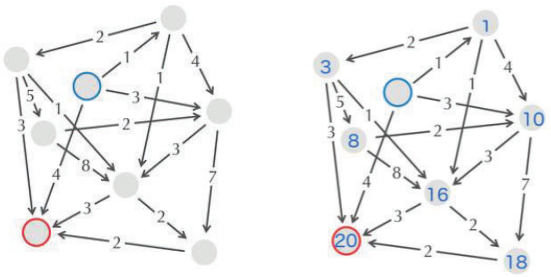
- **Topological Ordering**: Ordering of nodes of a DAG on a line such that all edges go from left to right.



- **Theorem**: Every DAG has a topological ordering.

Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

Q: What is the longest path from blue to red node?



Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

LongestPath

LongestPath(*Graph, source, sink*)
  for each node *a* in *Graph*
    $s_a \leftarrow$ -infinity
  $s_{source} \leftarrow 0$
  topologically order *Graph*
  for each node *a* (from *source* to *sink* in topological order)
    $s_a \leftarrow \max_{\text{all predecessors } b \text{ of node } a}\{s_b + \text{weight of edge from } b \text{ to } a\}$
  return s_{sink}

Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

How Do We Compare Biological Sequences

- The Alignment Game and the Longest Common Subsequence
- The Manhattan Tourist Problem
- Dynamic Programming and Backtracking Pointers
- From Manhattan to the Alignment Graph
- **From Global to Local Alignment**
- Penalizing Insertions and Deletions in Sequence Alignment
- Space-Efficient Sequence Alignment
- Multiple Sequence Alignment

Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

Current (Primitive) Scoring

#matches

Mismatches and Indel Penalties

#matches – μ · #mismatches – σ · #indels

```
A T - G T T A T A
A T C G T - C - C
+1+1-2+1+1-2-3-2-3=-7
```

	A	C	G	T	–
A	+1	$-\mu$	$-\mu$	$-\mu$	$-\sigma$
C	$-\mu$	+1	$-\mu$	$-\mu$	$-\sigma$
G	$-\mu$	$-\mu$	+1	$-\mu$	$-\sigma$
T	$-\mu$	$-\mu$	$-\mu$	+1	$-\sigma$
–	$-\sigma$	$-\sigma$	$-\sigma$	$-\sigma$	

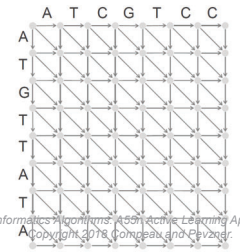
Scoring matrix

	A	C	G	T	–
A	+1	−3	−5	−1	−3
C	−4	+1	−3	−2	−3
G	−9	−7	+1	−1	−3
T	−3	−5	−8	+1	−4
–	−4	−2	−2	−1	

Even more general scoring matrix

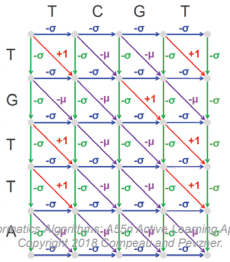
Dynamic Programming Recurrence for the Alignment Graph

$$s_{i,j} = \max \begin{cases} s_{i-1,j} + \text{weight of edge "}\downarrow\text{" into } (i,j) \\ s_{i,j-1} + \text{weight of edge "}\rightarrow\text{" into } (i,j) \\ s_{i-1,j-1} + \text{weight of edge "}\searrow\text{" into } (i,j) \end{cases}$$



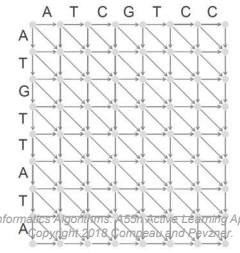
Dynamic Programming Recurrence for the Alignment Graph

$$s_{i,j} = \max \begin{cases} s_{i-1,j} - \sigma \\ s_{i,j-1} - \sigma \\ s_{i-1,j-1} + 1, \text{ if } v_i = w_j \\ s_{i-1,j-1} - \mu, \text{ if } v_i \neq w_j \end{cases}$$



Dynamic Programming Recurrence for the Alignment Graph

$$s_{i,j} = \max \begin{cases} s_{i-1,j} + score(v_i, -) \\ s_{i,j-1} + score(-, w_j) \\ s_{i-1,j-1} + score(v_i, w_j) \end{cases}$$



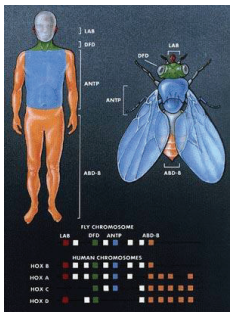
Global Alignment

Global Alignment Problem: Find the highest-scoring alignment between two strings by using a scoring matrix.

- **Input:** Strings *v* and *w* as well as a matrix ***score***.
- **Output:** An alignment of *v* and *w* whose alignment score (as defined by the scoring matrix ***score***) is maximal among all possible alignments of *v* and *w*.

Homeobox Genes

- Two genes in different species may be similar over short conserved regions and dissimilar over remaining regions.
- Homeobox genes have a short region called the **homeodomain** that is highly conserved among species.
  - A global alignment may not find the homeodomain because it would try to align the *entire* sequence.



Which Alignment is Better?

- Alignment 1: score = 22 (matches) - 20 (indels)=2.

```
GCC-C-AGT--TATGT-CAGGGGGCACG--A-GCATGCAGA-
GCCGCC-GTCGT-T-TTCAG----CA-GTTATG--T-CAGAT
```

- Alignment 2: score = 17 (matches) - 30 (indels)=-13.

```
---G----C-----C--CAGTTATGTCAGGGGGCACGAGCATGCAGA
GCCGCCGTCGTTTTCAGCAGTTATGTCAG-----A------T-----
```

Which Alignment is Better?

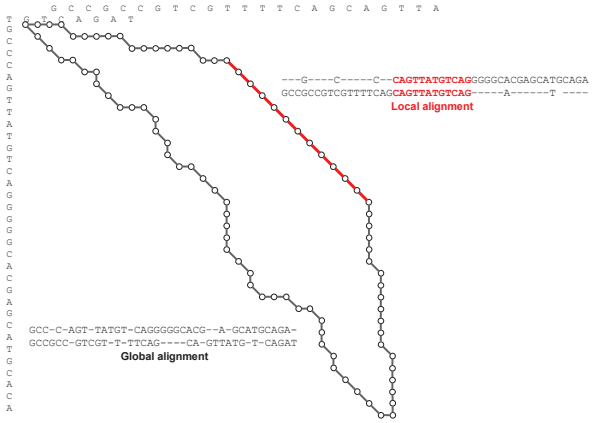
- Alignment 1: score = 22 (matches) - 20 (indels)=2.

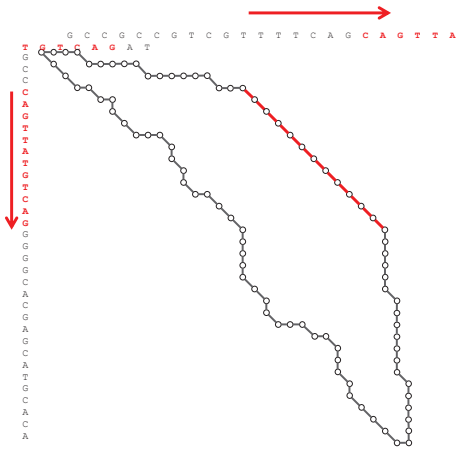
```
GCC-C-AGT--TATGT-CAGGGGGCACG--A-GCATGCAGA-
GCCGCC-GTCGT-T-TTCAG----CA-GTTATG--T-CAGAT
```

- Alignment 2: score = 17 (matches) - 30 (indels)=-13.

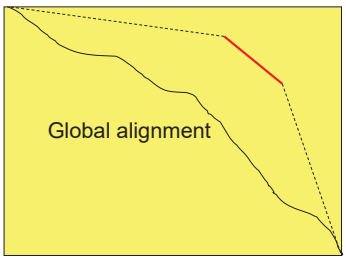
```
---G----C-----C--CAGTTATGTCAGGGGGCACGAGCATGCAGA
GCCGCCGTCGTTTTCAGCAGTTATGTCAG-----A------T-----
```

local alignment

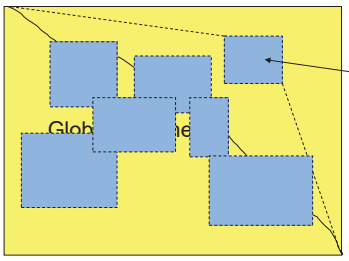




Local Alignment



Local Alignment= Global Alignment in a Subrectangle



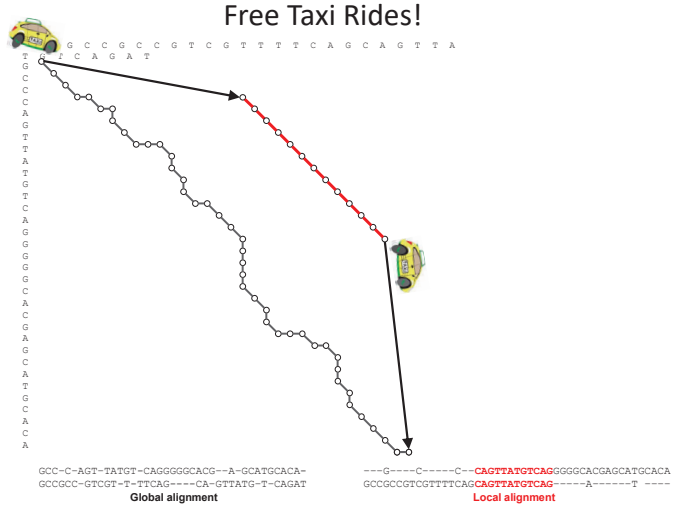
Compute a Global Alignment within each rectangle to get a Local Alignment

Local Alignment Problem

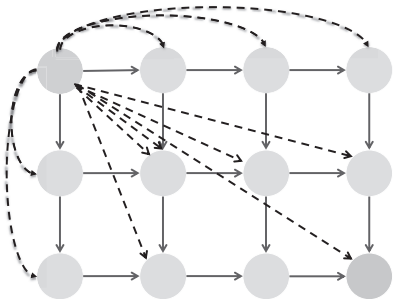
Local Alignment Problem: Find the highest-scoring local alignment between two strings.

- **Input:** Strings *v* and *w* as well as a matrix ***score***.
- **Output:** Substrings of *v* and *w* whose global alignment (as defined by the matrix ***score***), is maximal among all global alignments of all substrings of *v* and *w*.

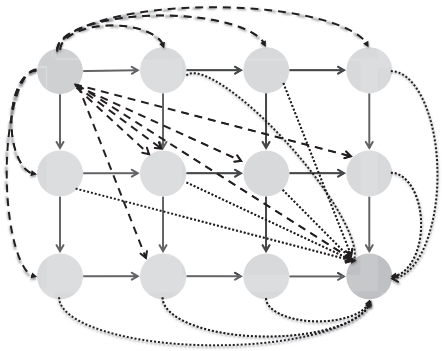
Free Taxi Rides!



What Do Free Taxi Rides Mean in the Terms of the Alignment Graph?



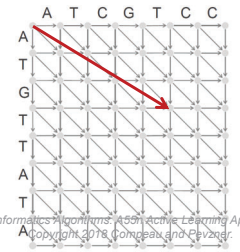
Building Manhattan for the Local Alignment Problem



How many edges have we added?

Dynamic Programming for the Local Alignment

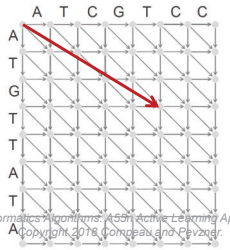
$$s_{i,j} = \max \begin{cases} 0 \\ s_{i-1,j} + \text{weight of edge "}\downarrow\text{" into } (i,j) \\ s_{i,j-1} + \text{weight of edge "}\rightarrow\text{" into } (i,j) \\ s_{i-1,j-1} + \text{weight of edge "}\searrow\text{" into } (i,j) \end{cases}$$



Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

Dynamic Programming for the Local Alignment

$$s_{i,j} = \max \begin{cases} \text{weight of edge from } (0,0) \text{ to } (i,j) \\ s_{i-1,j} + \text{weight of edge "}\downarrow\text{" into } (i,j) \\ s_{i,j-1} + \text{weight of edge "}\rightarrow\text{" into } (i,j) \\ s_{i-1,j-1} + \text{weight of edge "}\searrow\text{" into } (i,j) \end{cases}$$



Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

How Do We Compare Biological Sequences

- The Alignment Game and the Longest Common Subsequence
- The Change Problem
- Dynamic Programming and Backtracking Pointers
- From Manhattan to the Alignment Graph
- From Global to Local Alignment
- **Penalizing Insertions and Deletions in Sequence Alignment**
- Space-Efficient Sequence Alignment
- Multiple Sequence Alignment

Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

Scoring Gaps

- We previously assigned a fixed penalty σ to each indel.
- However, this fixed penalty may be too severe for a series of 100 consecutive indels.
- A series of k indels often represents a single evolutionary event (**gap**) rather than k events:

two gaps (lower score)	GATCCAG GA-C-AG	GATCCAG GA--CAG	a single gap (higher score)

Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

More Adequate Gap Penalties

Affine gap penalty for a gap of length k: $\sigma+\varepsilon\cdot(k-1)$

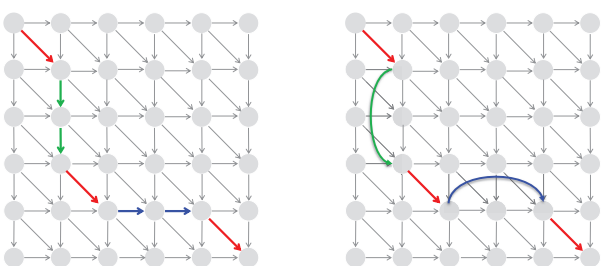
σ - the **gap opening penalty**

ε - the **gap extension penalty**

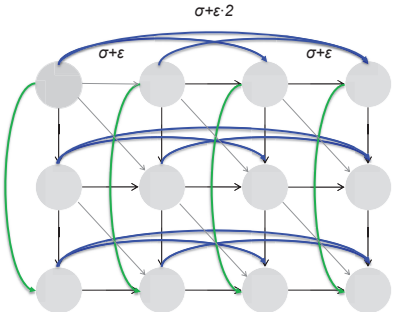
$\sigma > \varepsilon$, since starting a gap should be penalized more than extending it.

Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

Modelling Affine Gap Penalties by Long Edges

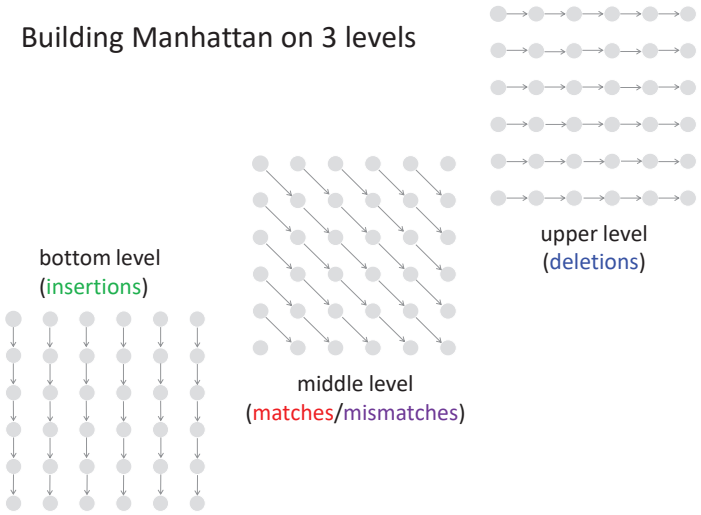


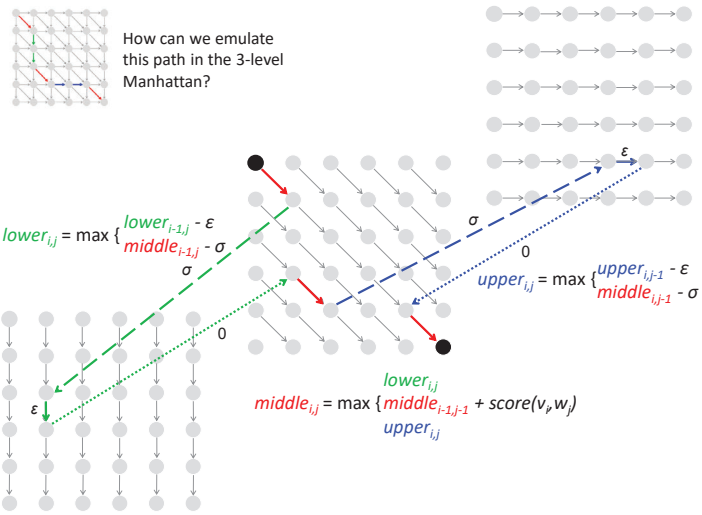
Building Manhattan with Affine Gap Penalties



We have just added $O(n^3)$ edges to the graph…

Building Manhattan on 3 levels



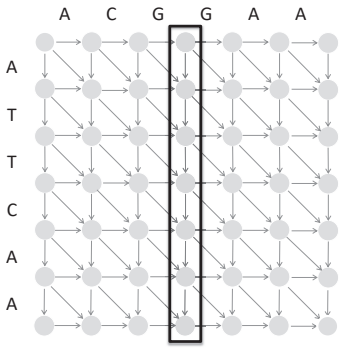


How Do We Compare Biological Sequences

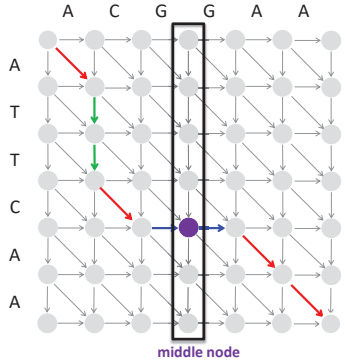
- The Alignment Game and the Longest Common Subsequence
- The Manhattan Tourist Problem
- Dynamic Programming and Backtracking Pointers
- From Manhattan to an Arbitrary Directed Acyclic Graph
- From Global to Local Alignment
- Penalizing Insertions and Deletions in Sequence Alignment
- **Space-Efficient Sequence Alignment**
- Multiple Sequence Alignment

Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

Middle Column of the Alignment



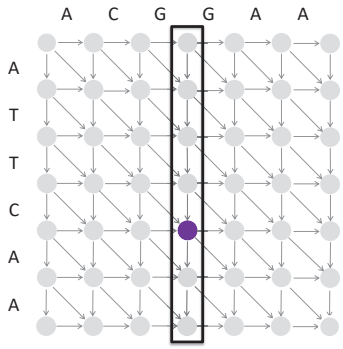
Middle Node of the Alignment



(a node where an optimal alignment path crosses the middle column)

Divide and Conquer Approach to Sequence Alignment

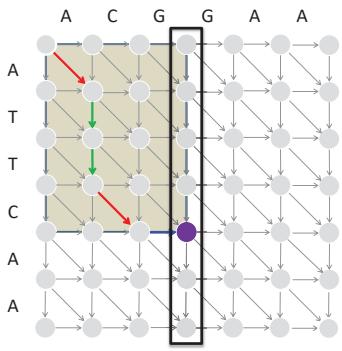
AlignmentPath(*source, sink*)
find *MiddleNode*



Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

Divide and Conquer Approach to Sequence Alignment

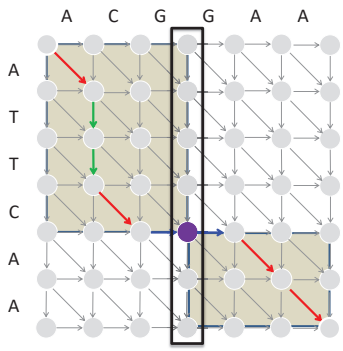
AlignmentPath(*source, sink*)
find *MiddleNode*
AlignmentPath(*source, MiddleNode*)



Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

Divide and Conquer Approach to Sequence Alignment

AlignmentPath(*source, sink*)
find *MiddleNode*
AlignmentPath(*source, MiddleNode*)
AlignmentPath(*MiddleNode, sink*)



The only problem left is how to find this middle node in **linear space**!

Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

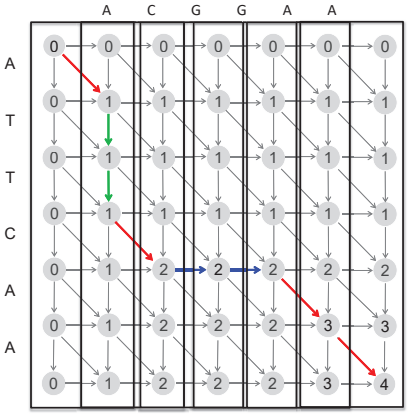
Computing Alignment Score in Linear Space

Finding the **longest path** in the alignment graph **requires** storing all backtracking pointers – O(*nm*) memory.

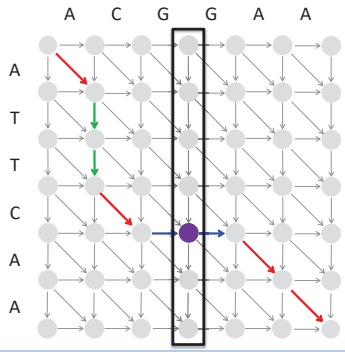
Finding the **length of the longest path** in the alignment graph **does not require** storing any backtracking pointers – O(*n*) memory.

Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

Recycling the Columns in the Alignment Graph



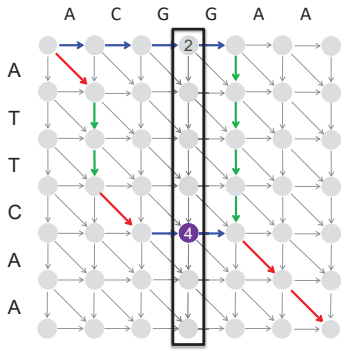
Can We Find the Middle Node without Constructing the Longest Path?



4-path that visits the node (4,middle) In the middle column

***i*-path** – a longest path among paths that visit the *i*-th node in the middle column

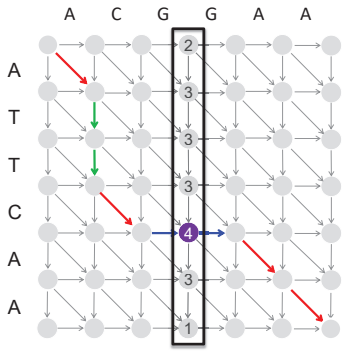
Can We Find The Lengths of All *i*-paths?



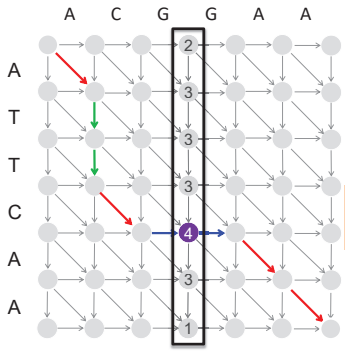
length(i): length of an *i*-path:

length(0)=2
length(4)=4

Can We Find The Lengths of All *i*-paths?



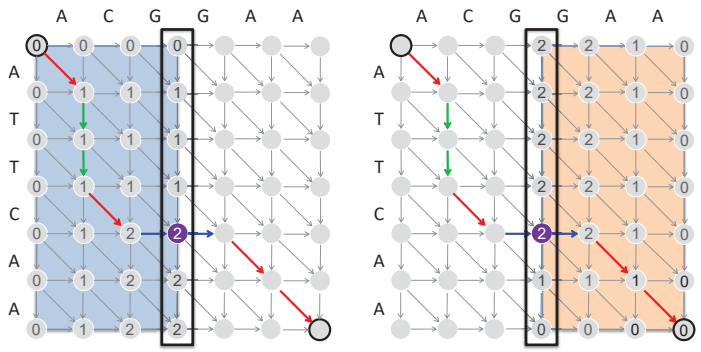
Can We Find The Lengths of *i*-paths?



length(i): length of an *i*-path

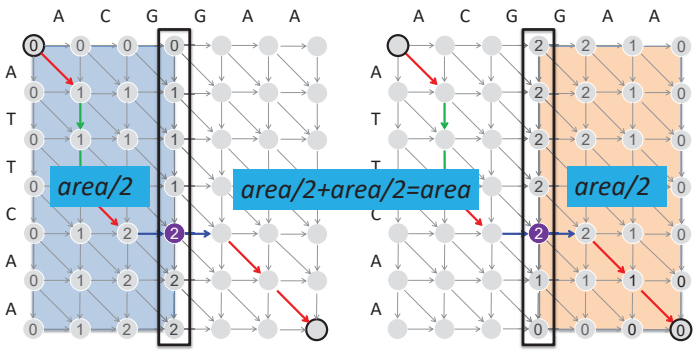
length(i)=fromSource(i)+toSink(i)

Computing *FromSource* and *toSink*



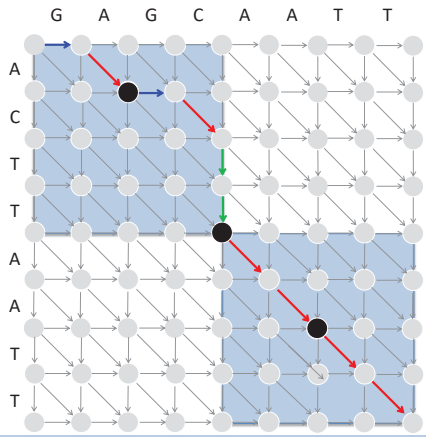
fromSource(i) *toSink(i)*

How Much Time Did It Take to Find the Middle Node ?



fromSource(i) *toSink(i)*

Laughable Progress: O(*nm*) Time to Find **ONE** Node!

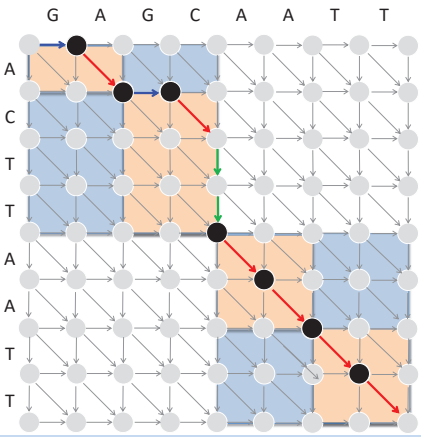


Each subproblem can be conquered in time proportional to its area:

area/4+area/4= ***area/2***

How much time would it take to conquer 2 subproblems?

Laughable Progress: O(*nm+nm/2*) Time to Find **THREE** Nodes!

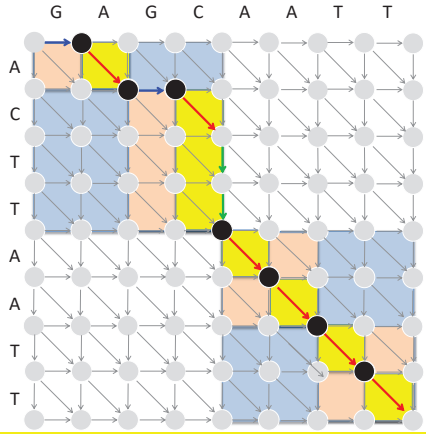


Each subproblem can be conquered in time proportional to its area:

area/8+area/8+ area/8+area/8= ***area/4***

How much time would it take to conquer 4 subproblems?

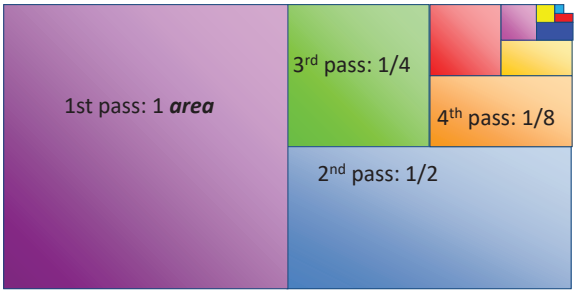
O(*nm+nm/2+nm/4*) Time to Find **NEARLY ALL** Nodes!



area+ area/2 +area/4 +area/8 +area/16 +….+ < ***2·area***

How much time would it take to conquer ALL subproblems?

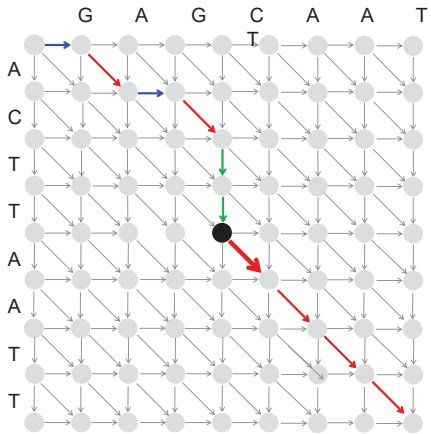
Total Time: ***area+area/2+area/4+area/8+area/16+…***



1 + ½ + ¼ +… < 2

Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

The Middle Edge



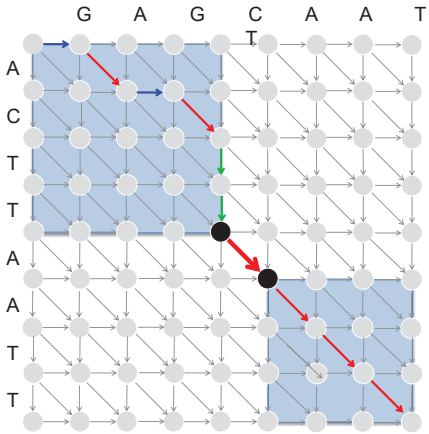
Middle Edge: an edge in an optimal alignment path starting at the middle node

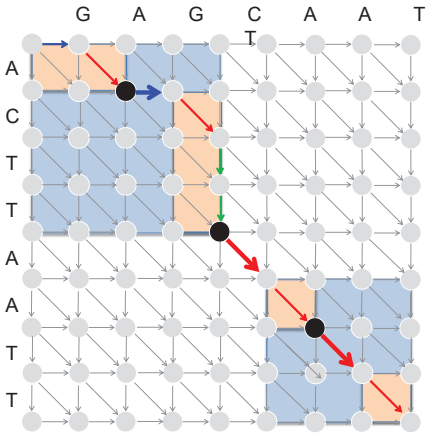
The Middle Edge Problem

Middle Edge in Linear Space Problem. Find a middle edge in the alignment graph in linear space.

- **Input:** Two strings and matrix ***score***.
- **Output:** A middle edge in the alignment graph of these strings (as defined by the matrix ***score***).

Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.





Recursive **LinearSpaceAlignment**

```
LinearSpaceAlignment(top,bottom,left,right)
  if left = right
    return alignment formed by bottom-top edges "↓"
  middle ← ⌊(left+right)/2⌋
  midNode ← MiddleNode(top,bottom,left,right)
  midEdge ←  MiddleEdge(top,bottom,left,right)
  LinearSpaceAlignment(top,midNode,left,middle)
  output midEdge
  if midEdge = "→" or midEdge = "↘"
    middle  ← middle+1
  if  midEdge = "↓" or midEdge = "↘"
    midNode  ← midNode+1
  LinearSpaceAlignment(midNode,bottom,middle,right)
```

How Do We Compare Biological Sequences

- The Alignment Game and the Longest Common Subsequence
- The Manhattan Tourist Problem
- Dynamic Programming and Backtracking Pointers
- From Manhattan to an Arbitrary Directed Acyclic Graph
- From Global to Local Alignment
- Penalizing Insertions and Deletions in Sequence Alignment
- Space-Efficient Sequence Alignment
- **Multiple Sequence Alignment**

From Pairwise to Multiple Alignment

- Up until now we have only tried to align two sequences.
- A faint (and statistically insignificant) similarity between two sequences becomes significant if it is present in many other sequences.
- Multiple alignments can reveal subtle similarities that pairwise alignments do not reveal.

Alignment of Three A-domains

```
YAFDLGYTCMFPVLLGGGELHIVQKETYTAPDEIAHYIKEHGITYIKLTPSLFHTIVNTASFAFDANFESLRLIVLGGEKIIPIDVIAFRKMYGHTE-FINHYGPTEATIGA
-AFDVSAGDFARALLTGGQLIVCPNEVKMDPASLYAIIKKYDITIFEATPALVIPLMEYI-YEQKLDISQLQILIVGSDSCSMEDFKTLVSRFGSTIRIVNSYGVTEACIDS
IAFDASSWEIYAPLLNGGTVVCIDYYTTIDIKALEAVFKQHHIRGAMLPPALLKQCLVSA----PTMISSLEILFAAGDRLSSQDAILARRAVGSGV-Y-NAYGPTENTVLS
```


Alignment of Three A-domains

```
YAFDLGYTCMFPVLLGGGELHIVQKETYTAPDEIAHYIKEHGITYIKLTPSLFHTIVNTASFAFDANFESLRLIVLGGEKIIPIDVIAFRKMYGHTE-FINHYGPTEATIGA
-AFDVSAGDFARALLTGGQLIVCPNEVKMDPASLYAIIKKYDITIFEATPALVIPLMEYI-YEQKLDISQLQILIVGSDSCSMEDFKTLVSRFGSTIRIVNSYGVTEACIDS
IAFDASSWEIYAPLLNGGTVVCIDYYTTIDIKALEAVFKQHHIRGAMLPPALLKQCLVSA----PTMISSLEILFAAGDRLSSQDAILARRAVGSGV-Y-NAYGPTENTVLS
```


Alignment of Three A-domains

```
YAFDLGYTCMFPVLLGGGELHIVQKETYTAPDEIAHYIKEHGITYIKLTPSLFHTIVNTASFAFDANFESLRLIVLGGEKIIPIDVIAFRKMYGHTE-FINHYGPTEATIGA
-AFDVSAGDFARALLTGGQLIVCPNEVKMDPASLYAIIKKYDITIFEATPALVIPLMEYI-YEQKLDISQLQILIVGSDSCSMEDFKTLVSRFGSTIRIVNSYGVTEACIDS
IAFDASSWEIYAPLLNGGTVVCIDYYTTIDIKALEAVFKQHHIRGAMLPPALLKQCLVSA----PTMISSLEILFAAGDRLSSQDAILARRAVGSGV-Y-NAYGPTENTVLS
```


Generalizing Pairwise to Multiple Alignment

- Alignment of 2 sequences is a 2-row matrix.
- Alignment of 3 sequences is a 3-row matrix

```
A T - G C G -
A - C G T - A
A T C A C - A
```

- Our scoring function should score alignments with conserved columns higher.

Alignments = Paths in 3-D

- Alignment of ATGC, AATC, and ATGC

	A	--	T	G	C

	A	A	T	--	C

	--	A	T	G	C

Alignments = Paths in 3-D

- Alignment of ATGC, AATC, and ATGC

0	1	1	2	3	4
	A	--	T	G	C

	A	A	T	--	C

	--	A	T	G	C

#symbols up to a given position

Alignments = Paths in 3-D

- Alignment of ATGC, AATC, and ATGC

0	1	1	2	3	4
	A	--	T	G	C
0	1	2	3	3	4
	A	A	T	--	C

	--	A	T	G	C

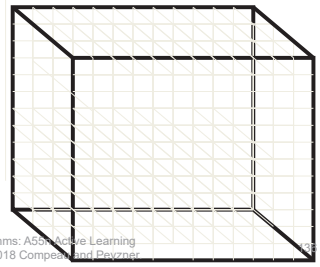
#symbols up to a given position

Alignments = Paths in 3-D

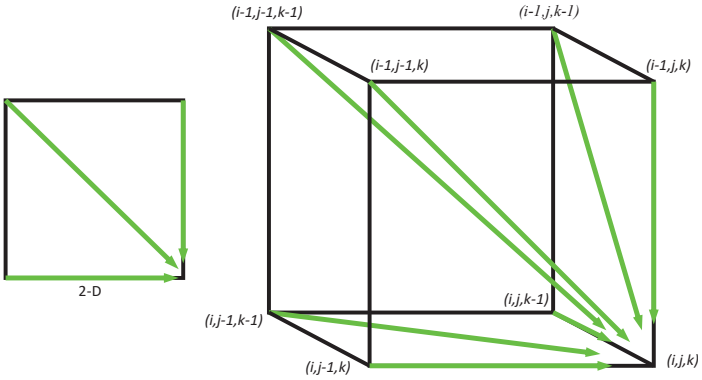
- Alignment of ATGC, AATC, and ATGC

(0,0,0)→(1,1,0)→(1,2,1) →(2,3,2) →(3,3,3) →(4,4,4)

0	1	1	2	3	4
	A	--	T	G	C
0	1	2	3	3	4
	A	A	T	--	C
0	0	1	2	3	4
	--	A	T	G	C



2-D Alignment Cell versus 3-D Alignment Cell



Multiple Alignment: Dynamic Programming

$$s_{i,j,k} = \max \begin{cases} s_{i-1,j-1,k-1} + \delta(v_i, w_j, u_k) \\ s_{i-1,j-1,k} + \delta(v_i, w_j, -) \\ s_{i-1,j,k-1} + \delta(v_i, -, u_k) \\ s_{i,j-1,k-1} + \delta(-, w_j, u_k) \\ s_{i-1,j,k} + \delta(v_i, -, -) \\ s_{i,j-1,k} + \delta(-, w_j, -) \\ s_{i,j,k-1} + \delta(-, -, u_k) \end{cases}$$

- $\delta(x, y, z)$ is an entry in the 3-D scoring matrix.

Multiple Alignment: Running Time

- For 3 sequences of length n, the run time is proportional to $7n^3$
- For a k-way alignment, build a k-dimensional Manhattan graph with
  - n^k nodes
  - most nodes have $2^k - 1$ incoming edges.
  - Runtime: $O(2^k n^k)$

Multiple Alignment Induces Pairwise Alignments

Every multiple alignment induces pairwise alignments:

```
AC-GCGG-C
AC-GC-GAG
GCCGC-GAG
```

```
ACGCGG-C   AC-GCGG-C   AC-GCGAG
ACGC-GAC   GCCGC-GAG   GCCGCGAG
```


Idea: Construct Multiple from Pairwise Alignments

Given a set of **arbitrary** pairwise alignments, can we construct a multiple alignment that induces them?

```
AAAATTTT----      ----AAAATTTT      TTTTGGGG----
----TTTTGGGG      GGGGAAAA----      ----GGGGAAAA
```

Profile Representation of Multiple Alignment

```
  - A G G C T A T C A C C T G
  T A G - C T A C C A - - - G
  C A G - C T A C C A - - - G
  C A G - C T A T C A C - G G
  C A G - C T A T C G C - G G

A 0 1 0 0 0 0 1 0 0 .8 0 0 0 0
C .6 0 0 0 1 0 0 .4 1 0 .6 .2 0 0
G 0 0 1 .2 0 0 0 0 0 0 .2 0 0 .4 1
T .2 0 0 0 0 1 0 .6 0 0 0 0 0 .2 0
- .2 0 0 .8 0 0 0 0 0 0 .4 .8 .4 0
```


Aligning Sequence Against Sequence

- In the past we were aligning a **sequence against a sequence**.

```
  - A G G C T A T C A C C T G
  T A G - C T A C C A - - - G
  C A G - C T A C C A - - - G
  C A G - C T A T C A C - G G
  C A G - C T A T C G C - G G

A 0 1 0 0 0 0 1 0 0 .8 0 0 0 0
C .6 0 0 0 1 0 0 .4 1 0 .6 .2 0 0
G 0 0 1 .2 0 0 0 0 0 0 .2 0 0 .4 1
T .2 0 0 0 0 1 0 .6 0 0 0 0 0 .2 0
- .2 0 0 .8 0 0 0 0 0 0 .4 .8 .4 0
```


Aligning Sequence Against Profile

- In the past we were aligning a **sequence against a sequence**.
  - Can we align a **sequence against a profile?**

```
  - A G G C T A T C A C C T G
  T A G - C T A C C A - - - G
  C A G - C T A C C A - - - G
  C A G - C T A T C A C - G G
  C A G - C T A T C G C - G G

A 0 1 0 0 0 0 1 0 0 .8 0 0 0 0
C .6 0 0 0 1 0 0 .4 1 0 .6 .2 0 0
G 0 0 1 .2 0 0 0 0 0 0 .2 0 0 .4 1
T .2 0 0 0 0 1 0 .6 0 0 0 0 0 .2 0
- .2 0 0 .8 0 0 0 0 0 0 .4 .8 .4 0
```


Aligning Profile Against Profile

- In the past we were aligning a **sequence against a sequence.**
  - Can we align a **sequence against a profile?**
  - Can we align a **profile against a profile?**

```
  - A G G C T A T C A C C T G
  T A G - C T A C C A - - - G
  C A G - C T A C C A - - - G
  C A G - C T A T C A C - G G
  C A G - C T A T C G C - G G

A 0 1 0 0 0 0 1 0 0 .8 0 0 0 0
C .6 0 0 0 1 0 0 .4 1 0 .6 .2 0 0
G 0 0 1 .2 0 0 0 0 0 0 .2 0 0 .4 1
T .2 0 0 0 0 1 0 .6 0 0 0 0 0 .2 0
- .2 0 0 .8 0 0 0 0 0 0 .4 .8 .4 0
```


Multiple Alignment: Greedy Approach

- Choose the most similar sequences and combine them into a profile, thereby reducing alignment of k sequences to an alignment of of $k - 2$ sequences and 1 profile.
- Iterate

Greedy Approach: Example

- Sequences: GATTCA, GTCTGA, GATATT, GTCAGC.
- 6 pairwise alignments (premium for match +1, penalties for indels and mismatches -1)

```
s2 GTCTGA                 s1 GATTCA--
s4 GTCAGC   (score = 2)   s4 G-T-CAGC  (score = 0)

s1 GAT-TCA                s2 G-TCTGA
s2 G-TCTGA  (score = 1)   s3 GATAT-T   (score = -1)

s1 GAT-TCA                s3 GAT-ATT
s3 GATAT-T  (score = 1)   s4 G-TCAGC   (score = -1)
```


Greedy Approach: Example

- Since s_2 and s_4 are closest, we consolidate them into a profile:

$s2$ GTCTGA
$s4$ GTCAGC
} $s_{2,4}$ = GTCt/aGa/cA

- New set of 3 sequences to align:

s_1 GATTCA
s_3 GATATT
$s_{2,4}$ GTCt/aGa/c

Are there Fragile Regions in the Human Genome?
Combinatorial Algorithms

Phillip Compeau and Pavel Pevzner
Bioinformatics Algorithms: an Active Learning Approach
©2013 by Compeau and Pevzner. All rights reserved

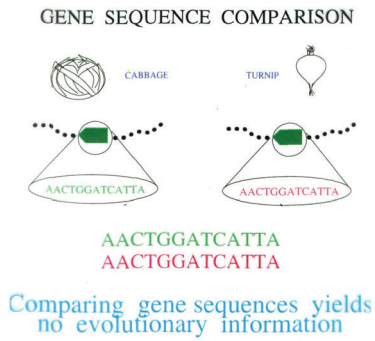
Are There Fragile Regions in the Human Genome?

- **Transforming Cabbage into Turnip**
- Sorting by Reversals
- Breakpoint Theorem
- Rearrangements in Tumor Genomes
- 2-Break Distance Problem
- Breakpoint Graphs
- 2-Break Distance Theorem
- Rearrangement Hotspots in the Human Genome

Turnip vs Cabbage: Look and Taste Different

- Although cabbages and turnips share a recent common ancestor, they look and taste different

Turnip vs Cabbage: Comparing Gene Sequences Yields No Evolutionary Information

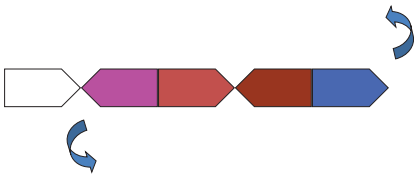


Turnip vs Cabbage: Almost Identical mtDNA gene sequences

- In 1980s Jeffrey Palmer studied evolution of plant organelles by comparing mitochondrial genomes of the cabbage and turnip
- 99% similarity between genes
- These surprisingly identical gene sequences differed in gene order
- This study helped pave the way to analyzing genome rearrangements in molecular evolution

Turnip vs Cabbage: Different mtDNA Gene Order

- Gene order comparison:



Turnip vs Cabbage: Different mtDNA Gene Order

- Gene order comparison:

Turnip vs Cabbage: Different mtDNA Gene Order

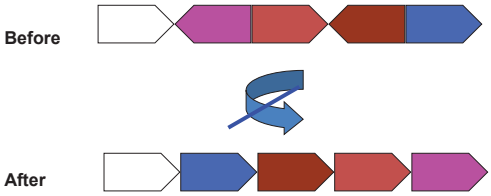
- Gene order comparison:

Turnip vs Cabbage: Different mtDNA Gene Order

- Gene order comparison:

Turnip vs Cabbage: Different mtDNA Gene Order

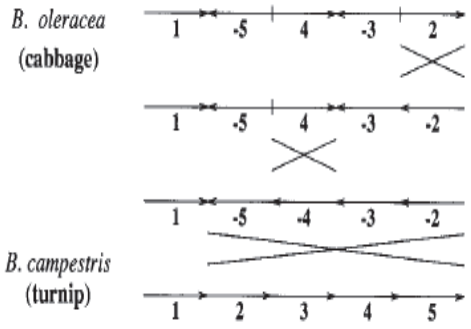
- Gene order comparison:



Evolution is manifested as the divergence in gene order

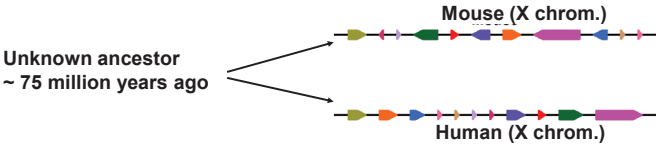
Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Transforming Cabbage into Turnip



Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

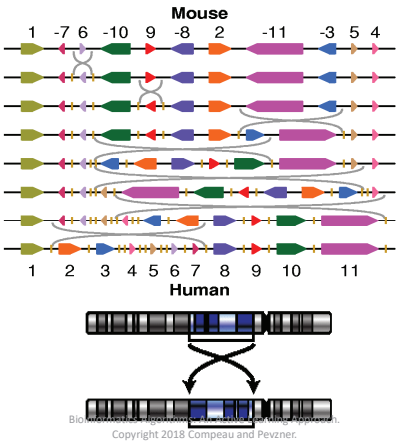
Genome rearrangements



- What are the similarity blocks and how to find them?
- What is the architecture of the ancestral genome?
- What is the evolutionary scenario for transforming one genome into the other?

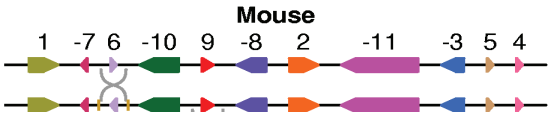
Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Transforming Mice into Men



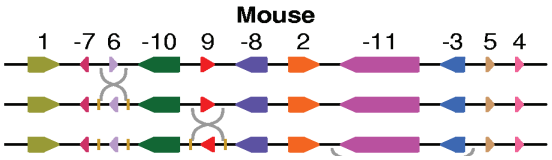
Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Series of Reversals



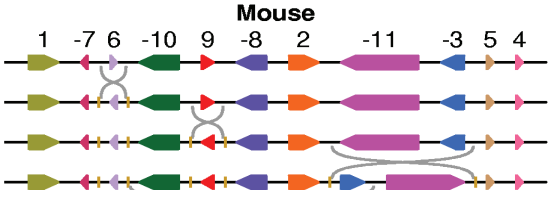
Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Series of Reversals



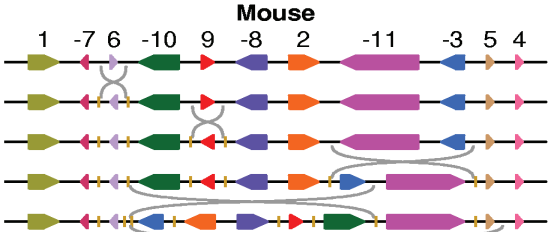
Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Series of Reversals



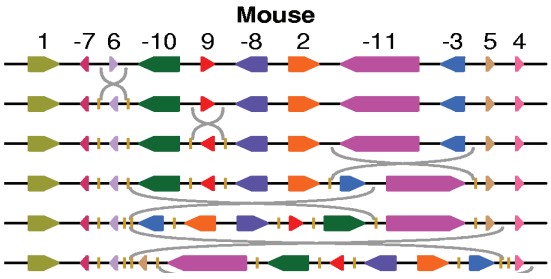
Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Series of Reversals



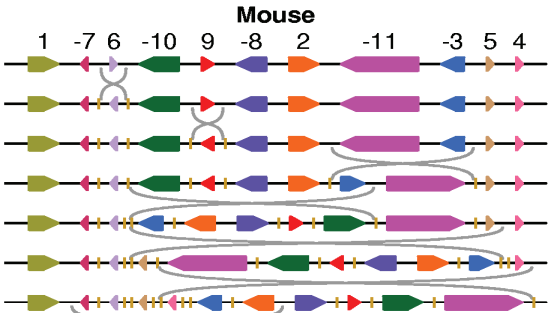
Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Series of Reversals



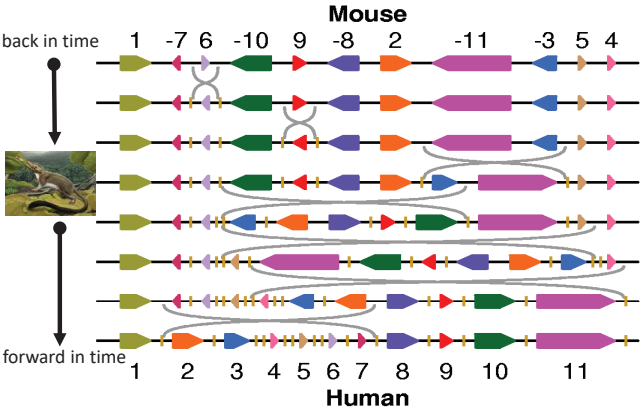
Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Series of Reversals



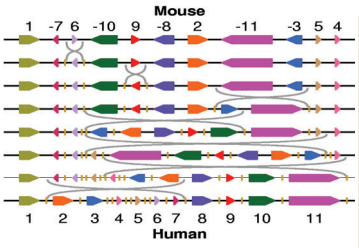
Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

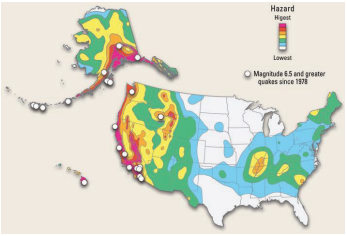
Human-Mouse Transformation Accomplished!



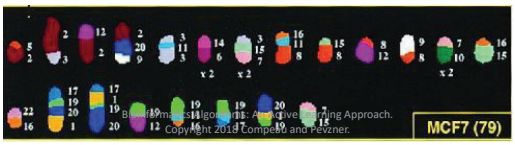
Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Evolutionary Earthquakes



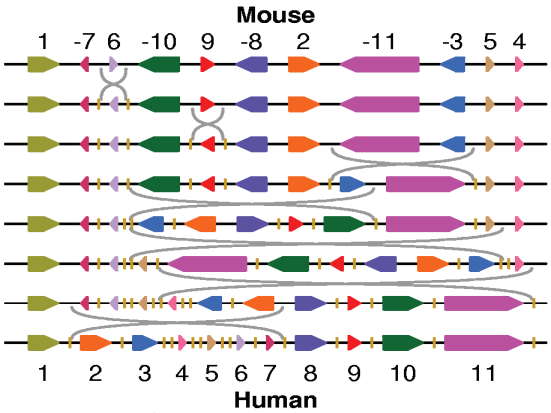


Are there any rearrangement hotspots in mammalian genomes?



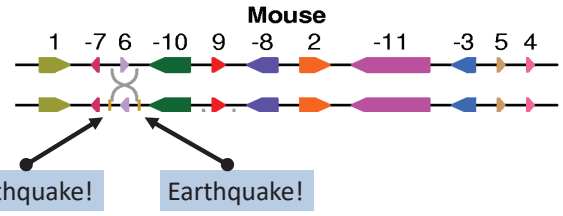
Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Are There Any Rearrangement Hotspots in This Scenario?



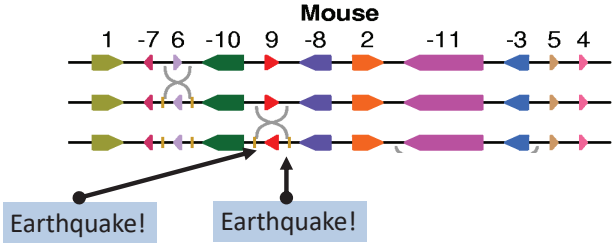
Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Are There Any Rearrangement Hotspots in This Scenario?



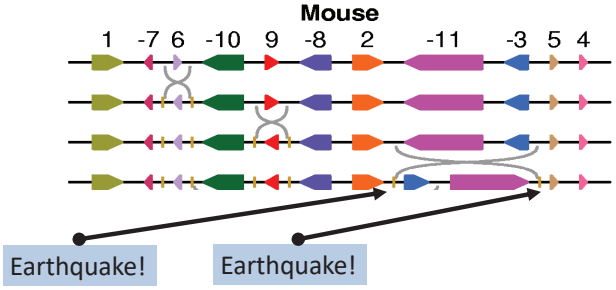
Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Are There Any Rearrangement Hotspots in This Scenario?



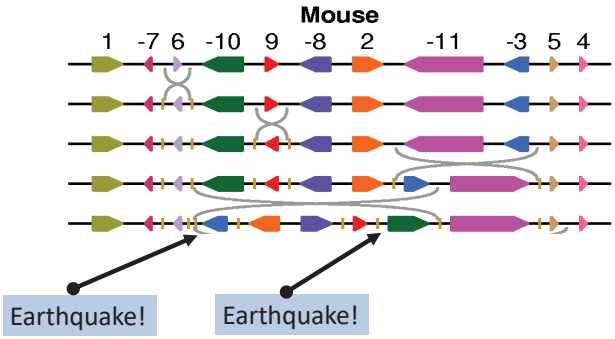
Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Are There Any Rearrangement Hotspots in This Scenario?



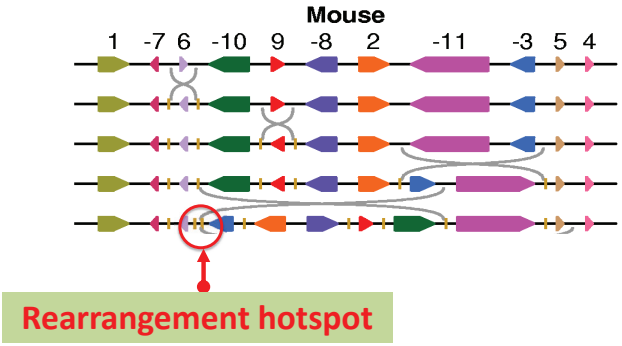
Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Are There Any Rearrangement Hotspots in This Scenario?



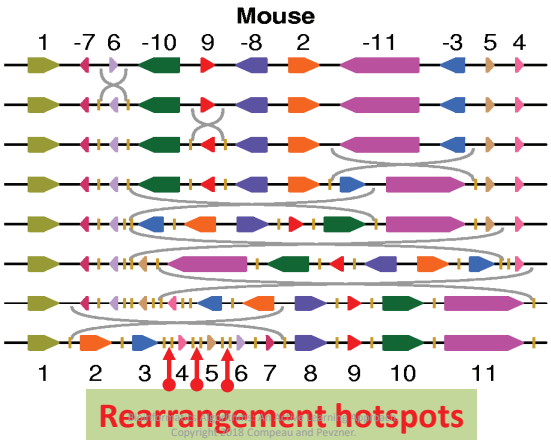
Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

A Rearrangement Hotspots Found!



Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

More Rearrangement Hotspots



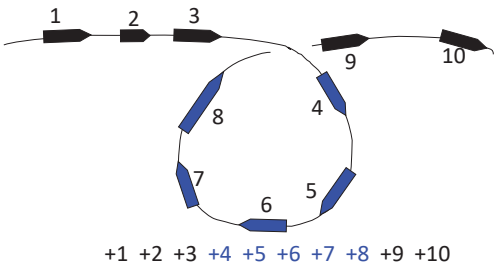
Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Are There Fragile Regions in the Human Genome?

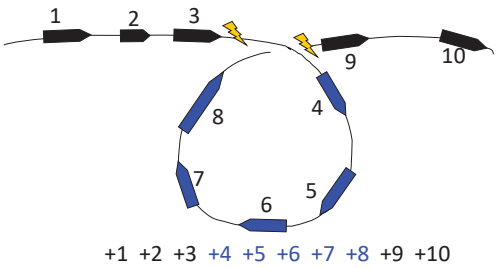
- Transforming Men into Mice
- **Sorting by Reversals**
- Breakpoint Theorem
- Rearrangements in Tumor Genomes
- 2-Breaks
- Breakpoint Graphs
- 2-Break Distance Theorem
- Rearrangement Hotspots in the Human Genome

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Reversals

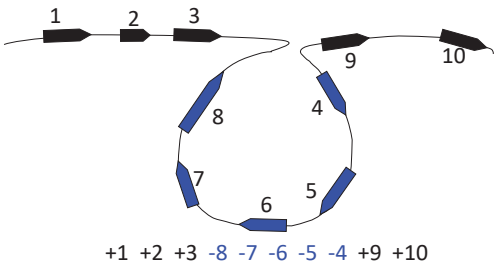


Reversals



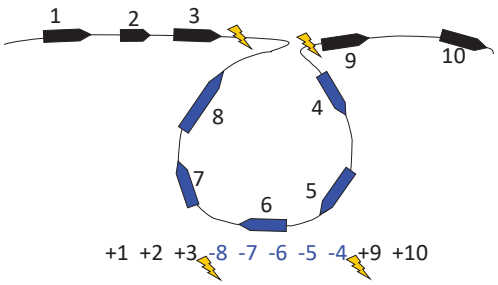
Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Reversals



Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Reversals



The reversal introduced two *breakpoints* (disruptions in gene order).

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Rearrangement Scenario with 5 Reversals

Step 0:	2	-4	-3	5	-8	-7	-6	1
Step 1:	2	3	4	5	-8	-7	-6	1
Step 2:	2	3	4	5	6	7	8	1
Step 3:	2	3	4	5	6	7	8	-1
Step 4:	-8	-7	-6	-5	-4	-3	-2	-1
Step 5:	1	2	3	4	5	6	7	8

Rearrangement Scenario with **4** Reversals

Step 0:	2	-4	-3	5	-8	-7	-6	1
Step 1:	2	3	4	5	-8	-7	-6	1
Step 2:	-5	-4	-3	-2	-8	-7	-6	1
Step 3:	-5	-4	-3	-2	-1	6	7	8
Step 4:	1	2	3	4	5	6	7	8

Reversal distance: the minimum number of reversals to transform one permutation into another.

Sorting by 4 Reversals

Step 0:	2	-4	-3	5	-8	-7	-6	1
Step 1:	2	3	4	5	-8	-7	-6	1
Step 2:	-5	-4	-3	-2	-8	-7	-6	1
Step 3:	-5	-4	-3	-2	-1	6	7	8
Step 4:	1	2	3	4	5	6	7	8

Sorting by Reversals Problem: Calculate the reversal distance between a permutation and the identity permutation (+1 +2 … +*n*).

- **Input**: A permutation *P*.
- **Output**: The reversal distance between *P* and the identity permutation.

Reversals and Gene Orders

- Gene order is represented by a permutation π:

$$\pi = \pi_1 \text{ ------ } \pi_{i-1}\ \pi_i\ \pi_{i+1} \text{ ------ } \pi_{j-1}\ \pi_j\ \pi_{j+1} \text{ ----- } \pi_n$$

$$\pi_1 \text{ ------ } \pi_{i-1}\ \pi_j\ \pi_{j-1} \text{ ------ } \pi_{i+1}\ \pi_i\ \pi_{j+1} \text{ ----- } \pi_n \quad \rho(i,j)$$

- Reversal $\rho(i, j)$ reverses (flips) the elements from i to j in π

Reversal Distance Problem

- Goal: Given two permutations, find the shortest series of reversals that transforms one into another
- Input: Permutations π and σ
- Output: A series of reversals $\rho_1,\ldots\rho_t$ transforming π into σ, such that t is minimum
- t - reversal distance between π and σ
- $d(\pi, \sigma)$ - smallest possible value of t, given π and σ

Sorting By Reversals Problem

- Goal: Given a permutation, find a shortest series of reversals that transforms it into the identity permutation (*1 2 … n*)
- Input: Permutation π
- Output: A series of reversals $\rho_1, \ldots \rho_t$ transforming π into the identity permutation such that t is minimum

Sorting By Reversals: Example

- $t = d(\pi)$ - reversal distance of π
- Example :

π = 3 4 2 1 5 6 7 10 9 8
4 3 2 1 5 6 7 10 9 8
4 3 2 1 5 6 7 8 9 10
1 2 3 4 5 6 7 8 9 10

So $d(\pi) = 3$

Sorting by reversals: 4 steps

Step 0: π	2	-4	-3	5	-8	-7	-6	1
Step 1:	2	3	4	5	-8	-7	-6	1
Step 2:	-5	-4	-3	-2	-8	-7	-6	1
Step 3:	-5	-4	-3	-2	-1	6	7	8
Step 4: γ	1	2	3	4	5	6	7	8

Sorting by reversals: 4 steps

Step 0: π	2	-4	-3	5	-8	-7	-6	1
Step 1:	2	3	4	5	-8	-7	-6	1
Step 2:	-5	-4	-3	-2	-8	-7	-6	1
Step 3:	-5	-4	-3	-2	-1	6	7	8
Step 4: γ	1	2	3	4	5	6	7	8

What is the reversal distance for this permutation? Can it be sorted in 3 steps?

Pancake Flipping Problem

- The chef is sloppy; he prepares an unordered stack of pancakes of different sizes
- The waiter wants to rearrange them (so that the smallest winds up on top, and so on, down to the largest at the bottom)
- He does it by flipping over several from the top, repeating this as many times as necessary

Christos Papadimitrou and Bill Gates flip pancakes

Pancake Flipping Problem: Formulation

- Goal: Given a stack of *n* pancakes, what is the minimum number of flips to rearrange them into perfect stack?
- Input: Permutation π
- Output: A series of prefix reversals $\rho_1, \ldots \rho_t$ transforming π into the identity permutation such that *t* is minimum

Sorting By Reversals: A Greedy Algorithm

- If sorting permutation π = 1 2 3 6 4 5, the first three elements are already in order so it does not make any sense to break them.
- The length of the already sorted prefix of π is denoted *prefix*(π)
  - *prefix*(π) = 3
- This results in an idea for a greedy algorithm: increase *prefix*(π) at every step

Greedy Algorithm: An Example

- Doing so, π can be sorted

1 2 3 6 4 5

↓

1 2 3 4 6 5

↓

1 2 3 4 5 6

- Number of steps to sort permutation of length *n* is at most *(n – 1)*

Greedy Algorithm: Pseudocode

SimpleReversalSort(π)

```
1 for i ← 1 to n – 1
2   j ← position of element i in π (i.e., π_j = i)
3   if j ≠ i
4     π ← π * ρ(i, j)
5     output π
6   if π is the identity permutation
7     return
```


Greedy Sorting by Reversals

(+1 −7 +6 −10 +9 −8 +2 −11 −3 +5 +4)

Greedy Sorting by Reversals

(+1 −7 +6 −10 +9 −8 +2 −11 −3 +5 +4)
(+1 −2 +8 −9 +10 −6 +7 −11 −3 +5 +4)
(+1 +2 +8 −9 +10 −6 +7 −11 −3 +5 +4)
(+1 +2 +3 +11 −7 +6 −10 +9 −8 +5 +4)
(+1 +2 +3 −4 −5 +8 −9 +10 −6 +7 −11)
(+1 +2 +3 +4 −5 +8 −9 +10 −6 +7 −11)
(+1 +2 +3 +4 +5 +8 −9 +10 −6 +7 −11)
(+1 +2 +3 +4 +5 +6 −10 +9 −8 +7 −11)
(+1 +2 +3 +4 +5 +6 −7 +8 −9 +10 −11)
(+1 +2 +3 +4 +5 +6 +7 +8 −9 +10 −11)
(+1 +2 +3 +4 +5 +6 +7 +8 +9 +10 −11)
(+1 +2 +3 +4 +5 +6 +7 +8 +9 +10 +11)

Analyzing SimpleReversalSort

- SimpleReversalSort does not guarantee the smallest number of reversals and takes five steps on $\pi = 6\ 1\ 2\ 3\ 4\ 5$:
  - Step 1: 1 6 2 3 4 5
  - Step 2: 1 2 6 3 4 5
  - Step 3: 1 2 3 6 4 5
  - Step 4: 1 2 3 4 6 5
  - Step 5: 1 2 3 4 5 6

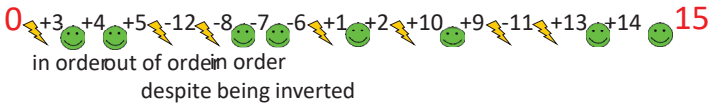
Analyzing SimpleReversalSort (cont'd)

- But it can be sorted in two steps:
  π = 6 1 2 3 4 5
  - Step 1: 5 4 3 2 1 6
  - Step 2: 1 2 3 4 5 6
- So, SimpleReversalSort(π) is not optimal
- Optimal algorithms are unknown for many problems; approximation algorithms are used

Are there Fragile Regions in the Human Genome?

- Transforming Men into Mice
- Sorting by Reversals
- **Breakpoint Theorem**
- Rearrangements in Tumor Genomes
- 2-Breaks
- Breakpoint Graphs
- 2-Break Distance Theorem
- Rearrangement Hotspots in the Human Genome

Adjacencies and Breakpoints



adjacencies(P) + breakpoints(P)=|P|+1

What is the number of breakpoints in the identity permutation (+1 +2 … +*n*)?

Adjacencies and Breakpoints

adjacencies(P) + breakpoints(P)=|P|+1

What is the number of breakpoints in the identity permutation (+1 +2 … +*n*)?

Sorting by Reversals as Breakpoint Elimination

breakpoints(P)

Step 0:	2 -4 -3 5 -8 -7 -6 1	**6**
Step 1:	2 3 4 5 -8 -7 -6 1	**4**
Step 2:	-5 -4 -3 -2 -8 -7 -6 1	**4**
Step 3:	-5 -4 -3 -2 -1 6 7 8	**2**
Step 4:	1 2 3 4 5 6 7 8	**0**

How many breakpoints can be eliminated by a single reversal?

Breakpoint Theorem ≥ *breakpoints(P)*/**2**

Hint:

Adjacencies and Breakpoints

$\pi = \pi_1\pi_2\pi_3 \ldots \pi_{n-1}\pi_n$

- A pair of elements π_i and π_{i+1} are adjacent if
  $\pi_{i+1} = \pi_i \pm 1$
- For example:
  π = 1 9 3 4 7 8 2 6 5
- (3, 4) or (7, 8) and (6,5) are adjacent pairs

Breakpoints: An Example

There is a breakpoint between any adjacent element that are non-consecutive:

π = 1 | 9 | 3 4 | 7 8 | 2 | 6 5

- Pairs (1,9), (9,3), (4,7), (8,2) and (2,5) form breakpoints of permutation π
- $b(\pi)$ - # breakpoints in permutation π

Adjacency & Breakpoints

•An adjacency - a pair of adjacent elements that are consecutive

• A breakpoint - a pair of adjacent elements that are not consecutive

$\pi = 5\ 6\ 2\ 1\ 3\ 4$ ⟶ Extend π with $\pi_0 = 0$ and $\pi_7 = 7$

adjacencies

0 5 6 2 1 3 4 7

breakpoints

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Extending Permutations

- We put two elements $\pi_0 = 0$ and $\pi_{n+1} = n+1$ at the ends of π

  Example:

$\pi = 1\ |9|\ 3\ 4\ |7\ 8\ |2\ |6\ 5$

Extending with *0* and *10*

$\pi = 0\ 1\ |9|\ 3\ 4\ |7\ 8\ |2\ |6\ 5\ |10$

Note: A new breakpoint was created after extending

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Reversal Distance and Breakpoints

- Each reversal eliminates at most 2 breakpoints.

$\pi = 2\ 3\ 1\ 4\ 6\ 5$

$0\ |2\ 3\ |1\ |\ 4\ |6\ 5\ |7$ $b(\pi) = 5$

$0\ 1\ |3\ 2\ |\ 4\ |6\ 5\ |7$ $b(\pi) = 4$

$0\ 1\ 2\ 3\ 4\ |\ 6\ 5\ |7$ $b(\pi) = 2$

$0\ 1\ 2\ 3\ 4\ 5\ 6\ 7$ $b(\pi) = 0$

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Reversal Distance and Breakpoints

- Each reversal eliminates at most 2 breakpoints.
- This implies:

reversal distance ≥ #breakpoints / 2

$\pi = 2\ 3\ 1\ 4\ 6\ 5$

$0\ |2\ 3\ |1\ |\ 4\ |6\ 5\ |7$ $b(\pi) = 5$

$0\ 1\ |3\ 2\ |\ 4\ |6\ 5\ |7$ $b(\pi) = 4$

$0\ 1\ 2\ 3\ 4\ |\ 6\ 5\ |7$ $b(\pi) = 2$

$0\ 1\ 2\ 3\ 4\ 5\ 6\ 7$ $b(\pi) = 0$

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Sorting By Reversals: A Better Greedy Algorithm

BreakPointReversalSort(π)

1 **while** $b(\pi) > 0$
2 Among all possible reversals,
choose reversal ρ minimizing $b(\pi \bullet \rho)$
3 $\pi \leftarrow \pi \bullet \rho(i, j)$
4 **output** π
5 **return**

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Sorting By Reversals: A Better Greedy Algorithm

BreakPointReversalSort(π)

1 **while** $b(\pi) > 0$
2 Among all possible reversals,
choose reversal ρ minimizing $b(\pi \bullet \rho)$
3 $\pi \leftarrow \pi \bullet \rho(i, j)$
4 **output** π
5 **return**

Problem: this algorithm may work forever

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Strips

- Strip: an interval between two consecutive breakpoints in a permutation
  - Decreasing strip: *strip* of elements in decreasing order (e.g. 6 5 and 3 2).
  - Increasing strip: *strip* of elements in increasing order (e.g. 7 8)

0 1 9 4 3 7 8 2 5 6 *10*

- A single-element strip can be declared either increasing or decreasing. We will choose to declare them as decreasing with exception of the strips with *0* and *n+1*

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Reducing the Number of Breakpoints

Theorem 1:

If permutation π contains at least one decreasing strip, then there exists a reversal ρ which decreases the number of breakpoints (i.e. $b(\pi \bullet \rho) < b(\pi)$)

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Things To Consider

- For π = 1 4 6 5 7 8 3 2

  0 1 | 4 | 6 5 | 7 8 | 3 2 | 9 $b(\pi) = 5$

  – Choose decreasing strip with the smallest element *k* in π (*k* = 2 in this case)

Things To Consider (cont'd)

- For π = 1 4 6 5 7 8 3 2

  0 1 | 4 | 6 5 | 7 8 | 3 2 | 9 $b(\pi) = 5$

  – Choose decreasing strip with the smallest element *k* in π (*k* = 2 in this case)

Things To Consider (cont'd)

- For π = 1 4 6 5 7 8 3 2

  0 1 | 4 | 6 5 | 7 8 | 3 2 | 9 $b(\pi) = 5$

  – Choose decreasing strip with the smallest element *k* in π (*k* = 2 in this case)

  – Find *k – 1* in the permutation

Things To Consider (cont'd)

- For π = 1 4 6 5 7 8 3 2

  0 1 | 4 | 6 5 | 7 8 | 3 2 | 9 $b(\pi) = 5$

  – Choose decreasing strip with the smallest element *k* in π (*k* = 2 in this case)

  – Find *k – 1* in the permutation

  – Reverse the segment between *k* and *k-1*:

  – 0 1 | 4 | 6 5 | 7 8 | 3 2 | 9 $b(\pi) = 5$

  – 0 1 2 3 | 8 7 | 5 6 | 4 | 9 $b(\pi) = 4$

Reducing the Number of Breakpoints Again

– If there is no decreasing strip, there may be no reversal ρ that reduces the number of breakpoints (i.e. $b(\pi \bullet \rho) \geq b(\pi)$ for any reversal ρ).

– By reversing an increasing strip (# of breakpoints stay unchanged), we will create a decreasing strip at the next step. Then the number of breakpoints will be reduced in the next step (theorem 1).

Things To Consider (cont'd)

- There are no decreasing strips in π, for:

π = 0 1 2 | 5 6 7 | 3 4 | 8 $b(\pi) = 3$

$\pi \bullet \rho(6,7)$ = 0 1 2 | 5 6 7 | 4 3 | 8 $b(\pi) = 3$

✓ $\rho(6,7)$ does not change the # of breakpoints

✓ $\rho(6,7)$ creates a decreasing strip thus guaranteeing that the next step will decrease the # of breakpoints.

ImprovedBreakpointReversalSort

ImprovedBreakpointReversalSort(π)
1 **while** $b(\pi) > 0$
2 **if** π has a decreasing strip
3 Among all possible reversals, choose reversal ρ
that minimizes $b(\pi \bullet \rho)$
4 **else**
5 Choose a reversal ρ that flips an increasing strip in π
6 $\pi \leftarrow \pi \bullet \rho$
7 **output** π
8 **return**

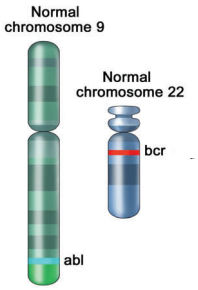
ImprovedBreakpointReversalSort: Performance Guarantee

- *ImprovedBreakPointReversalSort* is an approximation algorithm with a performance guarantee of at most 4
  – It eliminates at least one breakpoint in every two steps; at most $2b(\pi)$ steps
  – Approximation ratio: $2b(\pi) \ / \ d(\pi)$
  – Optimal algorithm eliminates at most 2 breakpoints in every step: $d(\pi) \geq b(\pi) / 2$
  – Performance guarantee:
    • $(2b(\pi) / d(\pi)) \geq [2b(\pi) / (b(\pi) / 2)] = 4$

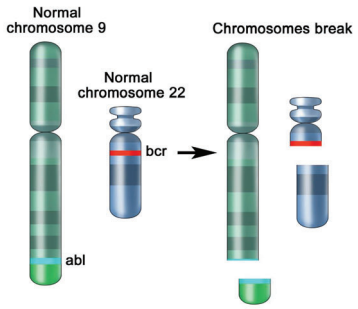
Are There Fragile Regions in the Human Genome?

- Transforming Men into Mice
- Sorting by Reversals
- Breakpoint Theorem
- **Rearrangements in Tumor Genomes**
- 2-Breaks
- Breakpoint Graphs
- 2-Break Distance Theorem
- Rearrangement Hotspots in the Human Genome

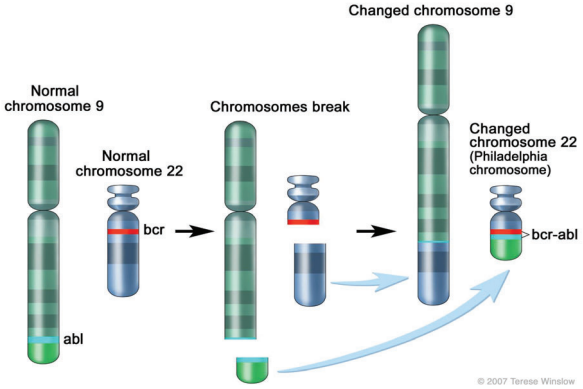
Philadelphia Chromosome



Philadelphia Chromosome



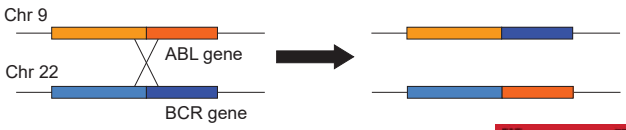
Philadelphia Chromosome



Oncogenic **bcr-abl** fusion gene:
biomarker for chronic myeloid leukemia (CML)

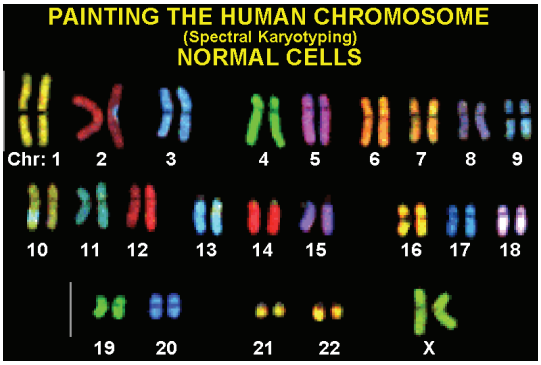
Translocations

Translocation in CML yields Philadelphia chromosome:



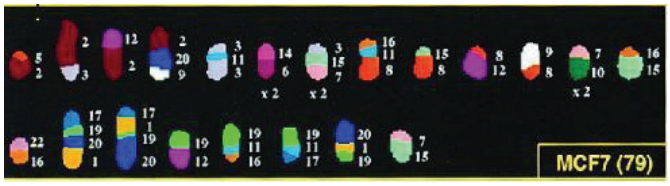
Thousands of rearrangements known for different tumors.

Painting Chromosomes (Normal Cells)



Painting Chromosomes (Tumor Cells)

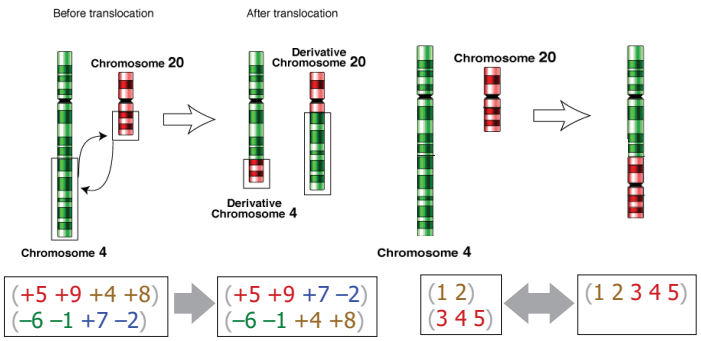
- MCF7 is human breast cancer cell line:



What sequence of rearrangements has produced MCF7?

Rearrangements in Multichromosomal Genomes

translocations **fusions and fissions**



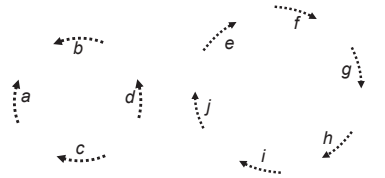
Are There Fragile Regions in the Human Genome?

- Transforming Men into Mice
- Sorting by Reversals
- Breakpoint Theorem
- Rearrangements in Tumor Genomes
- **2-Break Distance Problem**
- Breakpoint Graphs
- 2-Break Distance Theorem
- Rearrangement Hotspots in the Human Genome

From Linear to Circular Chromosomes

$(+a\ -b\ -d\ +c)$ $(+e\ +f\ +g\ +h\ +i\ +j)$

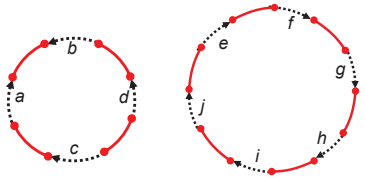
From Linear to Circular Chromosomes



$(+a\ -b\ -d\ +c)$ $(+e\ +f\ +g\ +h\ +i\ +j)$

Black directed edges represent synteny blocks.

From Linear to Circular Chromosomes



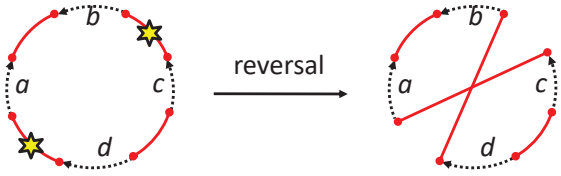
$(+a\ -b\ -d\ +c)$ $(+e\ +f\ +g\ +h\ +i\ +j)$

Black directed edges represent synteny blocks.
Red undirected edges connect adjacent synteny blocks.

Equivalent Representations of a Circular Genome

$Q=(+a-b-d+c)$ $Q=(+a-b-d+c)$

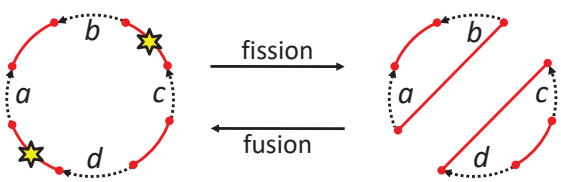
Reversals on Circular Chromosomes



$P=(+a-b-c+d)$ $Q=(+a-b-d+c)$

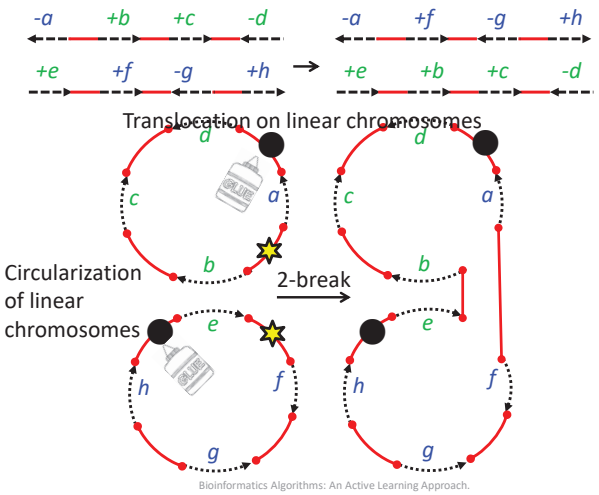
A reversal deletes two red edges and replaces them by two other red edges (on the same 4 nodes)

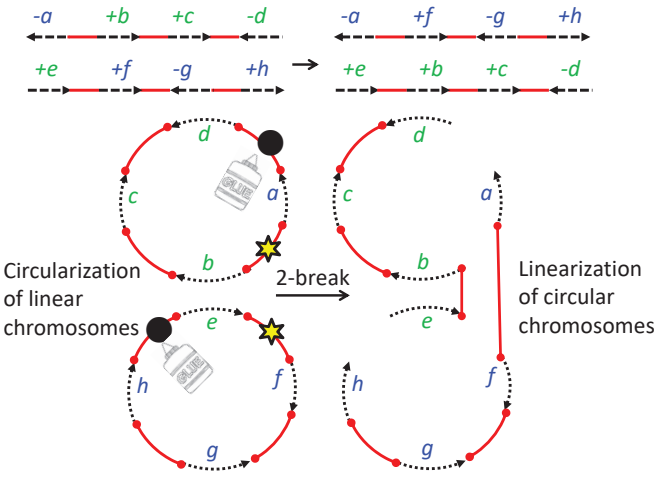
Fusion/Fission as Replacement of Two Red Edges



$P=(+a-b-c+d)$ $Q= (+a-b)\ (-c+d)$

2-break: a rearrangement that replaces two red edges with two new red edges on the same 4 nodes.





2-Break Distance

2-Break distance *d(P,Q):*
minimum number of 2-breaks transforming genome *P* into genome *Q*

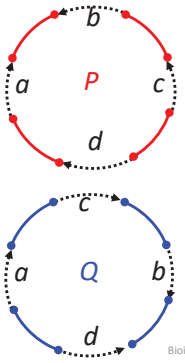
2-Break Distance Problem. Find the 2-break distance between two genomes.

- **Input.** Two genomes on the same set of synteny blocks.
- **Output.** The 2-break distance between these genomes.

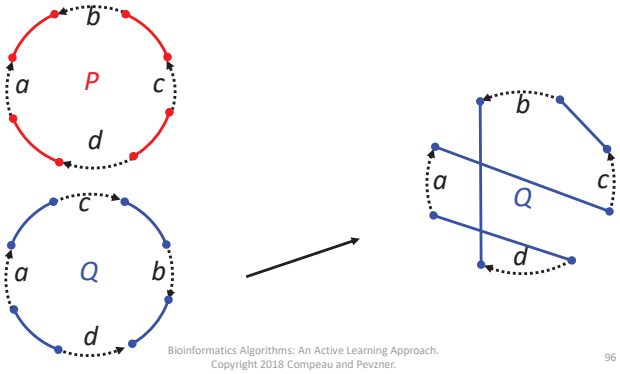
Are There Fragile Regions in the Human Genome?

- Transforming Men into Mice
- Sorting by Reversals
- Breakpoint Theorem
- Rearrangements in Tumor Genomes
- 2-Break Distance Problem
- **Breakpoint Graphs**
- 2-Break Distance Theorem
- Rearrangement Hotspots in the Human Genome

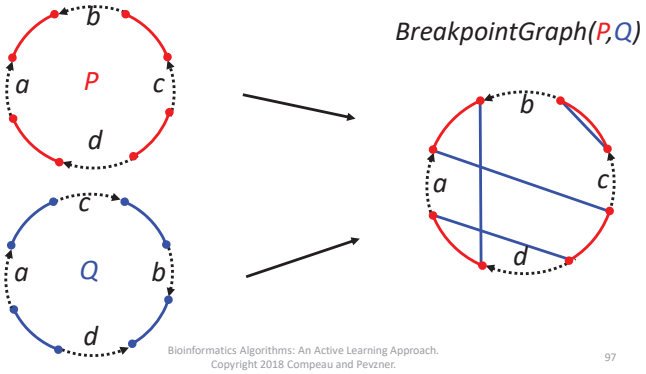
Comparing Genomes *P* and *Q*



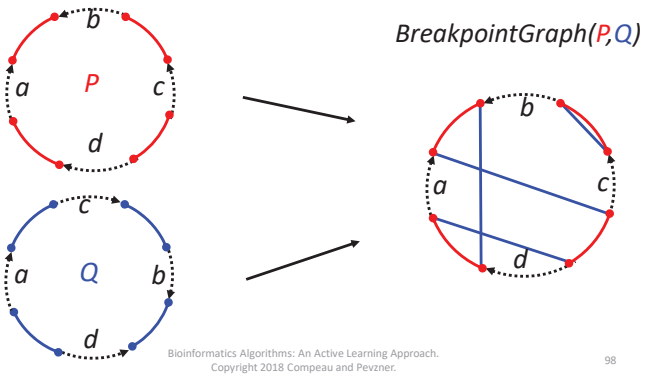
Different Drawing of *Q*



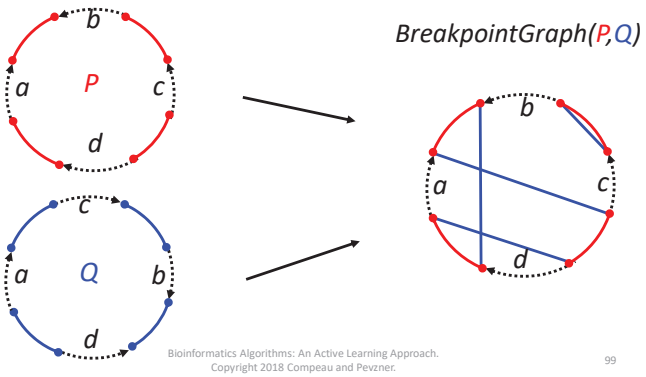
Superimposing *P* and *Q*



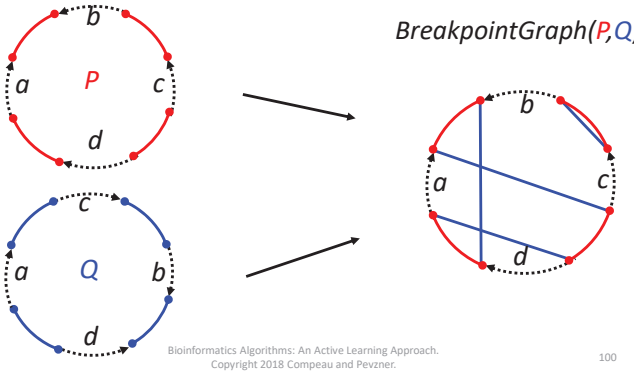
Red and Black Edges in Breakpoint Graph Form..

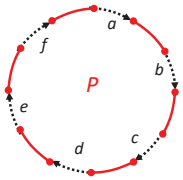


Blue and Black Edges in Breakpoint Graph Form...



What About Red and Blue Edges???

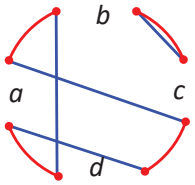




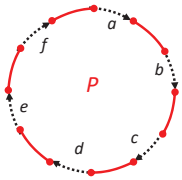
Alternating Red-Blue Cycles

Red and blue edges form **alternating red-blue cycles.**

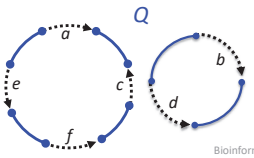
BreakpointGraph(P,Q)



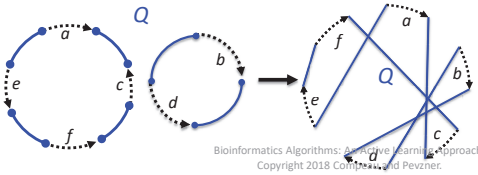
cycle(P,Q): number of red-blue alternating cycles

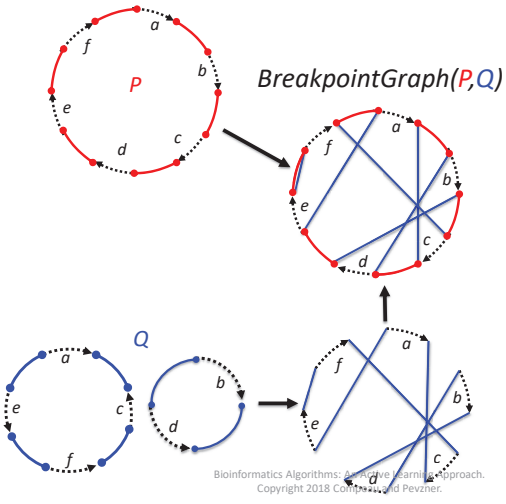


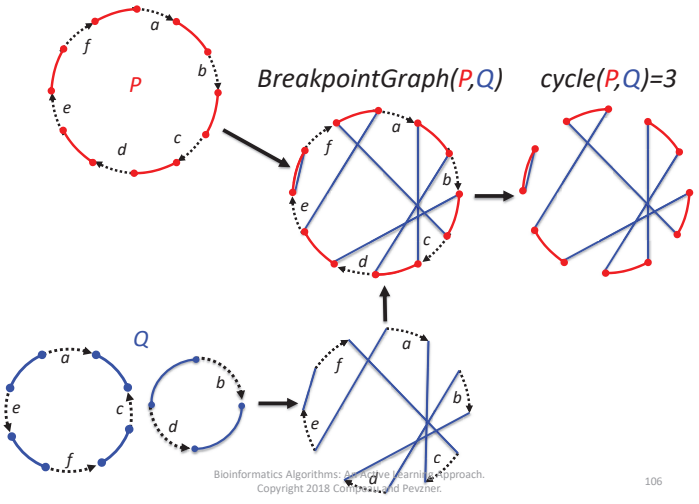
What is *cycle(P,Q)*?



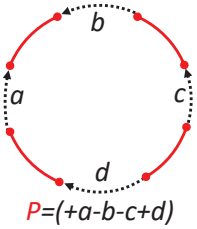
Arranging black edges of *Q* in the same order as black edges in *P*







Given *P*, What Genome *Q* Maximizes *cycle(P,Q)*?

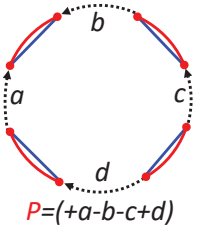


P=(+a-b-c+d)

Given *P*, What Genome *Q* Maximizes *cycle(P,Q)*?

BreakpointGraph(P,P)

Identity Breakpoint Graph



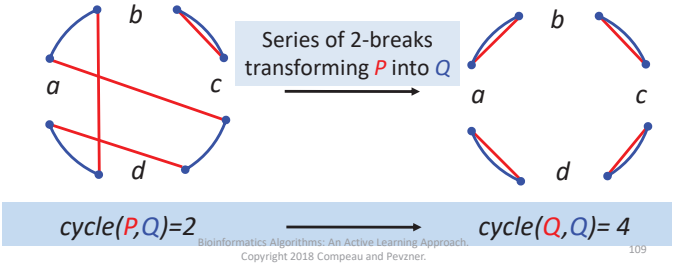
cycle(P,P)=#blocks=4

P=(+a-b-c+d)

Genome Rearrangements Affect Red-Blue Cycles

Each transformation $P \rightarrow Q$ corresponds to a transformation:

BreakpointGraph(P,Q) *BreakpointGraph(Q,Q)*

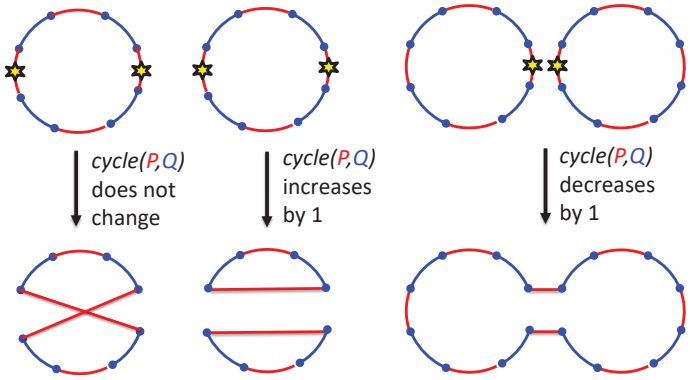


cycle(P,Q)=2 → *cycle(Q,Q)= 4*

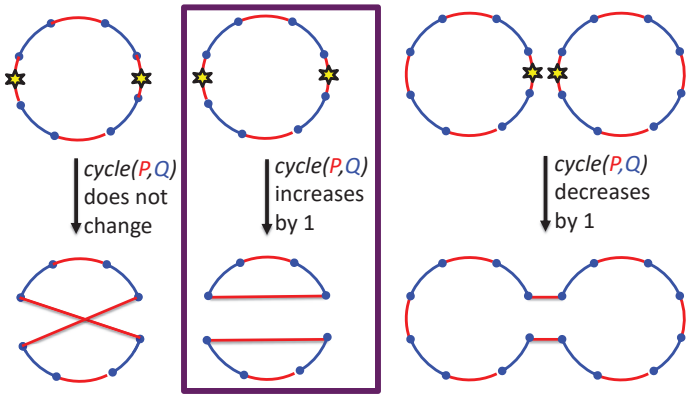
Are There Fragile Regions in the Human Genome?

- Transforming Men into Mice
- Sorting by Reversals
- Breakpoint Theorem
- Rearrangements in Tumor Genomes
- 2-Break Distance Problem
- Breakpoint Graphs
- **2-Break Distance Theorem**
- Rearrangement Hotspots in the Human Genome

A 2-Break May Change *cycle(P,Q)* by …

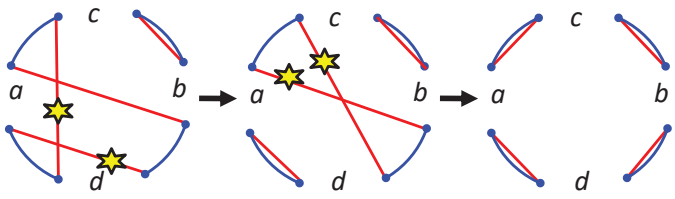


There Exists a 2-Break Increasing *cycle(P,Q)* by 1



Rearrangements Change *cycle(P,Q)*

*P=(+a –b –c **+d**)* → *P'=(+a **–b –c** -d)* → *P''=Q=(+a +c +b -d)*
BreakpointGraph(P,Q) → *BreakpointGraph(P',Q)* → *BreakpointGraph(Q,Q)*
cycle(P,Q)=2 → *cycle(P',Q)=3* → *cycle(Q,Q)=4*



Sorting by 2-Breaks

2-breaks
$P \rightarrow \ldots \rightarrow Q$

BreakpointGraph(P,Q) → … → *BreakpointGraph(Q,Q)*

cycle(P,Q) → … → *cycle(Q,Q)=blocks(Q,Q)*

# of red-blue cycles increases by ***blocks(P,Q) - cycle(P,Q)***

How much each 2-break can contribute to this increase?

Each 2-Break Increases #Cycles by at Most 1

A 2-break:

adds 2 new red edges and thus creates at most **2** new cycles (containing two new red edges)

removes 2 red edges and thus destroys at least **1** old cycle (containing two old edges)

change in the number of cycles ≤ **2-1=1**.

2-Break Distance Theorem

- A 2-break increases #cycles by at most 1.
- There exists a 2-break increasing #cycles by 1.
- Every sorting by 2-breaks must increase #cycles by *blocks(P,Q) - cycle(P,Q)*
- 2-break distance between genomes *P* and *Q*:

d(P,Q) = blocks(P,Q) - cycle(P,Q)

2-Break Distance between Human and Mouse Genomes

- *Human* and *Mouse* genomes can be decomposed into **280** synteny blocks (at least 0.5 million nucleotides in length)
- The breakpoint graph on these blocks has **35** cycles
- The 2-break distance between *Human* and *Mouse:*

$d(H,M) = blocks(H,M) - cycle(H,M) = \mathbf{280-35=245}$

- There are numerous **245**-step scenarios.
- The true scenario may have more than **245** steps.

Are There Fragile Regions in the Human Genome?

- Transforming Men into Mice
- Sorting by Reversals
- Breakpoint Theorem
- Rearrangements in Tumor Genomes
- 2-Break Distance Problem
- Breakpoint Graphs
- 2-Break Distance Theorem
- **Rearrangement Hotspots in the Human Genome**

Random Breakage Model of Chromosome Evolution

- *Susumu Ohno, 1973*: Rearrangements occur at random positions (no fragile regions).
- *Nadeau & Taylor, 1984:* the first statistical arguments in favor of the Random Breakage Model (RBM).

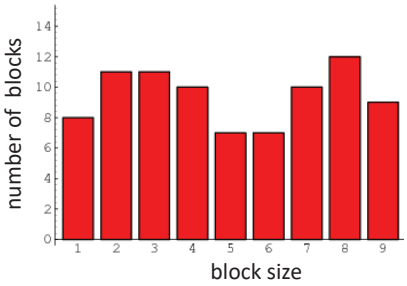
Does RBM Have Predictive Power?

Apply *N* **random reversals** to a chromosome consisting of *M* "genes."
Can we predict how many blocks of length *k* will be generated?

If we can predict it, will these predictions fit what we observe in real genomes?

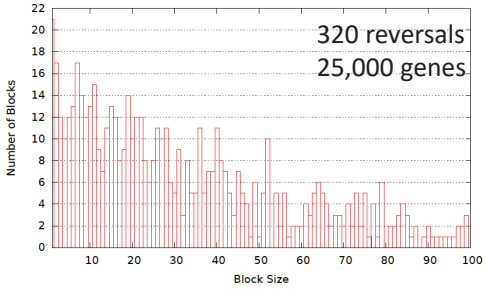
Does RBM Have Predictive Power?

Apply *N* **random reversals** to a chromosome consisting of *M* "genes."
Can we predict how many blocks of length *k* will be generated?



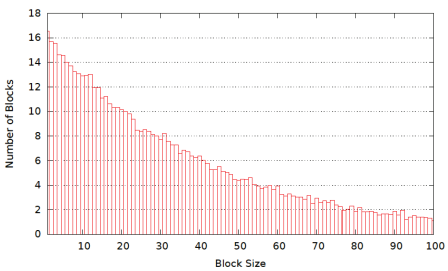
Does RBM Have Predictive Power?

Apply *N* **random reversals** to a chromosome consisting of *M* "genes."
Can we predict how many blocks of length *k* will be generated?



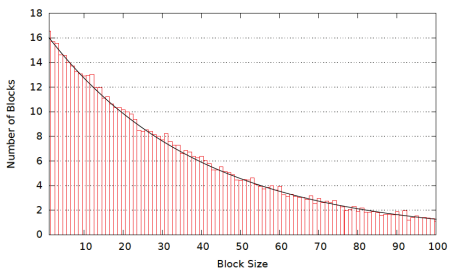
Despite the fact that reversals occur at random positions, we can predict (roughly) how many blocks of each length will be generated!

Histogram of Synteny Block Lengths (averaged over 100 simulations)



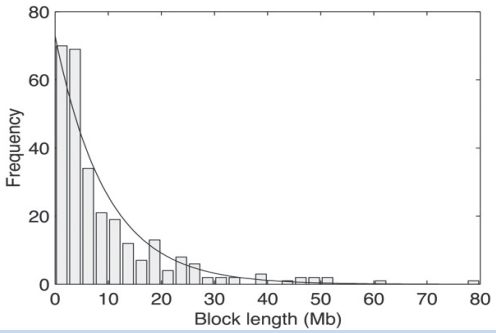
Exponential Distribution (simulated data)

$y=\lambda e^{-\lambda \cdot x}$



Lengths of Real Human-Mouse Synteny Blocks

$y=\lambda e^{-\lambda \cdot x}$



1990s: RBM was embraced by biologists and has become *de facto* theory of chromosome evolution.

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Are There Fragile Regions in the Human Genome?

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Are There Fragile Regions in the Human Genome?

Rearrangement Hotspots Theorem: Yes!

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Are There Fragile Regions in the Human Genome?

Rearrangement Hotspots Theorem: Yes!

Proof: *If the Random Breakage Model is correct*, then *N* rearrangements applied to circular chromosomes will produce approximately *2N* synteny blocks.

- Since there are 280 human-mouse synteny blocks, there must have been approximately 280/2 = **140** 2-breaks on the human-mouse evolutionary path.
- However, the 2-Break Distance Theorem implies that there are at least **245** 2-breaks on this path.

STOP and Think. Is **245 ≈ 140**?

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

A Contradiction!

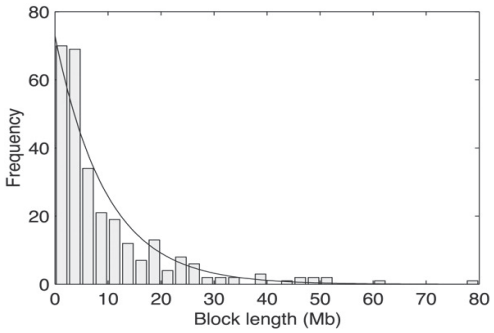
Since 245 is much larger than 140, we arrived at a contradiction implying that **one of our assumptions is incorrect! Which one?**

But the only assumption we made in this proof was:

"If the Random Breakage Model is correct…"

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

If RBM Is Wrong, How Would You Explain the Exponential Distribution?



Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Computational Tests vs. Biological Models

- Why have biologists embraced the Random Breakage Model?

Test

Computational Tests vs. Biological Models

- Why have biologists embraced the Random Breakage Model?
  - **A logical fallacy**: RBM is not the only model that complies with the "exponential distribution" test.

Model \ Test	Exponential distribution
RBM	YES

Computational Tests vs. Biological Models

- Why have biologists embraced the Random Breakage Model?
  - A logical fallacy: RBM is not the only model that complies with the "exponential distribution" test.
- Why was RBM refuted?
  - It does not comply with the observed "breakpoint reuse."

Model \ Test	Exponential distribution	Breakpoint reuse
RBM	YES	NO

A New Model Needed

- Why have biologists embraced the Random Breakage Model?
  - A logical fallacy: RBM is not the only model that complies with the "exponential distribution" test.
- Why was RBM refuted?
  - It does not comply with the "breakpoint reuse" observed in genomes.
- **Is there a model that complies with both the "exponential distribution" and the "breakpoint reuse" tests?**

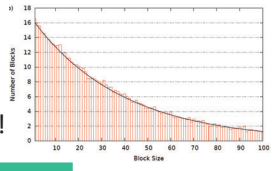
Model \ Test	Exponential distribution	Breakpoint reuse
RBM	YES	NO
???	YES	YES

Fragile Breakage Model

- Genome is a mosaic of:
  - *fragile regions* with high propensity for rearrangements and
  - *solid regions* with low propensity for rearrangements.
- Fragile regions (regions between consecutive synteny blocks) are small, accounting for less than ~ 5% of genome.

Does FBM Explain BOTH Exponential Distribution and Rearrangement Hotspots?

- A small number of short fragile regions explain rearrangement hotspots.
- If the fragile regions are somewhat randomly distributed throughout the genome, the synteny blocks follow the exponential distribution!

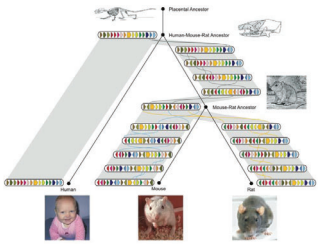


Model \ Test	Exponential distribution	Breakpoint reuse
RBM	YES	NO
FBM	YES	YES

But Is There a Test that **BOTH** RBM and FBM Fail?

Model \ Test	Exponential distribution	Breakpoint reuse	A New Test?
RBM	YES	NO	NO
FBM	YES	YES	NO

Information About Multiple Genomes Enables a New Test



Multiple Breakpoint Reuse Test: analysis of breakpoints across multiple genomes

Model \ Test	Exponential distribution	Breakpoint reuse	A New Test?
RBM	YES	NO	NO
FBM	YES	YES	NO

Birth and Death of Fragile Regions

- Recent studies revealed evidence for the "***birth and death***" of the fragile regions, implying that they move to different locations in different lineages.
- This discovery resulted in the ***Turnover Fragile Breakage Model (TFBM)*** that complies with a new Multiple Breakpoint Reuse (**MBR**) Test.
- TFBM points to locations of the ***currently*** fragile regions.

Birth and Death of Fragile Regions

- Recent studies revealed evidence for the "***birth and death***" of the fragile regions, implying that they move to different locations in different lineages.
- This discovery resulted in the ***Turnover Fragile Breakage Model (TFBM)*** that complies with a new Multiple Breakpoint Reuse (**MBR**) Test.
- TFBM points to locations of the ***currently*** fragile regions.

Model \ Test	Exponential distribution	Breakpoint reuse	MBR
RBM	YES	NO	NO
FBM	YES	YES	NO
TFBM	YES	YES	YES

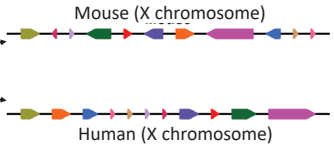
Are There Fragile Regions in the Human Genome?

- Transforming Men into Mice
- Sorting by Reversals
- Breakpoint Theorem
- Rearrangements in Tumor Genomes
- 2-Break Distance Problem
- Breakpoint Graphs
- 2-Break Distance Theorem
- Rearrangement Hotspots in the Human Genome
- **Synteny Block Construction**

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

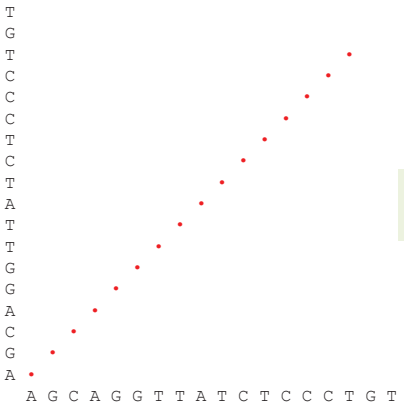
How Do We Construct the Synteny Blocks?

Unknown ancestor
~ 75 million years ago



Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

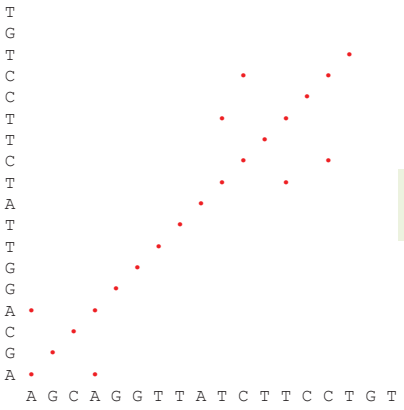
Comparing Genome with Itself (in 2-D)



Identical *k*-mers (*k*=3)

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

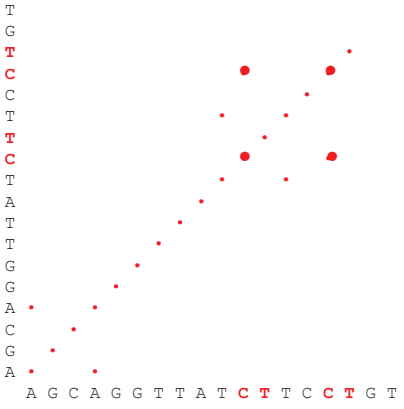
From Identical 3-mers to Identical 2-mers



Identical *k*-mers (*k*=**2**)

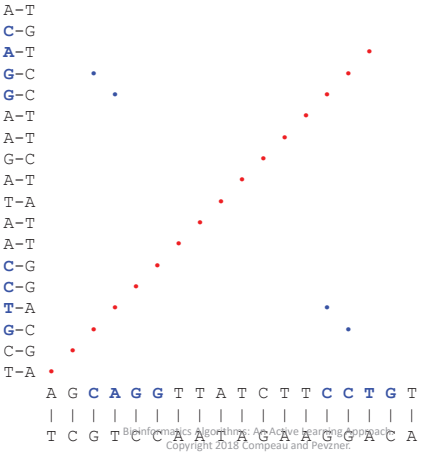
Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

From Identical 3-mers to Identical 2-mers



Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

We Forgot about the Complementary Strand

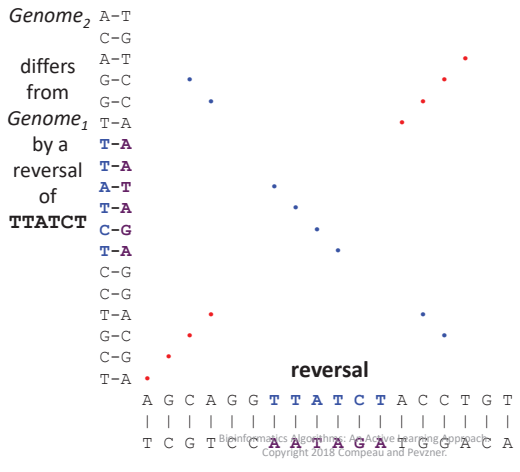


Identical and reverse complementary *k*-mers (*k*=3).

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Comparing Different Genomes

Genome$_2$ differs from *Genome*$_1$ by a reversal of **TTATCT**



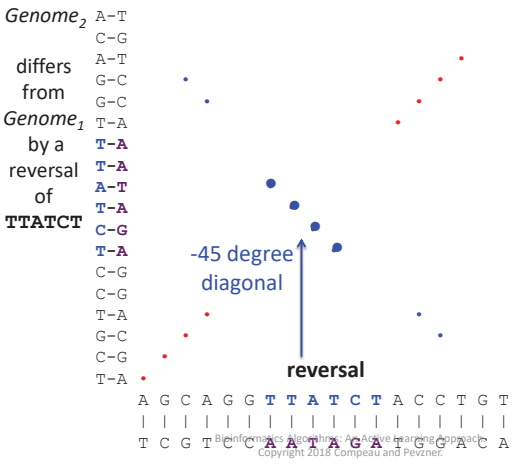
Genome$_1$

Identical and reverse complementary *k*-mers (*k*=3).

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Comparing Different Genomes

Genome$_2$ differs from *Genome*$_1$ by a reversal of **TTATCT**

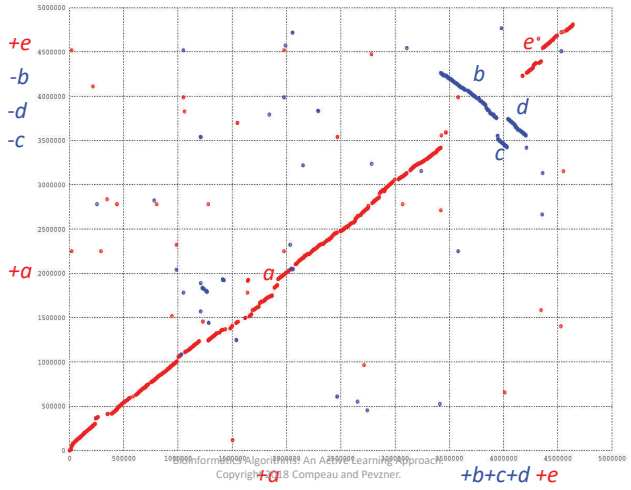


Genome$_1$

Identical and reverse complementary *k*-mers (*k*=3).

Bioinformatics Algorithms: An Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

E. Coli versus *S. enterica*



Bioinformatics Algorithms: An Active Learning Approach
Copyright 2015 Compeau and Pevzner.